HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO TIỂU LUẬN MÔN HỌC

ĐỀ TÀI: NGHIÊN CỨU VÀ TRIỂN KHAI CÁC MÔ HÌNH HỌC MÁY CƠ BẢN, MẠNG NƠ-RON TÍCH CHẬP (CNN) VÀ MẠNG NƠ-RON HỒI QUY (RNN) TRONG HỆ THỐNG THÔNG MINH

Lớp:	D23CTPM01
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 10/2026

Mục lục

Danh mục Hình vẽ iv

Danh mục Bảng biểu vi

Danh mục Đoạn mã nguồn vii

Phần Mở Đầu 1

1 Lịch Sử Phát Triển Trí Tuệ Nhân Tạo 3
- 1.1 Tổng quan bức tranh hơn tám thập kỷ của AI . . . 3
- 1.2 Khởi nguồn và kỷ nguyên logic biểu tượng (1940s – 1960s) . . . 3
  - 1.2.1 Ý niệm về máy tính có tư duy và phép thử Turing . . . 3
  - 1.2.2 Mô hình nơ-ron hình thức và Hội thảo Dartmouth (1956) . . . 4
  - 1.2.3 Perceptron của Rosenblatt và các hệ suy diễn ký hiệu đầu tiên . . . 4
- 1.3 "Mùa đông AI" lần thứ nhất (1970s) . . . 4
  - 1.3.1 Giới hạn toán học của Perceptron và cuốn sách của Minsky & Papert (1969) . . . 4
  - 1.3.2 Báo cáo Lighthill (1973) và sự cắt giảm tài trợ của DARPA . . . 5
- 1.4 Sự trỗi dậy của hệ chuyên gia và "mùa đông AI" lần thứ hai (1980s – 1990s) . . . 5
  - 1.4.1 Kỷ nguyên vàng của hệ chuyên gia . . . 5
  - 1.4.2 Sự tái sinh của mạng nơ-ron: thuật toán lan truyền ngược (1986) . . . 5
  - 1.4.3 "Mùa đông AI" lần thứ hai (1987 – 1993) . . . 5
- 1.5 Kỷ nguyên học máy thống kê (1990s – 2010) . . . 6
  - 1.5.1 Lý thuyết học thống kê và máy vector hỗ trợ (SVM) . . . 6
  - 1.5.2 Cây quyết định và các phương pháp học tập hợp . . . 6
  - 1.5.3 LeNet-5 (1998): nền tảng của mạng tích chập hiện đại . . . 6
- 1.6 Cuộc cách mạng học sâu (2012 – 2020) . . . 6
  - 1.6.1 Bước ngoặt ImageNet 2012 và AlexNet . . . 7
  - 1.6.2 Sự tiến hóa của các kiến trúc thị giác: VGG, ResNet, MobileNet . . . 7
  - 1.6.3 Mô hình hóa chuỗi: từ RNN, LSTM đến Transformer . . . 7
- 1.7 Kỷ nguyên AI tạo sinh, mô hình nền tảng và hệ đa tác tử (2020 – nay) . . . 7
  - 1.7.1 Mô hình ngôn ngữ lớn và định luật tỷ lệ . . . 7
  - 1.7.2 Giới hạn của mô hình thuần tham số và vai trò của đồ thị tri thức, RAG . . . 8
  - 1.7.3 Xu hướng: hệ đa tác tử tự trị . . . 8

2 Các Kỹ Thuật Học Máy Cơ Bản 9
- 2.1 Cơ sở lý thuyết các thuật toán học máy cốt lõi . . . 9
  - 2.1.1 Hồi quy tuyến tính (Linear Regression) . . . 9
  - 2.1.2 Hồi quy Logistic (Logistic Regression) . . . 9
  - 2.1.3 Cây quyết định và Rừng ngẫu nhiên (Random Forest) . . . 9
  - 2.1.4 Máy vector hỗ trợ (Support Vector Machine) . . . 10
- 2.2 Khảo sát thực nghiệm và tiền xử lý ba bộ dữ liệu . . . 10
  - 2.2.1 Tập dữ liệu 1: Sàng lọc nguy cơ tiểu đường (100.000 mẫu) . . . 10
  - 2.2.2 Tập dữ liệu 2: Định giá bất động sản Việt Nam (30.229 mẫu) . . . 12
  - 2.2.3 Tập dữ liệu 3: Phân loại cảm xúc nhận xét thương mại điện tử (22.641 mẫu) . . . 14
- 2.3 Hiện thực hóa và thực nghiệm các mô hình học máy cơ bản . . . 16

2.3.1 Pipeline hồi quy tuyến tính và Ridge . . . 16
2.3.2 Pipeline hồi quy Logistic cho dữ liệu y tế . . . 17
2.3.3 Pipeline Rừng ngẫu nhiên . . . 17
2.3.4 Pipeline Máy vector hỗ trợ . . . 17
2.3.5 Kết quả thực nghiệm các mô hình học máy cơ bản . . . 18
2.3.6 Phân tích kết quả và giới hạn của các mô hình cơ bản . . . 19
2.4 Mạng nơ-ron đa tầng (MLP) trên ba nền tảng: NumPy, Keras, PyTorch . . . 19
2.4.1 Cơ sở lý thuyết và kiến trúc mạng . . . 19
2.4.2 Hiện thực từ đầu bằng NumPy . . . 20
2.4.3 Hiện thực bằng Keras / TensorFlow . . . 21
2.4.4 Hiện thực bằng PyTorch . . . 21
2.5 Đánh giá và đối chuẩn trên ba tập dữ liệu . . . 22
2.5.1 Đối chuẩn hiệu năng chi tiết trên từng tập dữ liệu . . . 22
2.6 Đóng gói triển khai và giao diện Web thực tế . . . 26
2.6.1 Kiến trúc triển khai hệ thống . . . 26
2.6.2 Giao diện thực tế các hệ thống thông minh . . . 27

3 Mạng Nơ-ron Tích Chập (CNN) 29
3.1 Cơ sở lý thuyết mạng nơ-ron tích chập và biểu diễn không gian . . . 29
3.1.1 Toán học của phép tích chập hai chiều . . . 29
3.1.2 Vùng cảm nhận và phép gộp (pooling) . . . 30
3.1.3 Tiến hóa kiến trúc: VGG, ResNet và MobileNet . . . 31
3.2 Khảo sát thực nghiệm và tiền xử lý ba bộ dữ liệu đa miền . . . 32
3.2.1 Tập dữ liệu 1: nhận dạng chữ số nhà SVHN . . . 32
3.2.2 Tập dữ liệu 2: phân loại ảnh vệ tinh EuroSAT . . . 33
3.2.3 Tập dữ liệu 3: chuỗi đặc trưng lâm sàng cho 1D CNN . . . 34
3.3 Hiện thực hóa mã nguồn trên ba nền tảng . . . 34
3.3.1 Tích chập 2D và Max Pooling viết tay bằng NumPy . . . 34
3.3.2 Custom CNN với khối kiểu VGG bằng Keras / TensorFlow . . . 35
3.3.3 Khối Inverted Residual kiểu MobileNetV2 bằng PyTorch . . . 36
3.3.4 Kiến trúc ResNet-18 bằng PyTorch . . . 37
3.3.5 Quy trình huấn luyện trên GPU . . . 37
3.4 Đánh giá và đối chuẩn các biến thể kiến trúc CNN . . . 38
3.5 Đóng gói triển khai hệ thống Web App thị giác máy tính . . . 41

4 Mạng Nơ-ron Hồi Quy (RNN) 43
4.1 Cơ sở lý thuyết mạng nơ-ron hồi quy và mô hình hóa chuỗi . . . 43
4.1.1 Kiến trúc Simple RNN và đồ thị tính toán mở rộng theo thời gian . . . 43
4.1.2 Lan truyền ngược qua thời gian (BPTT) và hiện tượng triệt tiêu đạo hàm . . . 44
4.1.3 Kỹ thuật cắt tỉa đạo hàm (Gradient Clipping) . . . 44
4.1.4 Kiến trúc tế bào LSTM và GRU . . . 45
4.1.5 Mạng hồi quy hai chiều (BiLSTM) và Recurrent Dropout . . . 46
4.2 Khảo sát thực nghiệm và tiền xử lý hai bộ dữ liệu chuỗi . . . 47
4.2.1 Tập dữ liệu 1: chuỗi giá chứng khoán Việt Nam (VN30, 2012-01-03 – 2026-09-30) . . . 47
4.2.2 Tập dữ liệu 2: quỹ đạo hành vi người dùng KKBox . . . 49
4.3 Hiện thực hóa mã nguồn trên ba nền tảng . . . 50
4.3.1 Tế bào LSTM viết tay bằng NumPy . . . 50

4.3.2 Hiện thực bằng Keras / TensorFlow 51
4.3.3 Hiện thực bằng PyTorch 52
4.4 Đánh giá và đối chuẩn các kiến trúc RNN trên hai tập dữ liệu 52
4.4.1 Dự báo chuỗi thời gian cổ phiếu FPT (hồi quy) 53
4.4.2 Phân loại chuỗi hành vi KKBox 53
4.5 Đóng gói triển khai Web App tương tác thời gian thực 55

Kết Luận **57**

Tài liệu tham khảo **58**

Danh mục Hình vẽ

1.1 Dòng thời gian tiến hóa của trí tuệ nhân tạo (1943 – 2026): các mốc then chốt và các kỷ nguyên công nghệ. Hai dải xám là hai mùa đông AI (1974 – 1980 và 1987 – 1993), dải cam giữa chúng là thời kỳ hệ chuyên gia (1980 – 1987). . . . 3
1.2 Chu kỳ kỳ vọng và hai thời kỳ "mùa đông AI" (minh họa định tính, không phải số đo). . . . 6
1.3 Quan hệ bao hàm giữa trí tuệ nhân tạo, học máy, học sâu và AI tạo sinh / hệ đa tác tử. . . . 7

2.1 Phân bố nhãn của tập tiểu đường: lớp dương chỉ chiếm 8,50 %. . . . 11
2.2 Mật độ xác suất của đường huyết, HbA1c, BMI và tuổi theo tình trạng bệnh. . . . 11
2.3 Ma trận tương quan Pearson giữa các chỉ số lâm sàng và nhãn tiểu đường. . . . 12
2.4 Phân bố giá bán, diện tích gốc và diện tích sau biến đổi $\log(1 + x)$. . . . 13
2.5 Trái: giá theo diện tích (thang log), tô màu theo độ rộng đường vào. Phải: giá theo sáu tỉnh/thành có nhiều tin nhất. . . . 13
2.6 Ma trận tương quan giữa các thuộc tính vật lý và giá bán. . . . 14
2.7 Phân bố số sao và nhãn cảm xúc: lớp tích cực chiếm 77,06 %. . . . 15
2.8 Phân bố độ dài nhận xét (số từ) theo nhãn; nét đứt là trung bình 60,20 từ. . . . 15
2.9 Mười lăm từ đặc trưng nhất của nhận xét tích cực và tiêu cực, xếp theo tỉ số log-odds so với phần còn lại. . . . 16
2.10 Kiến trúc MLP dùng chung cho cả ba nền tảng và ba bài toán. . . . 20
2.11 Đường cong mất mát huấn luyện (nét đứt) và kiểm định (nét liền) của MLP trên ba nền tảng. . . . 24
2.12 Giá thực tế và giá dự báo trên tập kiểm thử của Random Forest và MLP PyTorch. . . . 24
2.13 So sánh đa tiêu chí trên tập tiểu đường. . . . 24
2.14 So sánh sai số (MAE, RMSE) và hệ số xác định R^2 trên tập giá nhà. . . . 25
2.15 So sánh Accuracy và các chỉ số macro trên tập nhận xét khách hàng. . . . 25
2.16 Ma trận nhầm lẫn trên tập kiểm thử tiểu đường: Logistic Regression, Random Forest, MLP PyTorch. 25
2.17 Ma trận nhầm lẫn trên tập kiểm thử nhận xét khách hàng. . . . 26
2.18 Kiến trúc ba tầng của Web App phục vụ sáu phân hệ thông minh. . . . 27
2.19 Phân hệ sàng lọc nguy cơ tiểu đường: người dùng nhập chỉ số lâm sàng, hệ thống trả xác suất của MLP và Random Forest, mức nguy cơ và khuyến nghị. . . . 27
2.20 Phân hệ định giá bất động sản: giá ước tính của Random Forest, khoảng hợp lý $\pm$ RMSE kiểm thử, dự báo của MLP và đơn giá theo m². . . . 28
2.21 Phân hệ phân tích cảm xúc nhận xét: xác suất ba lớp của MLP, đối chiếu Logistic Regression và các từ đóng góp nhiều nhất. . . . 28

3.1 Biểu diễn ảnh số: từ ma trận điểm ảnh đến tensor ba kênh và lô ảnh 4D trong học sâu. . . . 29
3.2 Minh họa số học từng bước của phép tích chập 2D: ảnh 5×5, bộ lọc cạnh dọc 3×3, bước 1. . . . 30
3.3 Cơ chế tích lũy vùng cảm nhận khi xếp chồng ba tầng tích chập 3×3 bước 1. . . . 30
3.4 So sánh max pooling, average pooling (2×2, bước 2) và Global Average Pooling trên cùng một bản đồ đặc trưng. . . . 31
3.5 Sơ đồ tiến hóa khối xây dựng CNN: khối VGG phẳng, khối Residual và khối Inverted Residual (tích chập tách biệt chiều sâu). . . . 31
3.6 Khối VGG: hai tầng 3×3 có cùng vùng nhìn 5×5 với một tầng 5×5 nhưng ít trọng số hơn. . . . 31
3.7 Khối Residual và cơ chế "đường cao tốc" của dòng gradient. . . . 32

3.8 Tích chập chuẩn so với tích chập tách biệt chiều sâu: lọc không gian riêng từng kênh rồi trộn kênh bằng 1×1. 32
3.9 Bốn ảnh mẫu mỗi lớp của SVHN: đa dạng phông chữ, ánh sáng, màu nền và có chữ số lân cận lọt vào khung. 33
3.10 Trái: số ảnh theo lớp của SVHN. Phải: phân bố cường độ điểm ảnh trên ba kênh màu. 33
3.11 Bốn ảnh mẫu mỗi lớp của EuroSAT. 34
3.12 Trái: số ảnh theo lớp của EuroSAT. Phải: phân bố cường độ điểm ảnh trên ba kênh màu. 34
3.13 Mất mát huấn luyện và độ chính xác kiểm thử theo epoch của các kiến trúc trên SVHN. 39
3.14 Ma trận nhầm lẫn (% theo hàng) của ResNet-18 trên tập kiểm thử SVHN. 39
3.15 Mất mát huấn luyện và độ chính xác kiểm thử theo epoch của các kiến trúc trên EuroSAT. 39
3.16 Ma trận nhầm lẫn (% theo hàng) của ResNet-18 trên tập kiểm thử EuroSAT. 40
3.17 Độ trễ suy luận (GPU và CPU, lô 1) và thông lượng GPU (lô 256) của bốn kiến trúc. 40
3.18 Phân hệ Vision AI: ảnh vệ tinh EuroSAT, bản đồ CAM của lớp dự đoán và xác suất năm lớp hàng đầu của ResNet-18. 42
3.19 Phân hệ Vision AI với ảnh số nhà SVHN và mô hình MobileNetV2. 42

4.1 Tensor chuỗi ba chiều $[B, T, D]$ dùng trong mô hình hóa học sâu tuần tự. 43
4.2 Đồ thị tính toán của RNN: dạng cuộn và dạng mở rộng theo thời gian với trọng số dùng chung. . . 43
4.3 Bản chất toán học của triệt tiêu và bùng nổ đạo hàm: chuẩn tích chuỗi Jacobian theo số bước lùi. . 44
4.4 Hình học của cắt tỉa đạo hàm: vector $\mathbf{g}$ được co về hình cầu bán kính θ, giữ nguyên hướng. 45
4.5 Kiến trúc vi mô của tế bào LSTM: đường cộng của ô nhớ và ba cổng điều khiển. 45
4.6 Kiến trúc vi mô của tế bào GRU với cổng đặt lại r_t và cổng cập nhật z_t. 46
4.7 Mạng hồi quy hai chiều BiLSTM tổng hợp ngữ cảnh theo cả hai chiều thời gian. 46
4.8 Dropout thông thường đổi mặt nạ ở mỗi bước; recurrent dropout giữ một mặt nạ cố định cho cả chuỗi. 47
4.9 Giá đóng cửa (nghìn đồng), đường SMA50 và khối lượng giao dịch (cột mờ) của bốn mã VN30. . 48
4.10 Phân phối lợi suất ngày, độ biến động 30 phiên và ma trận tương quan lợi suất giữa bốn mã. . . . 48
4.11 ACF của giá đóng cửa và ACF, PACF của lợi suất ngày FPT (vùng tô là khoảng tin cậy 95 %). . . 48
4.12 Cơ chế tạo mẫu cửa sổ trượt 30 phiên và chia theo thời gian chống rò rỉ thông tin tương lai. 49
4.13 Quỹ đạo trung bình theo ngày của số giây nghe, số bài khác nhau và tỉ lệ ngày hoạt động: nhóm rời bỏ đầu tháng nghe nhiều ngang hoặc hơn nhóm ở lại, rồi giảm dần về cuối tháng. 50
4.14 Tỉ lệ rời bỏ (8,96 %) và ma trận tương quan giữa các đặc trưng hành vi (trung bình 31 ngày) với nhãn. 50
4.15 Đường cong mất mát huấn luyện (nét liền) và kiểm định (nét đứt) của bốn kiến trúc trên hai tập dữ liệu. 54
4.16 Giá thực tế và dự báo trên tập kiểm thử FPT, kèm mốc "giữ nguyên giá"; bên phải phóng to 60 phiên cuối. 54
4.17 Ma trận nhầm lẫn của LSTM và BiLSTM trên tập kiểm thử KKBox. 54
4.18 So sánh PyTorch và Keras: RMSE trên FPT (đường chấm là mốc giữ nguyên giá), F1 trên KKBox, số tham số và độ trễ CPU. 55
4.19 Phân hệ dự báo giá cổ phiếu FPT: đường giá thực tế và dự báo của LSTM, dự báo phiên kế tiếp, so sánh với mốc giữ nguyên giá và các chỉ báo hiện tại. 56
4.20 Phân hệ dự đoán khách hàng rời bỏ: quỹ đạo nghe nhạc 31 ngày và xác suất rời bỏ của bốn kiến trúc. 56

Danh mục Bảng biểu

1.1 Bảng chân lý của hàm XOR: hai lớp nằm trên hai đường chéo nên không đường thẳng nào tách được. 4

2.1 Năm bản ghi ngẫu nhiên của tập dữ liệu tiểu đường. 11
2.2 Năm tin đăng ngẫu nhiên của tập dữ liệu giá nhà. 13
2.3 Năm nhận xét tiêu biểu kèm nhãn cảm xúc. 15
2.4 Kết quả các mô hình học máy cơ bản trên tập tiểu đường (100.000 hồ sơ, kiểm thử 20 %). 18
2.5 Kết quả các mô hình học máy cơ bản trên tập giá nhà Việt Nam. 18
2.6 Kết quả các mô hình học máy cơ bản trên tập nhận xét khách hàng (TF-IDF 5.000 chiều). 18
2.7 Đối chuẩn mọi mô hình trên tập tiểu đường. 22
2.8 Đối chuẩn mọi mô hình trên tập giá nhà Việt Nam. 23
2.9 Đối chuẩn mọi mô hình trên tập nhận xét khách hàng. 23
2.10 Cùng một kiến trúc MLP trên ba nền tảng. 23

3.1 Đối chuẩn đa chiều các biến thể CNN huấn luyện từ đầu trên SVHN, EuroSAT và 1D CNN trên dữ liệu tiểu đường. 38

4.1 Năm phiên giao dịch tiêu biểu của FPT kèm chỉ báo kỹ thuật. 47
4.2 Đối chuẩn dự báo giá đóng cửa FPT trên tập kiểm thử (2023-11-17 – 2026-09-30). 53
4.3 Đối chuẩn phân loại người dùng rời bỏ trên tập kiểm thử KKBox. 53
4.4 Đối chuẩn PyTorch và Keras với cùng cấu hình mạng. 53

Danh mục Đoạn mã nguồn

2.1 Bộ tiền xử lý dữ liệu giá nhà: điền thiếu kèm cờ, chuẩn hóa, one-hot có gộp nhóm hiếm 16
2.2 Pipeline hồi quy tuyến tính và Ridge với scikit-learn 17
2.3 Pipeline hồi quy Logistic có cân bằng trọng số lớp 17
2.4 Pipeline Random Forest cho cả hồi quy và phân loại 17
2.5 Pipeline SVM: SVC/SVR nhân RBF cho dữ liệu bảng, LinearSVC cho văn bản 18
2.6 Lan truyền tiến của MLP viết tay bằng NumPy, kèm Dropout ngược 20
2.7 Lan truyền ngược của MLP viết tay: một sai số tầng ra chung cho ba bài toán 21
2.8 Bộ tối ưu Adam viết tay 21
2.9 MLP bằng Keras Sequential API cho ba loại đầu ra 21
2.10 MLP bằng PyTorch với lớp `nn.Module` 22
2.11 Một epoch huấn luyện PyTorch trên dữ liệu đã nằm sẵn trên GPU 22
2.12 Endpoint sàng lọc tiểu đường: MLP PyTorch và Random Forest cho cùng một hồ sơ 27
3.1 Tích chập 2D nhiều kênh bằng vòng lặp, bám sát công thức (3.1) 35
3.2 Tích chập 2D kiểu im2col bằng NumPy 35
3.3 Max Pooling bằng NumPy 35
3.4 Khối tích chập kiểu VGG bằng Keras 36
3.5 Custom CNN bằng Keras Functional API 36
3.6 Khối Inverted Residual: mở rộng 1×1, depthwise 3×3, chiếu tuyến tính 1×1 36
3.7 Khối Residual cơ bản với kết nối tắt chiếu 1×1 khi đổi kích thước 37
3.8 Kiến trúc ResNet-18: thân 3×3, bốn tầng $[2, 2, 2, 2]$ khối, GAP và FC 37
3.9 Tăng cường dữ liệu trên GPU: dịch ngẫu nhiên từng ảnh bằng chỉ số nâng cao, lật và chuyển vị cho ảnh vệ tinh 38
3.10 Tính bản đồ kích hoạt lớp CAM từ bản đồ đặc trưng cuối và trọng số tầng FC 41
4.1 Tế bào LSTM và lan truyền tiến qua chuỗi viết tay bằng NumPy 51
4.2 Mô hình hồi quy many-to-one bằng Keras cho cả hồi quy và phân loại 51
4.3 Mô hình hồi quy tổng quát bằng PyTorch: Simple RNN, LSTM, GRU, BiLSTM 52
4.4 Một epoch huấn luyện PyTorch có cắt tỉa chuẩn gradient 52
4.5 Endpoint dự đoán rời bỏ: chạy bốn mô hình hồi quy trên cùng một chuỗi 31 ngày 56

Phần Mở Đầu

1. Bối cảnh và tính cấp thiết của đề tài

Trí tuệ nhân tạo (Artificial Intelligence – AI) và học máy (Machine Learning – ML) đã rời phòng thí nghiệm để trở thành một thành phần thường trực của phần mềm nghiệp vụ: hệ thống sàng lọc bệnh, công cụ định giá tài sản, bộ lọc phản hồi khách hàng, ứng dụng nhận dạng ảnh, mô hình dự báo tài chính. Điểm chung của các hệ thống này là chúng không được lập trình bằng luật viết tay mà *học từ dữ liệu*. Sự chuyển dịch đó gắn với ba điều kiện xuất hiện cùng lúc trong thập niên 2010: dữ liệu lớn, phần cứng tính toán song song (GPU) và các thuật toán tối ưu cho mạng nhiều tầng [1].

Một kỹ sư phát triển hệ thống thông minh hiếm khi chỉ gặp một dạng dữ liệu. Trong thực tế có ít nhất ba hình thái, mỗi hình thái hợp với một họ mô hình riêng:

- **Dữ liệu bảng** (y tế, bất động sản, tài chính doanh nghiệp): mỗi dòng là một đối tượng độc lập với các thuộc tính số và phân loại. Đây là địa hạt của các thuật toán học máy cổ điển (hồi quy tuyến tính, Logistic Regression, cây quyết định, Random Forest, SVM) và của mạng nơ-ron đa tầng (MLP).
- **Dữ liệu không gian** (ảnh): các điểm ảnh kề nhau tương quan chặt, và cùng một mẫu hình có thể xuất hiện ở bất kỳ vị trí nào. Mạng nơ-ron tích chập (CNN) khai thác đúng hai tính chất này nhờ chia sẻ trọng số và vùng tiếp nhận cục bộ.
- **Dữ liệu chuỗi** (giá chứng khoán, nhật ký hành vi người dùng): thứ tự thời gian mang thông tin, các mẫu không còn độc lập. Mạng nơ-ron hồi quy (RNN) và các biến thể có cổng (LSTM, GRU, BiLSTM) mang một trạng thái ẩn đi qua từng bước thời gian để ghi nhớ ngữ cảnh.

Từ yêu cầu đó, tiểu luận *"Nghiên cứu và triển khai các mô hình học máy cơ bản, mạng nơ-ron tích chập (CNN) và mạng nơ-ron hồi quy (RNN) trong hệ thống thông minh"* đặt mục tiêu đi trọn một vòng từ lý thuyết đến sản phẩm cho cả ba hình thái dữ liệu: hiểu toán học của mô hình, tự cài đặt lại nó, huấn luyện trên dữ liệu thật, đo đạc khách quan rồi đóng gói thành ứng dụng web chạy được.

2. Mục tiêu và nhiệm vụ

Mục tiêu tổng quát là xây dựng nền tảng kiến thức vững cùng năng lực thực hành trên toàn bộ tiến trình từ học máy cổ điển đến các kiến trúc học sâu xử lý không gian và thời gian. Sáu nhiệm vụ cụ thể:

Nhiệm vụ 1 – Khảo cứu lịch sử AI.
Hệ thống hóa các mốc phát triển từ thập niên 1940 đến nay, phân tích nguyên nhân hai "mùa đông AI", sự chuyển dịch sang học máy thống kê, cuộc cách mạng học sâu và xu hướng hiện tại (mô hình nền tảng, RAG, hệ đa tác tử).

Nhiệm vụ 2 – Cơ sở lý thuyết.
Trình bày hàm mất mát và cách tối ưu của từng họ mô hình: lan truyền ngược trong MLP, phép tích chập và pooling trong CNN, lan truyền ngược qua thời gian và cơ chế cổng trong RNN.

Nhiệm vụ 3 – Khảo sát dữ liệu thật.
Phân tích khám phá (EDA) và tiền xử lý **bảy** tập dữ liệu thuộc năm lĩnh vực: y tế (100.000 hồ sơ tiểu đường), bất động sản Việt Nam (30.229 tin đăng), thương mại điện tử (22.641 nhận xét khách hàng), thị giác máy tính (SVHN 73.257 + 26.032 ảnh, EuroSAT 21.600 + 5.400 ảnh vệ tinh), tài chính (14.338 phiên giao dịch của bốn mã VN30) và hành vi người dùng (100.000 chuỗi nghe nhạc của KKBox).

Nhiệm vụ 4 – Hiện thực trên ba nền tảng.
Cài đặt mô hình bằng **NumPy thuần** (tự viết lan truyền tiến, lan truyền ngược, tích chập, tế bào LSTM), bằng **Keras/TensorFlow** (API cấp cao) và bằng **PyTorch** (lớp `nn.Module`, vòng huấn luyện tự viết, chạy GPU).

Nhiệm vụ 5 – Đánh giá và đối chuẩn.
So sánh các mô hình trên cùng một cách chia dữ liệu theo độ chính xác (Accuracy, Precision, Recall, F1, ROC-AUC, MAE, RMSE, R^2), số tham số, dung lượng tệp, thời gian huấn luyện và độ trễ suy luận đo trong cùng điều kiện phần cứng.

Nhiệm vụ 6 – Triển khai.
Đóng gói các mô hình tốt nhất thành một ứng dụng web Flask gồm sáu phân hệ, nhận dữ liệu người dùng và trả kết quả dự báo theo thời gian thực.

3. Đối tượng và phương pháp nghiên cứu

Đối tượng. Các mô hình học máy cơ bản (Linear/Ridge Regression, Logistic Regression, Random Forest, SVM, MLP); các kiến trúc CNN (Custom CNN, VGG-11, MobileNetV2, ResNet-18 và 1D CNN); các kiến trúc RNN (Simple RNN, LSTM, GRU, BiLSTM); cùng bảy tập dữ liệu kể trên. Mọi mô hình học sâu được *huấn luyện từ đầu* với trọng số ngẫu nhiên, không dùng trọng số tiền huấn luyện, để kết quả phản ánh đúng năng lực của từng kiến trúc trên dữ liệu đích.

Phương pháp. Kết hợp nghiên cứu lý thuyết với kiểm chứng thực nghiệm có kiểm soát:

- cùng một hạt giống ngẫu nhiên (`seed = 42`) và cùng một cách chia huấn luyện/kiểm thử cho mọi mô hình trên một tập dữ liệu; bộ tiền xử lý chỉ được khớp trên phần huấn luyện;
- chia phân tầng theo nhãn cho bài toán phân loại mất cân bằng, chia theo trục thời gian (không xáo trộn) cho chuỗi giá chứng khoán để tránh rò rỉ thông tin tương lai;
- tập kiểm định tách riêng cho dừng sớm và chọn ngưỡng; tập kiểm thử chỉ dùng một lần để báo cáo;
- mỗi kết luận so sánh đi kèm một mốc tham chiếu (ví dụ dự báo "giữ nguyên giá hôm nay" cho chứng khoán);
- mọi con số trong báo cáo được sinh tự động từ tệp kết quả JSON của chính lần chạy, không chép tay.

Phân bổ phần cứng. Máy thực nghiệm có CPU 32 luồng và GPU NVIDIA GeForce RTX 4060 Laptop (8 GB). PyTorch dùng GPU; TensorFlow 2.21 trên Windows không hỗ trợ GPU nên các mô hình Keras chạy CPU. Các tiến trình độc lập chạy song song: scikit-learn, mạng NumPy và Keras trên CPU (giới hạn số luồng mỗi tiến trình), PyTorch trên GPU.

4. Bố cục báo cáo

- **Chương 1 – Lịch sử phát triển trí tuệ nhân tạo:** bức tranh hơn 80 năm từ nơ-ron hình thức đến hệ đa tác tử.
- **Chương 2 – Các kỹ thuật học máy cơ bản:** lý thuyết bốn họ thuật toán, EDA ba tập dữ liệu bảng và văn bản, pipeline scikit-learn, MLP trên ba nền tảng, đối chuẩn và Web App.
- **Chương 3 – Mạng nơ-ron tích chập:** phép tích chập, vùng cảm nhận, pooling, ba thế hệ kiến trúc; SVHN, EuroSAT và 1D CNN; hiện thực NumPy/Keras/PyTorch; đối chuẩn độ chính xác – chi phí; Web App thị giác.
- **Chương 4 – Mạng nơ-ron hồi quy:** RNN, BPTT, triệt tiêu đạo hàm, cắt tỉa gradient, LSTM, GRU, BiLSTM; chứng khoán VN30 và KKBox; hiện thực ba nền tảng; đối chuẩn PyTorch – Keras; Web App dự báo.

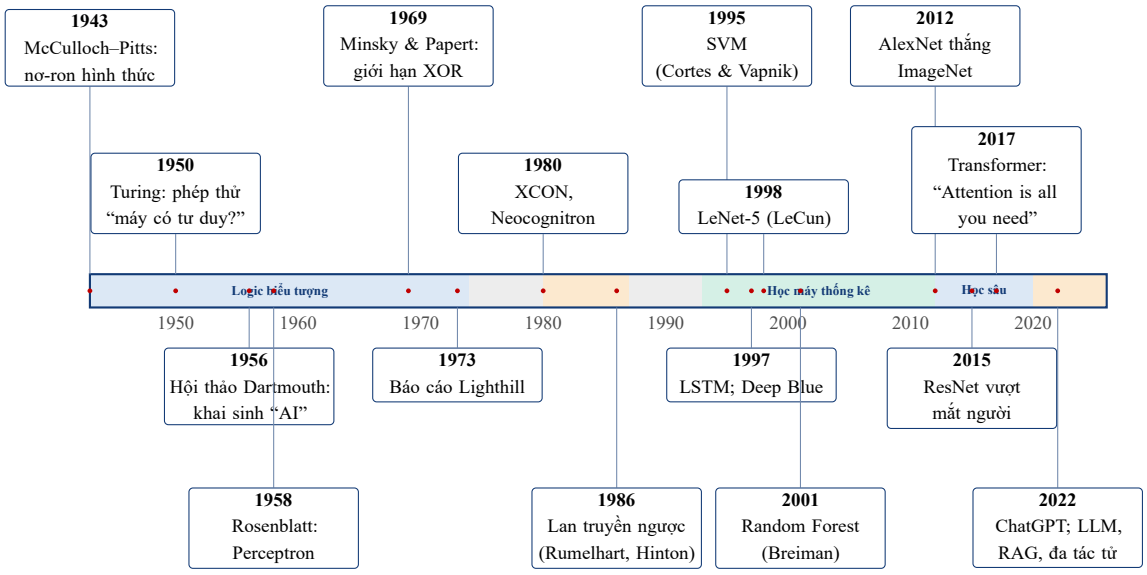
Phần Kết luận tổng hợp các phát hiện và hướng phát triển. Mã nguồn đầy đủ nằm trong thư mục `src/TieuLuan 01` của kho lưu trữ; mỗi đoạn mã trong báo cáo được trích nguyên văn từ đó.

Chương 1

Lịch Sử Phát Triển Trí Tuệ Nhân Tạo

1.1 Tổng quan bức tranh hơn tám thập kỷ của AI

Lịch sử trí tuệ nhân tạo không phải một đường đi lên đều đặn. Đó là chuỗi các chu kỳ kỳ vọng: mỗi đột phá kéo theo lời hứa vượt quá năng lực thật của công nghệ, rồi một giai đoạn thoái trào khi tiền tài trợ cạn, gọi là *mùa đông AI*. Qua các chu kỳ đó, cách máy tính "suy nghĩ" đổi ba lần: từ suy diễn trên ký hiệu do con người viết, sang mô hình xác suất học từ dữ liệu, rồi tới học biểu diễn nhiều tầng của học sâu. Hình 1.1 đặt mười sáu mốc then chốt lên cùng một trục thời gian, kèm các kỷ nguyên tương ứng.



Hình 1.1. Dòng thời gian tiến hóa của trí tuệ nhân tạo (1943 – 2026): các mốc then chốt và các kỷ nguyên công nghệ. Hai dải xám là hai mùa đông AI (1974 – 1980 và 1987 – 1993), dải cam giữa chúng là thời kỳ hệ chuyên gia (1980 – 1987).

Hiểu lịch sử không chỉ để biết ai làm gì, năm nào. Mỗi mô hình được hiện thực ở các chương sau đều là lời đáp cho một thất bại cụ thể trong quá khứ: MLP và lan truyền ngược trả lời giới hạn XOR của Perceptron, SVM và Random Forest trả lời sự giòn của hệ chuyên gia, LSTM trả lời hiện tượng triệt tiêu đạo hàm của RNN, ResNet trả lời sự thoái hóa của mạng rất sâu.

1.2 Khởi nguồn và kỷ nguyên logic biểu tượng (1940s – 1960s)

1.2.1 Ý niệm về máy tính có tư duy và phép thử Turing

Năm 1950, Alan Turing công bố bài báo *"Computing Machinery and Intelligence"* trên tạp chí *Mind* [2], mở đầu bằng câu hỏi "Máy móc có thể suy nghĩ không?". Để tránh tranh cãi triết học về khái niệm "ý thức", Turing thay câu hỏi bằng một thí nghiệm hành vi, *trò chơi bắt chước*: một người thẩm vấn trò chuyện bằng văn bản với hai đối tượng giấu mặt, một người và một máy. Nếu người thẩm vấn không phân biệt được đâu là máy, ta nói máy đã thể hiện hành vi thông minh. Phép thử Turing là phương pháp luận đầu tiên định lượng hóa trí tuệ nhân tạo bằng hành vi quan sát được, và đến nay vẫn là điểm quy chiếu khi bàn về các mô hình ngôn ngữ lớn.

1.2.2 Mô hình nơ-ron hình thức và Hội thảo Dartmouth (1956)

Trước cả Turing, năm 1943, nhà thần kinh học Warren McCulloch và nhà logic học Walter Pitts đề xuất mô hình nơ-ron hình thức [3]: một nơ-ron tính tổng có trọng số các tín hiệu nhị phân rồi so với một ngưỡng,

$$y = f\Big(\sum_{i=1}^{n} w_i x_i - \theta\Big), \qquad x_i \in \{0,1\}, \tag{1.1}$$

với f là hàm bước Heaviside. Hai tác giả chứng minh mạng các nơ-ron như vậy thực hiện được mọi hàm logic AND, OR, NOT. Năm 1949, Donald Hebb bổ sung quy tắc học "các nơ-ron cùng phát xung thì liên kết mạnh lên", cơ sở sinh học đầu tiên cho việc *thay đổi* trọng số.

Ngành AI chính thức ra đời mùa hè năm 1956 tại Hội thảo Dartmouth do John McCarthy, Marvin Minsky, Claude Shannon và Nathaniel Rochester tổ chức. McCarthy đặt tên "Artificial Intelligence" với giả thuyết: mọi khía cạnh của việc học hay của trí tuệ, về nguyên tắc, đều mô tả được chính xác tới mức một cỗ máy có thể mô phỏng nó.

1.2.3 Perceptron của Rosenblatt và các hệ suy diễn ký hiệu đầu tiên

Năm 1958, Frank Rosenblatt công bố Perceptron [4], mô hình nơ-ron đầu tiên *tự học* trọng số từ dữ liệu có nhãn theo quy tắc

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta\,(y - \hat{y})\,\mathbf{x}, \tag{1.2}$$

với $\eta > 0$ là tốc độ học. Định lý hội tụ Perceptron bảo đảm thuật toán dừng sau hữu hạn bước nếu dữ liệu khả tách tuyến tính. Song song với hướng kết nối (connectionism) này, hướng biểu tượng (symbolic AI) cũng gặt hái thành công: chương trình Logic Theorist (1956) của Newell, Simon và Shaw chứng minh được 38 trên 52 định lý ở chương 2 của *Principia Mathematica*; General Problem Solver (1959) giải các bài toán đố bằng phân tích phương tiện – mục đích.

1.3 "Mùa đông AI" lần thứ nhất (1970s)

1.3.1 Giới hạn toán học của Perceptron và cuốn sách của Minsky & Papert (1969)

Năm 1969, Minsky và Papert xuất bản *Perceptrons* [5], chứng minh chặt chẽ rằng Perceptron một tầng chỉ học được các hàm khả tách tuyến tính, và vì vậy thất bại với cả hàm XOR đơn giản (Bảng 1.1).

Bảng 1.1. Bảng chân lý của hàm XOR: hai lớp nằm trên hai đường chéo nên không đường thẳng nào tách được.

x_1	x_2	$y = x_1 \oplus x_2$	Vị trí trên mặt phẳng $\mathbb{R}^2$
0	0	0	gốc tọa độ $(0,0)$ – nhãn âm
0	1	1	điểm $(0,1)$ – nhãn dương
1	0	1	điểm $(1,0)$ – nhãn dương
1	1	0	điểm $(1,1)$ – nhãn âm

Muốn tách $\{(0,1),(1,0)\}$ khỏi $\{(0,0),(1,1)\}$ cần một đường thẳng $w_1x_1 + w_2x_2 + b = 0$ để hai nhóm nằm hai phía, nhưng từ $b < 0$, $w_1 + b > 0$, $w_2 + b > 0$ suy ra $w_1 + w_2 + b > -b > 0$, mâu thuẫn với yêu cầu điểm $(1,1)$ phải âm. Minsky và Papert biết mạng nhiều tầng giải được XOR, nhưng thời đó chưa ai có cách huấn luyện các tầng ẩn. Cuốn sách khiến nguồn tài trợ cho hướng nơ-ron gần như đóng băng suốt hơn một thập kỷ.

1.3.2 Báo cáo Lighthill (1973) và sự cắt giảm tài trợ của DARPA

Năm 1973, Báo cáo Lighthill [6] gửi Hội đồng Nghiên cứu Khoa học Anh kết luận rằng các hệ AI ký hiệu chỉ chạy được trong "thế giới đồ chơi" và bất lực trước bài toán thật vì *bùng nổ tổ hợp*: số trạng thái cần tìm kiếm tăng theo hàm mũ của kích thước bài toán. Chính phủ Anh cắt tài trợ AI ở hầu hết các trường đại học; cùng lúc, tu chính án Mansfield khiến DARPA của Hoa Kỳ ngừng chi cho nghiên cứu không có ứng dụng quân sự trực tiếp. Giai đoạn 1974 – 1980 được gọi là mùa đông AI thứ nhất.

1.4 Sự trỗi dậy của hệ chuyên gia và "mùa đông AI" lần thứ hai (1980s – 1990s)

1.4.1 Kỷ nguyên vàng của hệ chuyên gia

Đầu thập niên 1980, AI hồi sinh khi chuyển từ tìm kiếm tổng quát sang *tri thức chuyên ngành hẹp*. Một hệ chuyên gia tách hai thành phần: **cơ sở tri thức** gồm sự kiện và luật dạng `IF` điều kiện `THEN` kết luận, và **máy suy diễn** chạy luật theo chiều tiến (từ dữ kiện ra kết luận) hoặc chiều lùi (từ giả thuyết truy về dữ kiện). Các hệ tiêu biểu gồm DENDRAL (xác định cấu trúc phân tử từ phổ khối), MYCIN (chẩn đoán nhiễm trùng máu, độ chính xác ngang bác sĩ chuyên khoa) và XCON/R1 của Digital Equipment Corporation, tự cấu hình máy VAX và được ước tính tiết kiệm khoảng 40 triệu USD mỗi năm. Thành công của XCON kéo theo làn sóng đầu tư vào máy Lisp chuyên dụng và dự án Máy tính Thế hệ Thứ năm của Nhật Bản.

1.4.2 Sự tái sinh của mạng nơ-ron: thuật toán lan truyền ngược (1986)

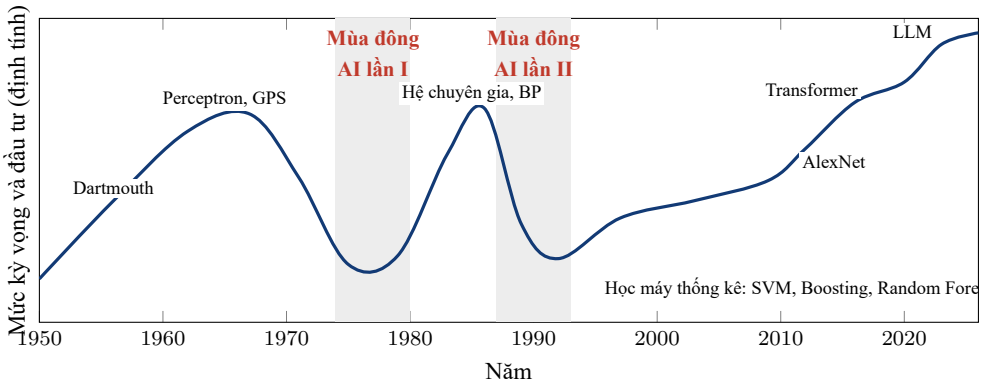
Năm 1986, Rumelhart, Hinton và Williams công bố trên *Nature* thuật toán lan truyền ngược cho mạng nhiều tầng [7]. Dùng quy tắc dây chuyền với hàm kích hoạt khả vi như sigmoid $\sigma(z) = 1/(1+e^{-z})$, đạo hàm của mất mát L theo một trọng số bất kỳ ở tầng ẩn l tách thành tích của sai số truyền ngược và kích hoạt tầng trước:

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = \delta_i^{(l)}\, a_j^{(l-1)}. \tag{1.3}$$

Kết quả này phá "lời nguyền XOR": mạng hai tầng học được biểu diễn ẩn phi tuyến. Cùng giai đoạn, Fukushima (1980) đề xuất Neocognitron [8] với các tầng trích đặc trưng cục bộ (S-cells) xen kẽ các tầng bất biến vị trí (C-cells), tiền thân trực tiếp của CNN. Chương 2 hiện thực lại công thức (1.3) bằng NumPy.

1.4.3 "Mùa đông AI" lần thứ hai (1987 – 1993)

Hệ chuyên gia sớm lộ ba điểm yếu cấu trúc. *Nút thắt thu nhận tri thức*: phỏng vấn chuyên gia để mã hóa hàng vạn luật vừa đắt vừa chậm, lại sinh mâu thuẫn. *Tính giòn*: hệ thống trả lời sai hoặc lặp vô tận khi gặp tình huống nằm ngoài các luật đã viết, chi phí bảo trì vượt lợi ích. *Sụp đổ phần cứng*: máy trạm Unix và vi xử lý x86 rẻ hơn khiến máy Lisp lỗi thời, kéo theo phá sản dây chuyền. Hình 1.2 minh họa định tính các chu kỳ kỳ vọng và hai mùa đông AI.



Hình 1.2. Chu kỳ kỳ vọng và hai thời kỳ "mùa đông AI" (minh họa định tính, không phải số đo).

Sau mỗi lần thoái trào, cộng đồng tái cấu trúc phương pháp luận theo hướng thực dụng hơn và có cơ sở toán học chắc hơn.

1.5 Kỷ nguyên học máy thống kê (1990s – 2010)

Bài học rút ra từ thất bại của hệ chuyên gia: thay vì cố viết tay mọi quy tắc của thế giới, hãy để máy học quy luật xác suất trực tiếp từ dữ liệu.

1.5.1 Lý thuyết học thống kê và máy vector hỗ trợ (SVM)

Vapnik và Chervonenkis xây dựng lý thuyết học thống kê với nguyên lý cực tiểu hóa rủi ro cấu trúc: kiểm soát cùng lúc sai số huấn luyện và độ phức tạp của lớp hàm (đo bằng chiều VC). Đỉnh cao là máy vector hỗ trợ [9], tìm siêu phẳng có *lề lớn nhất* tới các điểm gần nhất:

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \ \forall i. \tag{1.4}$$

Nhờ thủ thuật hàm nhân, ví dụ nhân RBF $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$, SVM tách được dữ liệu phi tuyến bằng cách ngầm ánh xạ lên không gian vô hạn chiều. Bài toán tối ưu là lồi nên không mắc cực tiểu địa phương; SVM thống trị nhận dạng ảnh và phân loại văn bản suốt hơn một thập kỷ.

1.5.2 Cây quyết định và các phương pháp học tập hợp

Cây quyết định (CART, C4.5) chia không gian bằng chuỗi câu hỏi đúng/sai, dễ diễn giải. Hai cách kết hợp nhiều mô hình yếu thành một mô hình mạnh xác lập vị thế trên dữ liệu bảng: **Boosting** (AdaBoost [10], sau đó Gradient Boosting) huấn luyện tuần tự các mô hình sửa lỗi của nhau, còn **Random Forest** [11] lấy trung bình hàng trăm cây học trên các mẫu bootstrap với tập con đặc trưng ngẫu nhiên, giảm phương sai mà không tăng độ chệch. Chương 2 cho thấy Random Forest đến nay vẫn là đối thủ khó vượt trên dữ liệu bảng.

1.5.3 LeNet-5 (1998): nền tảng của mạng tích chập hiện đại

Năm 1998, LeCun và cộng sự công bố LeNet-5 [12], ghép ba ý tưởng: vùng tiếp nhận cục bộ, chia sẻ trọng số và lấy mẫu giảm (pooling). LeNet-5 đọc hàng triệu tấm séc ngân hàng mỗi ngày ở Hoa Kỳ, nhưng giới hạn CPU và thiếu dữ liệu ảnh lớn khiến nó chưa mở rộng được sang ảnh tự nhiên phức tạp.

1.6 Cuộc cách mạng học sâu (2012 – 2020)

1.6.1 Bước ngoặt ImageNet 2012 và AlexNet

Tại cuộc thi ILSVRC 2012, AlexNet của Krizhevsky, Sutskever và Hinton [13] đạt sai số top-5 15,3 %, bỏ xa vị trí thứ hai (26,2 %) dùng đặc trưng thủ công SIFT/HOG cùng SVM. Ba yếu tố hội tụ: **dữ liệu lớn** (ImageNet hơn 1,2 triệu ảnh, 1.000 lớp), **GPU** với CUDA tăng tốc nhân ma trận hàng chục lần so với CPU, và **các cải tiến thuật toán** như hàm kích hoạt ReLU $f(x) = \max(0, x)$ không bão hòa ở vùng dương và điều chuẩn Dropout [14] ngắt ngẫu nhiên nơ-ron để chống đồng thích nghi.

1.6.2 Sự tiến hóa của các kiến trúc thị giác: VGG, ResNet, MobileNet

VGGNet [15] chuẩn hóa thiết kế bằng cách xếp chồng các bộ lọc 3×3: hai tầng 3×3 có cùng vùng nhìn với một tầng 5×5 nhưng ít hơn 28 % trọng số và thêm một hàm phi tuyến. **ResNet** [16] giải quyết hiện tượng thoái hóa (mạng sâu hơn lại có sai số huấn luyện cao hơn) bằng kết nối tắt $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$, cho phép huấn luyện mạng 152 tầng và đưa sai số top-5 ImageNet xuống 3,57 %. **MobileNet** [17, 18] tách tích chập chuẩn thành depthwise và pointwise, giảm chi phí tính toán 8 – 9 lần để chạy trên điện thoại. Cả ba kiến trúc được hiện thực và đối chuẩn ở Chương 3.

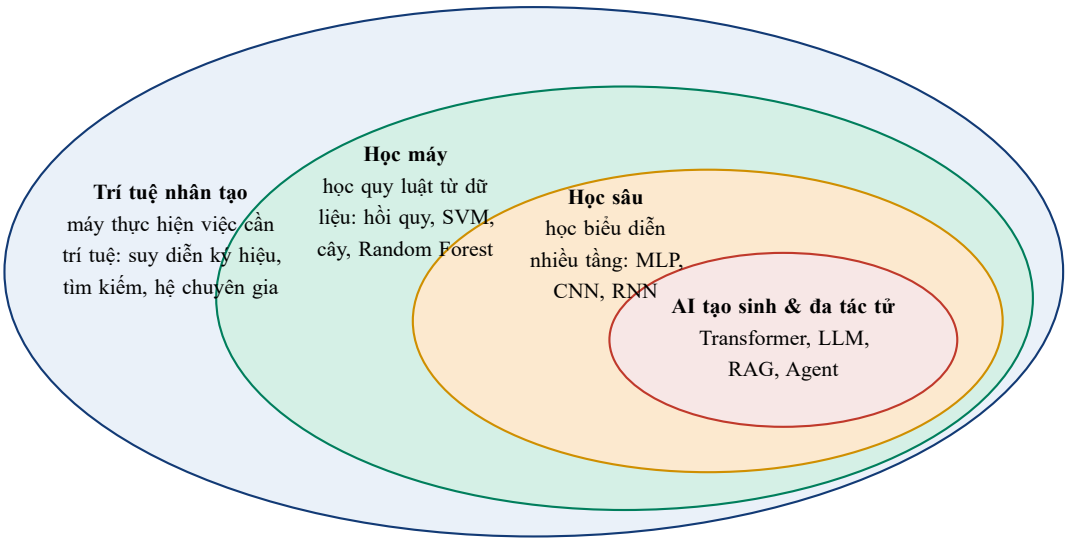
1.6.3 Mô hình hóa chuỗi: từ RNN, LSTM đến Transformer

RNN duy trì một trạng thái ẩn qua các bước thời gian, nhưng lan truyền ngược qua thời gian nhân liên tiếp các ma trận Jacobian nên gradient suy giảm theo hàm mũ [19]. LSTM [20] thêm ô nhớ được điều khiển bởi ba cổng (quên, vào, ra), tạo đường đi cộng cho gradient; GRU [21] rút gọn còn hai cổng. Năm 2017, Transformer [22] bỏ hẳn liên kết hồi quy, dùng cơ chế tự chú ý đa đầu để mọi vị trí trong chuỗi nhìn thấy nhau trực tiếp và tính song song hoàn toàn trên GPU, mở đường cho các mô hình ngôn ngữ khổng lồ. Chương 4 hiện thực và so sánh bốn kiến trúc hồi quy.

1.7 Kỷ nguyên AI tạo sinh, mô hình nền tảng và hệ đa tác tử (2020 – nay)

1.7.1 Mô hình ngôn ngữ lớn và định luật tỷ lệ

Kaplan và cộng sự [23], rồi Hoffmann và cộng sự (Chinchilla) [24] cho thấy mất mát của mô hình ngôn ngữ giảm theo luật lũy thừa khá đều với ba đại lượng: số tham số, lượng dữ liệu và ngân sách tính toán. Ở quy mô hàng trăm tỷ tham số xuất hiện các năng lực nổi trội: học trong ngữ cảnh từ vài ví dụ mà không cập nhật trọng số, suy luận từng bước (chain-of-thought), sinh mã nguồn. Hình 1.3 tóm tắt quan hệ bao hàm giữa các thế hệ.



Hình 1.3. Quan hệ bao hàm giữa trí tuệ nhân tạo, học máy, học sâu và AI tạo sinh / hệ đa tác tử.

1.7.2 Giới hạn của mô hình thuần tham số và vai trò của đồ thị tri thức, RAG

Mô hình ngôn ngữ chỉ lưu tri thức trong trọng số nên có ba giới hạn: *ảo giác* (sinh câu trả lời trôi chảy nhưng sai sự thật, vì mô hình tối ưu xác suất từ tiếp theo chứ không kiểm chứng), *tri thức đóng băng* tại thời điểm tiền huấn luyện, và *khó truy vết nguồn* trong các lĩnh vực cần trích dẫn như y tế, pháp luật. Xu hướng khắc phục là ghép thêm bộ nhớ phi tham số: **đồ thị tri thức** lưu sự kiện dưới dạng bộ ba (thực thể, quan hệ, thực thể) để suy diễn tường minh; **mạng nơ-ron đồ thị** mở rộng học sâu sang dữ liệu có cấu trúc đồ thị bằng truyền thông điệp giữa các nút; **RAG** [25] truy xuất tài liệu liên quan rồi đưa vào ngữ cảnh trước khi sinh câu trả lời.

1.7.3 Xu hướng: hệ đa tác tử tự trị

Bước tiếp theo là các *tác tử* (agent) tự chủ có chu trình nhận thức đầy đủ: quan sát, lập kế hoạch và suy luận, gọi công cụ hay API, rồi tự đánh giá và sửa sai. Nhiều tác tử chuyên biệt (lập kế hoạch, nghiên cứu, lập trình, kiểm thử) phối hợp và phản biện nhau trong một hệ đa tác tử giải được những việc kỹ thuật mà một mô hình đơn lẻ không làm nổi. Đây là định hướng then chốt của hệ thống thông minh thế hệ mới, và cũng là nội dung các chương sau của học phần (đồ thị tri thức, mạng nơ-ron đồ thị, hệ đa tác tử) sẽ kế thừa từ nền tảng mô hình hóa được xây dựng trong tiểu luận này.

Chương 2

Các Kỹ Thuật Học Máy Cơ Bản

2.1 Cơ sở lý thuyết các thuật toán học máy cốt lõi

Học máy có giám sát giải hai lớp bài toán: *hồi quy* (dự báo một giá trị liên tục) và *phân loại* (gán nhãn rời rạc). Cho tập huấn luyện $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ với $\mathbf{x} \in \mathbb{R}^D$, ta chọn một họ hàm f_θ và tìm tham số θ cực tiểu hóa một hàm mất mát trung bình trên dữ liệu. Bốn họ thuật toán dưới đây khác nhau ở họ hàm và cách tối ưu [26, 1].

2.1.1 Hồi quy tuyến tính (Linear Regression)

Mô hình giả định đầu ra là tổ hợp tuyến tính của đặc trưng:

$$\hat{y} = f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^{D} w_j x_j + b, \tag{2.1}$$

và cực tiểu hóa sai số toàn phương trung bình

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)}\right)^2 = \frac{1}{2N} \|X\mathbf{w} + b\mathbf{1} - \mathbf{y}\|_2^2. \tag{2.2}$$

Nghiệm đóng là phương trình chuẩn $\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}$; khi D lớn có thể dùng hạ gradient $\mathbf{w} \leftarrow \mathbf{w} - \eta \, \frac{1}{N} X^\top (X\mathbf{w} - \mathbf{y})$. Khi các đặc trưng tương quan mạnh (đa cộng tuyến), $X^\top X$ gần suy biến và hệ số dao động lớn; **Ridge** thêm phạt $\alpha \|\mathbf{w}\|_2^2$ để nghiệm thành $(X^\top X + \alpha I)^{-1} X^\top \mathbf{y}$, luôn khả nghịch.

2.1.2 Hồi quy Logistic (Logistic Regression)

Với nhãn $y \in \{0, 1\}$, tổ hợp tuyến tính được ép về khoảng $(0, 1)$ bằng hàm sigmoid:

$$p = P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad z = \mathbf{w}^\top \mathbf{x} + b. \tag{2.3}$$

Ước lượng hợp lý cực đại dẫn tới hàm mất mát entropy chéo nhị phân

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \Big[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \Big], \tag{2.4}$$

có đạo hàm theo z rút gọn thành $\partial \mathcal{L} / \partial z = \hat{y} - y$, nên $\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \sum_i (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$. Mất mát là hàm lồi, mọi cực tiểu đều là toàn cục. Với K lớp, sigmoid được thay bằng softmax $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$ và mất mát là entropy chéo $-\log \hat{y}_y$. Khi dữ liệu mất cân bằng, trọng số lớp `class_weight="balanced"` nhân mất mát của mỗi mẫu với $N / (K \, N_k)$ để lớp hiếm có tiếng nói ngang lớp phổ biến.

2.1.3 Cây quyết định và Rừng ngẫu nhiên (Random Forest)

Cây quyết định chia không gian bằng chuỗi câu hỏi nhị phân "$x_j \le \theta$?". Độ thuần của một nút t có các lớp với tỉ lệ p_k được đo bằng chỉ số Gini hoặc entropy:

$$\text{Gini}(t) = 1 - \sum_{k=1}^{K} p_k^2, \qquad \text{Entropy}(t) = -\sum_{k=1}^{K} p_k \log_2 p_k. \tag{2.5}$$

Tại mỗi nút, thuật toán chọn cặp (j, θ) làm giảm tạp chất nhiều nhất:

$$\Delta\text{Gini} = \text{Gini}(t) - \Big(\frac{N_L}{N_t}\text{Gini}(t_L) + \frac{N_R}{N_t}\text{Gini}(t_R)\Big). \tag{2.6}$$

Với hồi quy, tạp chất là phương sai của nhãn trong nút. Một cây sâu khớp rất sát dữ liệu huấn luyện nhưng có phương sai lớn. **Random Forest** [11] trung bình hóa B cây với hai nguồn ngẫu nhiên: mỗi cây học trên một mẫu bootstrap (rút có hoàn lại N mẫu) và mỗi lần chia chỉ xét một tập con ngẫu nhiên $\sqrt{D}$ đặc trưng. Hai cơ chế này làm các cây ít tương quan với nhau, nên trung bình của chúng có phương sai nhỏ hơn hẳn từng cây mà độ chệch gần như giữ nguyên.

2.1.4 Máy vector hỗ trợ (Support Vector Machine)

SVM [9] tìm siêu phẳng phân tách có lề lớn nhất, cho phép vi phạm qua biến nới lỏng ξ_i:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \quad \text{với} \quad y^{(i)}(\mathbf{w}^\top\mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \ \ \xi_i \geq 0, \tag{2.7}$$

trong đó C cân bằng giữa lề rộng và lỗi huấn luyện. Bài toán đối ngẫu chỉ phụ thuộc vào tích vô hướng giữa các mẫu, nên có thể thay bằng một hàm nhân. Nhân RBF

$$K(\mathbf{x}, \mathbf{x}') = \exp\big(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2\big) \tag{2.8}$$

tương ứng với ánh xạ lên không gian vô hạn chiều. Cái giá là bộ giải phải làm việc với ma trận nhân $N \times N$: thời gian huấn luyện tăng cỡ $O(N^2)$ tới $O(N^3)$ và mô hình phải lưu toàn bộ vector hỗ trợ. **SVR** là phiên bản hồi quy, phạt các điểm nằm ngoài ống sai số rộng ε. Với văn bản TF-IDF hàng nghìn chiều, dữ liệu thường đã gần khả tách tuyến tính nên **LinearSVC** (nhân tuyến tính, độ phức tạp tuyến tính theo N) là lựa chọn hợp lý hơn.

2.2 Khảo sát thực nghiệm và tiền xử lý ba bộ dữ liệu

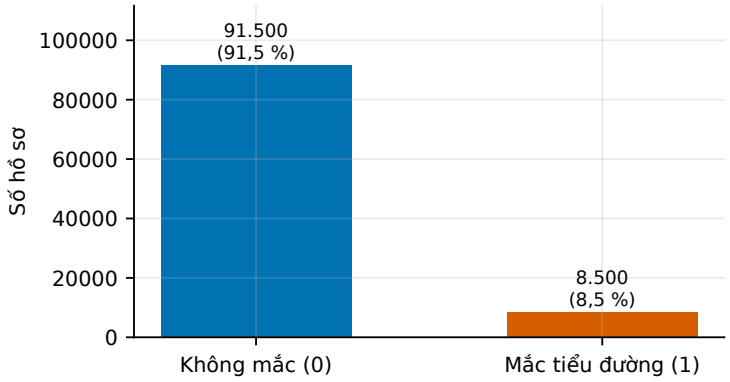
Ba tập dữ liệu được chọn để phủ ba dạng bài toán: phân loại nhị phân mất cân bằng (y tế), hồi quy (bất động sản Việt Nam) và phân loại văn bản ba lớp (thương mại điện tử). Mọi tập đều chia 80 % huấn luyện / 20 % kiểm thử với `random_state = 42` (phân tầng theo nhãn với bài toán phân loại); mạng MLP tách thêm 10 % của phần huấn luyện làm tập kiểm định cho dừng sớm.

2.2.1 Tập dữ liệu 1: Sàng lọc nguy cơ tiểu đường (100.000 mẫu)

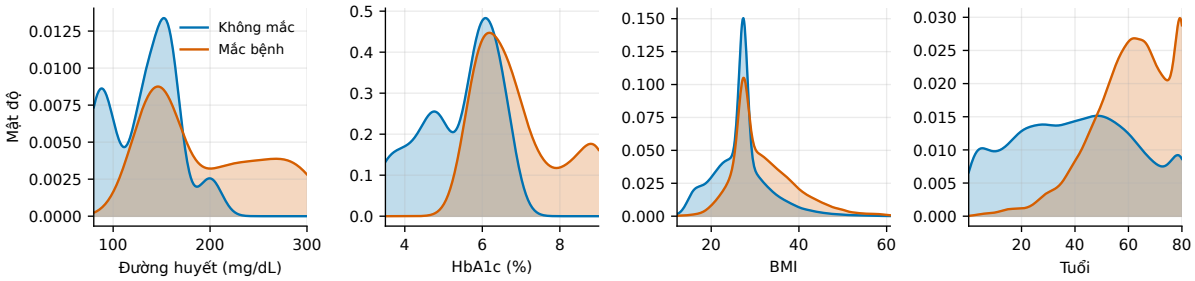
- **Nguồn:** Diabetes Prediction Dataset trên Kaggle [27], tổng hợp hồ sơ khám bệnh.
- **Quy mô:** 100.000 hồ sơ, 8 thuộc tính: giới tính, tuổi, tăng huyết áp, bệnh tim, tiền sử hút thuốc, chỉ số khối cơ thể BMI, HbA1c (%), đường huyết (mg/dL); nhãn `diabetes` (0: không mắc, 1: mắc).

Bảng 2.1. Năm bản ghi ngẫu nhiên của tập dữ liệu tiểu đường.

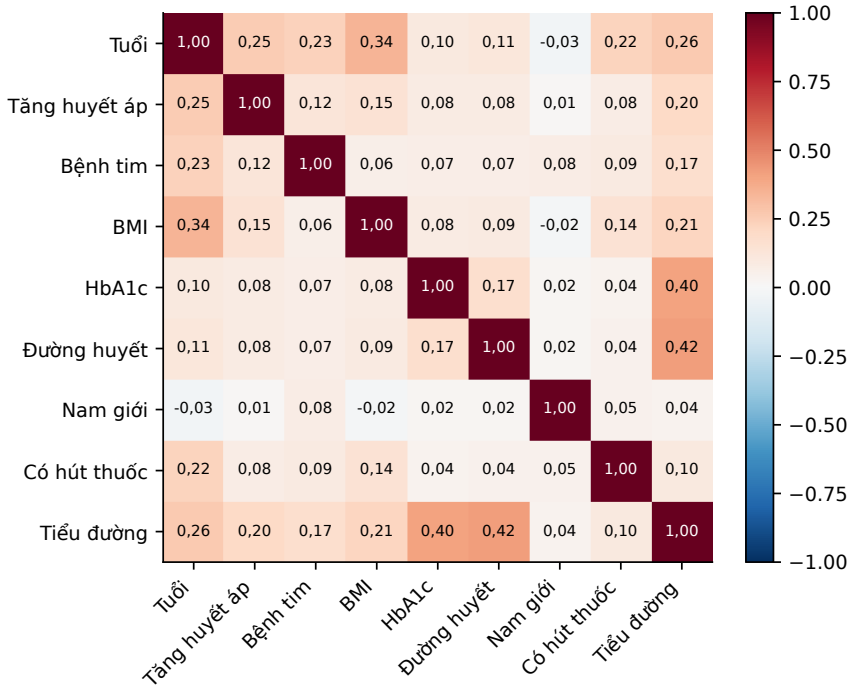
STT	Giới	Tuổi	Huyết áp	Tim	Hút thuốc	BMI	HbA1c	Glucose	Nhãn y
1	Nam	67	0	0	not current	29,15	6,2	100	0
2	Nữ	4	0	0	never	27,32	6,5	140	0
3	Nữ	40	0	0	never	29,47	6,2	130	0
4	Nam	53	0	0	never	27,32	6,1	155	0
5	Nam	65	1	0	No Info	49,71	5,8	130	0



Hình 2.1. Phân bố nhãn của tập tiểu đường: lớp dương chỉ chiếm 8,50 %.



Hình 2.2. Mật độ xác suất của đường huyết, HbA1c, BMI và tuổi theo tình trạng bệnh.



Hình 2.3. Ma trận tương quan Pearson giữa các chỉ số lâm sàng và nhãn tiểu đường.

Nhận xét EDA.

- **Mất cân bằng lớp:** chỉ 8.500 trên 100.000 hồ sơ là dương tính (Hình 2.1). Một mô hình ngây thơ luôn đoán "không mắc" đã đạt Accuracy $100 - 8{,}50 \approx 91{,}5\,\%$ mà không phát hiện được ca bệnh nào; vì vậy Accuracy không đủ để đánh giá, cần Precision, Recall và F1 của lớp dương cùng ROC-AUC.
- **Hai đặc trưng quyết định:** đường huyết và HbA1c có tương quan với nhãn lần lượt $r = 0{,}4196$ và $r = 0{,}4007$, cao nhất trong các chỉ số (Hình 2.3). Phân bố của người mắc bệnh dịch hẳn sang phải (Hình 2.2): tỉ lệ mắc bệnh là 36,08 % khi đường huyết ≥ 200 mg/dL, 36,82 % khi HbA1c $\geq 6{,}6\,\%$, nhưng chỉ 0,00 % khi HbA1c $< 5{,}7\,\%$.
- **Quan hệ dạng ngưỡng:** cả hai chỉ số đều có "ngưỡng gãy" rõ (đặc biệt HbA1c), một dạng phi tuyến mà mô hình tuyến tính không biểu diễn trọn được nhưng cây quyết định và mạng nơ-ron thì có thể.
- **Tiền xử lý:** chuẩn hóa z-score bốn biến số, giữ nguyên hai biến nhị phân, mã hóa one-hot giới tính và tiền sử hút thuốc (35,82 % hồ sơ ghi "No Info", được giữ thành một nhóm riêng).

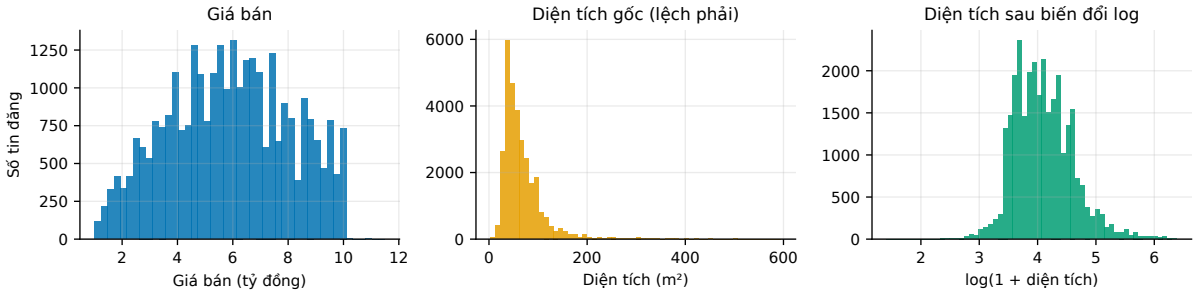
2.2.2 Tập dữ liệu 2: Định giá bất động sản Việt Nam (30.229 mẫu)

- **Nguồn:** Vietnam Housing Dataset [28], tin đăng bán nhà trên các trang bất động sản Việt Nam.
- **Quy mô:** 30.229 tin đăng thuộc 78 tỉnh/thành và 342 quận/huyện; các thuộc tính diện tích, mặt tiền, độ rộng đường vào, số tầng, số phòng ngủ, số phòng tắm, hướng nhà, hướng ban công, tình trạng pháp lý, nội thất, địa chỉ; nhãn là giá bán (tỷ đồng).

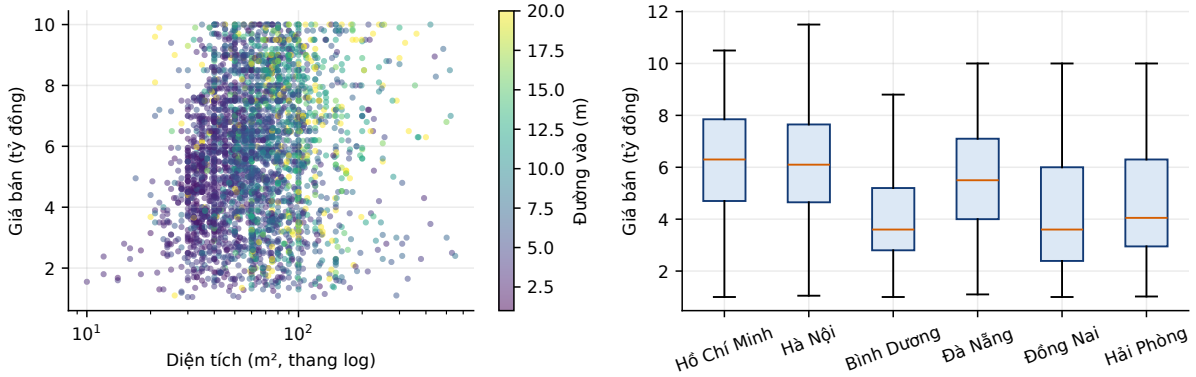
Bảng 2.2. Năm tin đăng ngẫu nhiên của tập dữ liệu giá nhà.

STT	Diện tích	Tầng	Phòng ngủ	Mặt tiền	Đường vào	Quận/huyện	Tỉnh/thành	Giá (tỷ)
1	36,0	4	2	4,0	–	Hai Bà Trưng	Hà Nội	6,00
2	30,0	2	1	3,7	6,0	Quận 3	Hồ Chí Minh	5,60
3	56,0	3	3	–	–	Sơn Trà	Đà Nẵng	6,85
4	60,0	5	6	–	–	Thanh Trì	Hà Nội	8,30
5	60,0	–	–	–	–	Thuận An	Bình Dương	3,70

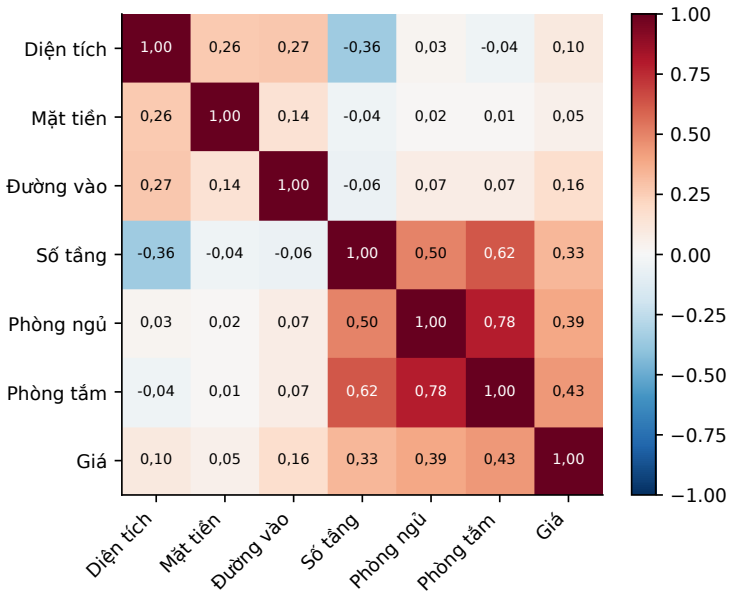
Dấu – là ô trống trong dữ liệu gốc (được điền trung vị và đánh dấu khi tiền xử lý).



Hình 2.4. Phân bố giá bán, diện tích gốc và diện tích sau biến đổi $\log(1+x)$.



Hình 2.5. Trái: giá theo diện tích (thang log), tô màu theo độ rộng đường vào. Phải: giá theo sáu tỉnh/thành có nhiều tin nhất.



Hình 2.6. Ma trận tương quan giữa các thuộc tính vật lý và giá bán.

Nhận xét EDA.

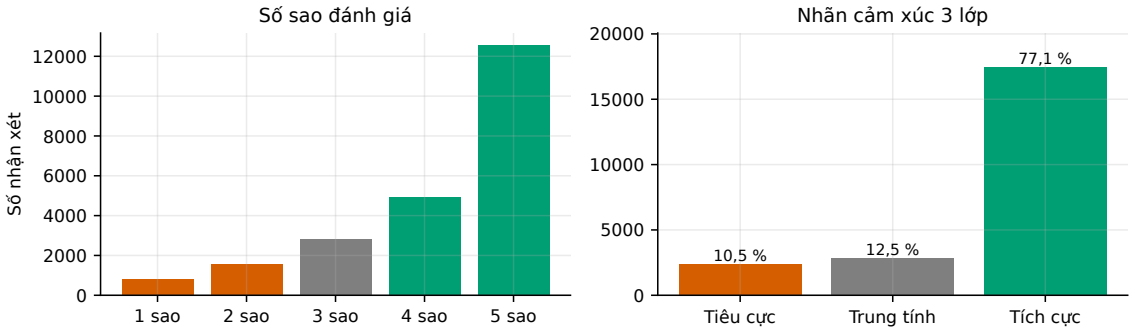
- **Giá đã bị chặn:** giá nằm trong $[1{,}000; 11{,}50]$ tỷ đồng, trung bình 5,872, độ lệch $-0{,}0290$ (Hình 2.4). Phân bố gần đối xứng nên *không* cần biến đổi log cho nhãn; ngược lại diện tích lệch phải mạnh (độ lệch 3,889), sau $\log(1+x)$ còn 0,6465.
- **Tương quan yếu và phi tuyến:** tương quan tuyến tính của giá với diện tích chỉ $r = 0{,}0976$, với số tầng $r = 0{,}3316$, với đường vào $r = 0{,}1643$ (Hình 2.6). Giá phụ thuộc mạnh vào vị trí: trung vị giá ở TP. Hồ Chí Minh và Hà Nội cao nhất, Đà Nẵng thấp hơn một chút, còn Bình Dương và Đồng Nai thấp hơn gần một nửa (Hình 2.5), mà vị trí là biến phân loại có hàng trăm giá trị.
- **Thiếu dữ liệu nhiều:** 38,25 % tin thiếu mặt tiền, 43,99 % thiếu đường vào, 70,26 % thiếu hướng nhà.
- **Tiền xử lý:** tách tỉnh và quận/huyện từ địa chỉ; biến số được điền trung vị *kèm cờ đánh dấu thiếu* rồi chuẩn hóa z-score; biến phân loại điền "Không rõ" rồi mã hóa one-hot, gộp các giá trị xuất hiện dưới 40 lần thành một nhóm "hiếm" để không sinh hàng trăm cột thưa.

2.2.3 Tập dữ liệu 3: Phân loại cảm xúc nhận xét thương mại điện tử (22.641 mẫu)

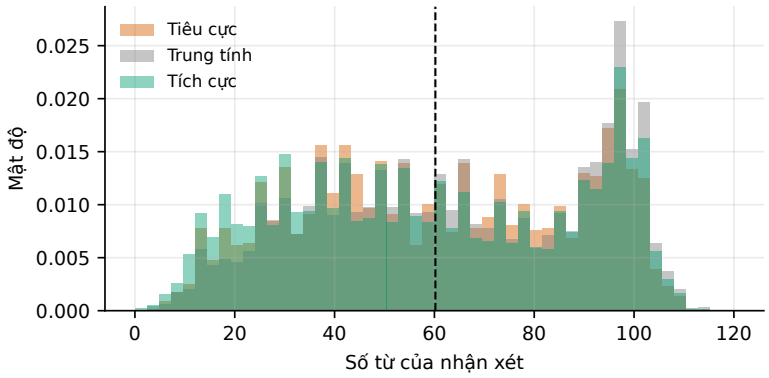
- **Nguồn:** Women's E-Commerce Clothing Reviews [29], nhận xét thật (tiếng Anh) của khách hàng một nhà bán lẻ thời trang trực tuyến.
- **Quy mô:** 23.486 bản ghi, giữ 22.641 bản có nội dung nhận xét. Nhãn cảm xúc ba lớp suy từ số sao: 1 – 2 sao là *tiêu cực*, 3 sao là *trung tính*, 4 – 5 sao là *tích cực*. Tiêu đề và nội dung được ghép thành một văn bản.

Bảng 2.3. Năm nhận xét tiêu biểu kèm nhãn cảm xúc.

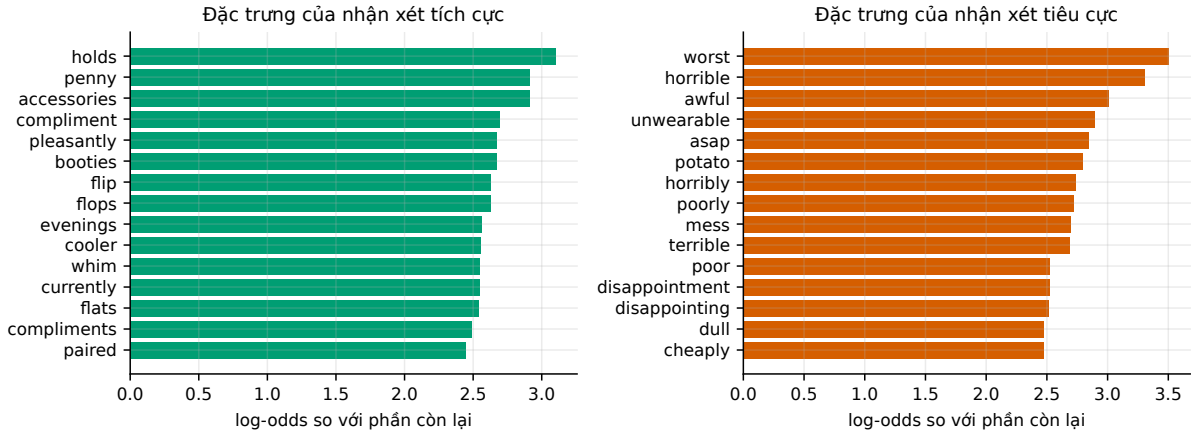
STT	Số sao	Nội dung nhận xét (nguyên văn tiếng Anh)	Nhãn
1	1 sao	I tried this top on thinking it would be cute for the summer. it looked more like a maternity top with the ful...	Tiêu cực
2	2 sao	I have been struggling to find a nice flowy summer dress that isn't too short. i am petite but do not like to ...	Tiêu cực
3	3 sao	Great idea, but this was not a flattering top on me. it almost worked, but ended up making me look like i am p...	Trung tính
4	3 sao	I didn't plan to wear this off shoulder. sadly the neck is too loose and big if worn on shoulder. the fabric i...	Trung tính
5	5 sao	I tried these on in-store not intending to buy them (was trying on shirts and needed to see how they would loo...	Tích cực



Hình 2.7. Phân bố số sao và nhãn cảm xúc: lớp tích cực chiếm 77,06 %.



Hình 2.8. Phân bố độ dài nhận xét (số từ) theo nhãn; nét đứt là trung bình 60,20 từ.



Hình 2.9. Mười lăm từ đặc trưng nhất của nhận xét tích cực và tiêu cực, xếp theo tỉ số log-odds so với phần còn lại.

Nhận xét EDA và quy trình tiền xử lý văn bản. Dữ liệu lệch mạnh về phía tích cực (77,06 %), trong khi trung tính chỉ 12,47 % và tiêu cực 10,47 % (Hình 2.7); vì vậy chỉ số chính là F1 trung bình macro trên ba lớp. Độ dài gần như không phân biệt được các lớp: trung bình 60,90 từ ở nhận xét tiêu cực, 64,20 ở trung tính và 59,50 ở tích cực (Hình 2.8), nên tín hiệu nằm ở *từ ngữ* chứ không ở độ dài. Từ đặc trưng nhất của lớp tiêu cực là *worst*, *horrible*, *awful*, *unwearable*; của lớp tích cực là *compliment*, *pleasantly*, *holds* (Hình 2.9). Quy trình tiền xử lý:

1. chuyển về chữ thường, bỏ URL và mọi ký tự không phải chữ cái hay dấu nháy;
2. loại từ dừng tiếng Anh (danh sách của scikit-learn);
3. vector hóa TF-IDF với unigram và bigram, giữ 5.000 đặc trưng tần suất cao nhất, bỏ từ xuất hiện dưới 3 văn bản, dùng $1 + \log(\text{tf})$ để giảm ảnh hưởng của từ lặp nhiều lần.

2.3 Hiện thực hóa và thực nghiệm các mô hình học máy cơ bản

Mỗi mô hình được đóng gói thành một `Pipeline` của scikit-learn [30] gồm bộ tiền xử lý và bộ ước lượng. Gọi `fit` trên phần huấn luyện sẽ khớp cả hai, nên không thống kê nào của tập kiểm thử rò vào quá trình học; lưu pipeline bằng `joblib` là có ngay đối tượng nhận dữ liệu thô cho Web App.

2.3.1 Pipeline hồi quy tuyến tính và Ridge

Bộ tiền xử lý giá nhà (Đoạn mã 2.1) xử lý riêng nhóm cột số và nhóm cột phân loại bằng `ColumnTransformer`; Đoạn mã 2.2 ghép nó với mô hình tuyến tính, `alpha=None` cho bình phương tối thiểu thường và `alpha=1.0` cho Ridge.

```
# tl/data_tabular.py
def house_preprocessor():
    num = Pipeline([("imp", SimpleImputer(strategy="median", add_indicator=True)), ("sc", StandardScaler())])
    cat = Pipeline([("imp", SimpleImputer(strategy="constant", fill_value="Không rõ")),
                    ("oh", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=40,
                                         sparse_output=False))])
    return ColumnTransformer([("num", num, HOUSE_NUM), ("cat", cat, HOUSE_CAT)])
```

Đoạn mã 2.1. Bộ tiền xử lý dữ liệu giá nhà: điền thiếu kèm cờ, chuẩn hóa, one-hot có gộp nhóm hiếm

```python
# tl/ml_classic.py
def build_linear_regression_pipeline(preprocessor, alpha=None):
    """alpha=None: bình phương tối thiểu thường (OLS); alpha>0: Ridge (phạt L2) chống đa cộng tuyến."""
    reg = LinearRegression() if alpha is None else Ridge(alpha=alpha, random_state=SEED)
    return Pipeline([("pre", preprocessor), ("model", reg)])
```

Đoạn mã 2.2. Pipeline hồi quy tuyến tính và Ridge với scikit-learn

2.3.2 Pipeline hồi quy Logistic cho dữ liệu y tế

`class_weight="balanced"` bù cho tỉ lệ dương 8,50 %; bộ giải `lbfgs` là phương pháp quasi-Newton hội tụ nhanh với số đặc trưng nhỏ.

```python
# tl/ml_classic.py
def build_logistic_regression_pipeline(preprocessor, balanced=True):
    clf = LogisticRegression(C=1.0, class_weight="balanced" if balanced else None,
                             solver="lbfgs", max_iter=2000, random_state=SEED)
    return Pipeline([("pre", preprocessor), ("model", clf)])
```

Đoạn mã 2.3. Pipeline hồi quy Logistic có cân bằng trọng số lớp

2.3.3 Pipeline Rừng ngẫu nhiên

Rừng gồm 200 cây không giới hạn độ sâu, mỗi lá tối thiểu 2 mẫu; với phân loại, trọng số lớp được cân bằng lại trên từng mẫu bootstrap. Cây không cần chuẩn hóa đặc trưng, nhưng dùng chung bộ tiền xử lý giúp mọi mô hình nhận cùng một đầu vào.

```python
# tl/ml_classic.py
def build_rf_pipeline(preprocessor, task="classification", n_estimators=200, max_depth=None,
                      min_samples_leaf=2, n_jobs=12):
    if task == "regression":
        model = RandomForestRegressor(n_estimators=n_estimators, max_depth=max_depth,
                                      min_samples_leaf=min_samples_leaf, n_jobs=n_jobs, random_state=SEED)
    else:
        model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth,
                                       min_samples_leaf=min_samples_leaf, class_weight="balanced_subsample",
                                       n_jobs=n_jobs, random_state=SEED)
    return Pipeline([("pre", preprocessor), ("model", model)])
```

Đoạn mã 2.4. Pipeline Random Forest cho cả hồi quy và phân loại

2.3.4 Pipeline Máy vector hỗ trợ

Nhân RBF tính khoảng cách Euclid nên đặc trưng bắt buộc cùng thang đo; bộ tiền xử lý đã chuẩn hóa z-score. Văn bản dùng `LinearSVC`.

```python
# tl/ml_classic.py
def build_svm_pipeline(preprocessor, task="classification", kernel="rbf", C=1.0):
    """SVM tính khoảng cách nên đặc trưng phải cùng thang đo: bộ tiền xử lý đã có StandardScaler.

    Văn bản TF-IDF thưa nhiều chiều dùng LinearSVC (độ phức tạp tuyến tính theo số mẫu); dữ liệu
    bảng ít chiều dùng nhân RBF.
    """
    if task == "regression":
        model = SVR(kernel=kernel, C=C, epsilon=0.1, cache_size=2000)
    elif kernel == "linear":
        model = LinearSVC(C=C, class_weight="balanced", max_iter=5000, random_state=SEED)
    else:
        model = SVC(kernel=kernel, C=C, gamma="scale", class_weight="balanced", cache_size=4000,
                    random_state=SEED)
    return Pipeline([("pre", preprocessor), ("model", model)])
```

Đoạn mã 2.5. Pipeline SVM: SVC/SVR nhân RBF cho dữ liệu bảng, LinearSVC cho văn bản

2.3.5 Kết quả thực nghiệm các mô hình học máy cơ bản

Ba bảng dưới đây tách theo bản chất bài toán. Thời gian là thời gian huấn luyện trên CPU (Random Forest dùng 12 luồng); dung lượng là kích thước tệp `joblib` nén.

Bảng 2.4. Kết quả các mô hình học máy cơ bản trên tập tiểu đường (100.000 hồ sơ, kiểm thử 20 %).

Mô hình	Acc (%)	Precision	Recall	F1	ROC-AUC	Huấn luyện (s)	Dung lượng
Logistic Regression (balanced)	88,89	0,4266	0,8941	0,5776	0,9629	0,1	1,9 KB
SVM nhân RBF (balanced)	89,58	0,4451	**0,9188**	0,5997	0,9718	93,1	308,2 KB
Random Forest (200 cây)	**96,39**	**0,8226**	0,7335	**0,7755**	**0,9738**	1,3	20,6 MB

Precision, Recall, F1 tính cho lớp dương (mắc bệnh), ngưỡng 0,5; chữ đậm: tốt nhất cột.

Bảng 2.5. Kết quả các mô hình học máy cơ bản trên tập giá nhà Việt Nam.

Mô hình	MAE (tỷ đ)	RMSE (tỷ đ)	R^2	Huấn luyện (s)	Dung lượng
Linear Regression (OLS)	1,150	1,492	0,5436	0,3	7,8 KB
Ridge ($\alpha = 1$)	1,150	1,492	0,5437	0,2	6,8 KB
SVR nhân RBF	1,024	1,380	0,6095	27,8	970,2 KB
Random Forest (200 cây)	**0,994**	**1,351**	**0,6258**	3,7	46,5 MB

Chữ đậm: tốt nhất cột.

Bảng 2.6. Kết quả các mô hình học máy cơ bản trên tập nhận xét khách hàng (TF-IDF 5.000 chiều).

Mô hình	Acc (%)	Precision	Recall	F1	Huấn luyện (s)	Dung lượng
Logistic Regression + TF-IDF	79,02	0,6055	**0,6653**	**0,6273**	1,7	169,9 KB
Linear SVM (LinearSVC)	81,70	0,6181	0,6278	0,6227	1,7	169,9 KB
Random Forest + TF-IDF	**82,09**	**0,6249**	0,5661	0,5850	4,7	30,1 MB

Precision, Recall, F1 lấy trung bình macro trên ba lớp; chữ đậm: tốt nhất cột.

2.3.6 Phân tích kết quả và giới hạn của các mô hình cơ bản

1. **Mô hình tuyến tính.** Trên tập tiểu đường, Logistic Regression có trọng số cân bằng đạt Recall 0,8941 nhưng Precision chỉ 0,4266: trọng số lớp đẩy ngưỡng quyết định về phía bắt nhiều ca bệnh, đổi lại hơn một nửa cảnh báo là báo động giả. ROC-AUC 0,9629 cho thấy khả năng *xếp hạng* vẫn tốt; điểm yếu nằm ở ranh giới tuyến tính không bắt được ngưỡng gãy của HbA1c. Trên tập giá nhà, OLS và Ridge cùng chỉ đạt $R^2 \approx 0{,}5436$ (MAE 1,150 tỷ): phạt L2 gần như không giúp gì vì vấn đề là thiếu khớp chứ không phải đa cộng tuyến.
2. **SVM nhân RBF.** SVC nâng ROC-AUC lên 0,9718 và F1 lên 0,5997, nhưng huấn luyện mất 93,09 s trên 80.000 mẫu, gấp hàng trăm lần Logistic Regression, và suy luận chậm nhất (1,140 ms mỗi mẫu) vì phải so mẫu mới với mọi vector hỗ trợ. SVR đạt $R^2 = 0{,}6095$ trên giá nhà. Với văn bản, LinearSVC huấn luyện trong 1,725 s và đạt Accuracy 81,70 %.
3. **Random Forest.** Là mô hình cơ bản mạnh nhất trên cả hai tập bảng: F1 0,7755 trên tiểu đường và $R^2 = 0{,}6258$, MAE 0,9940 tỷ trên giá nhà. Đổi lại, tệp mô hình nặng 47.583 KB (200 cây sâu), không ngoại suy được ra ngoài khoảng giá đã thấy, và trên văn bản 5.000 chiều thưa thì kém hơn mô hình tuyến tính (F1 macro 0,5850) vì mỗi lần chia chỉ xét một từ.
4. **Động lực chuyển sang MLP.** Mô hình tuyến tính thiếu khả năng biểu diễn, SVM nhân không mở rộng theo số mẫu, Random Forest cồng kềnh. Mạng nơ-ron đa tầng hứa hẹn cả ba: xấp xỉ được hàm phi tuyến bất kỳ (định lý xấp xỉ phổ quát), huấn luyện theo lô nhỏ nên chi phí tuyến tính theo N và chạy song song trên GPU, tệp mô hình chỉ vài chục đến vài trăm KB.

2.4 Mạng nơ-ron đa tầng (MLP) trên ba nền tảng: NumPy, Keras, PyTorch

2.4.1 Cơ sở lý thuyết và kiến trúc mạng

Một MLP gồm các tầng kết nối đầy đủ. Tầng l nhận kích hoạt $\mathbf{a}^{[l-1]}$ của tầng trước và tính

$$\mathbf{z}^{[l]} = W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \qquad \mathbf{a}^{[l]} = g^{[l]}\big(\mathbf{z}^{[l]}\big), \tag{2.9}$$

với g là ReLU ở tầng ẩn và sigmoid, softmax hoặc hàm đồng nhất ở tầng ra tùy bài toán. Lan truyền ngược áp dụng quy tắc dây chuyền từ tầng ra về tầng vào:

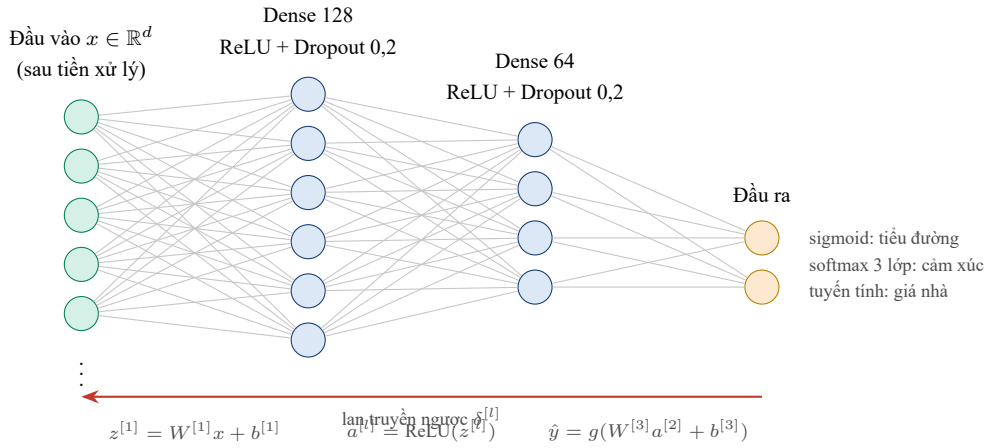
$$\boldsymbol{\delta}^{[L]} = \hat{\mathbf{y}} - \mathbf{y}, \tag{2.10}$$

$$\boldsymbol{\delta}^{[l]} = \big((W^{[l+1]})^{\top}\boldsymbol{\delta}^{[l+1]}\big) \odot g'^{[l]}\big(\mathbf{z}^{[l]}\big), \tag{2.11}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \boldsymbol{\delta}^{[l]}\big(\mathbf{a}^{[l-1]}\big)^{\top}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \boldsymbol{\delta}^{[l]}. \tag{2.12}$$

Đẳng thức đầu đúng cho cả ba cặp đầu ra – mất mát dùng ở đây (sigmoid + BCE, softmax + entropy chéo, tuyến tính + MSE với hệ số 2), nên một hàm `backward` phục vụ cả ba bài toán. Trọng số khởi tạo theo He [31] ($\mathcal{N}(0, 2/n_{in})$) để biên độ kích hoạt ổn định qua các tầng ReLU, và cập nhật bằng Adam [32].

Để so sánh ba nền tảng một cách công bằng, cả ba cài đặt dùng *cùng một kiến trúc* (Hình 2.10): hai tầng ẩn 128 và 64 nơ-ron, ReLU, Dropout 0,2 [14]; Adam với tốc độ học 10^{-3}, lô 256; dừng sớm khi mất mát kiểm định không giảm trong 8 epoch và khôi phục trọng số tốt nhất.



Hình 2.10. Kiến trúc MLP dùng chung cho cả ba nền tảng và ba bài toán.

2.4.2 Hiện thực từ đầu bằng NumPy

Lớp `MLPScratch` chỉ dùng phép toán ma trận của NumPy. Lan truyền tiến (Đoạn mã 2.6) lưu lại **z**, **a** và mặt nạ Dropout của từng tầng; lan truyền ngược (Đoạn mã 2.7) dùng lại chúng để tính gradient đúng như các công thức trên; Adam (Đoạn mã 2.8) giữ hai mômen cho mỗi tham số và hiệu chỉnh độ chệch khởi đầu.

```python
# tl/mlp_scratch.py
def forward(self, X, train=False):
    a, self.cache = X, []
    for i, (W, b) in enumerate(zip(self.W, self.b)):
        z = a @ W + b
        last = i == len(self.W) - 1
        if last:
            a = self._out(z)
            mask = None
        else:
            a = np.maximum(z, 0)
            # Dropout ngược: chia cho (1 - p) lúc huấn luyện để lúc suy luận không phải nhân lại
            mask = (self.rng.random(a.shape) >= self.p) / (1 - self.p) if train and self.p > 0 else None
            if mask is not None:
                a = a * mask
        self.cache.append((z, a, mask))
    self.X_in = X
    return a
```

Đoạn mã 2.6. Lan truyền tiến của MLP viết tay bằng NumPy, kèm Dropout ngược

```python
# tl/mlp_scratch.py
def backward(self, y):
    m = len(y)
    y_hat = self.cache[-1][1]
    if self.task == "multi":
        delta = y_hat.copy()
        delta[np.arange(m), y] -= 1
    else:
        delta = y_hat - y.reshape(-1, 1)
    if self.task == "reg":
        delta = 2 * delta                                   # đạo hàm của (y_hat - y)^2
    delta /= m
    grads_W, grads_b = [None] * len(self.W), [None] * len(self.W)
    for i in reversed(range(len(self.W))):
        a_prev = self.X_in if i == 0 else self.cache[i - 1][1]
        grads_W[i] = a_prev.T @ delta if not sp.issparse(a_prev) else (a_prev.T @ delta)
        grads_b[i] = delta.sum(0, keepdims=True)
        if i > 0:
            z_prev, _, mask = self.cache[i - 1]
            delta = (delta @ self.W[i].T) * (z_prev > 0)    # quy tắc dây chuyền qua ReLU
            if mask is not None:
                delta = delta * mask
    self._adam(grads_W + grads_b)
```

Đoạn mã 2.7. Lan truyền ngược của MLP viết tay: một sai số tầng ra chung cho ba bài toán

```python
# tl/mlp_scratch.py
def _adam(self, grads, b1=0.9, b2=0.999, eps=1e-8):
    self.t += 1
    params = self.W + self.b
    for k, (p, g) in enumerate(zip(params, grads)):
        g = np.asarray(g, dtype=np.float32)
        self.m[k] = b1 * self.m[k] + (1 - b1) * g
        self.v[k] = b2 * self.v[k] + (1 - b2) * g * g
        m_hat = self.m[k] / (1 - b1 ** self.t)
        v_hat = self.v[k] / (1 - b2 ** self.t)
        p -= self.lr * m_hat / (np.sqrt(v_hat) + eps)       # cập nhật tại chỗ
```

Đoạn mã 2.8. Bộ tối ưu Adam viết tay

2.4.3 Hiện thực bằng Keras / TensorFlow

Keras gói cả kiến trúc, hàm mất mát và vòng huấn luyện vào API cấp cao; phần đầu ra chọn theo bảng `HEADS`.

```python
# tl/mlp_keras.py
def build_keras_mlp(input_dim, task="binary", n_classes=3, hidden=(128, 64), dropout=0.2, lr=1e-3):
    units, act, loss = HEADS[task]
    model = keras.Sequential([keras.Input(shape=(input_dim,))])
    for h in hidden:
        model.add(layers.Dense(h, activation="relu"))
        model.add(layers.Dropout(dropout))
    model.add(layers.Dense(units or n_classes, activation=act))
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr), loss=loss)
    return model
```

Đoạn mã 2.9. MLP bằng Keras Sequential API cho ba loại đầu ra

2.4.4 Hiện thực bằng PyTorch

PyTorch tách mô hình (`nn.Module`) khỏi vòng huấn luyện do ta tự viết. Mô hình trả logit thô; sigmoid và softmax nằm trong `BCEWithLogitsLoss` và `CrossEntropyLoss` để ổn định số học. Vì dữ liệu bảng nhỏ, toàn bộ tập được đưa

lên GPU một lần và chia lô bằng chỉ số thay vì `DataLoader`.

```python
# tl/mlp_torch.py
class TorchMLP(nn.Module):
    def __init__(self, input_dim, output_dim=1, hidden=(128, 64), dropout=0.2):
        super().__init__()
        layers, d = [], input_dim
        for h in hidden:
            layers += [nn.Linear(d, h), nn.ReLU(), nn.Dropout(dropout)]
            d = h
        layers.append(nn.Linear(d, output_dim))   # logit / giá trị thô; sigmoid, softmax nằm trong hàm mất mát
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)
```

Đoạn mã 2.10. MLP bằng PyTorch với lớp `nn.Module`

```python
# tl/mlp_torch.py
def train_epoch(model, X, y, criterion, optimizer, batch=256):
    model.train()
    order = torch.randperm(len(X), device=DEVICE)
    total = 0.0
    for s in range(0, len(X), batch):
        idx = order[s:s + batch]
        optimizer.zero_grad()
        loss = criterion(model(X[idx]), y[idx])
        loss.backward()
        optimizer.step()
        total += loss.item() * len(idx)
    return total / len(X)
```

Đoạn mã 2.11. Một epoch huấn luyện PyTorch trên dữ liệu đã nằm sẵn trên GPU

2.5 Đánh giá và đối chuẩn trên ba tập dữ liệu

2.5.1 Đối chuẩn hiệu năng chi tiết trên từng tập dữ liệu

Ba bảng sau đặt ba bản MLP cạnh các mô hình cơ bản, trên cùng tập kiểm thử.

Bảng 2.7. Đối chuẩn mọi mô hình trên tập tiểu đường.

Mô hình	Acc (%)	Precision	Recall	F1	ROC-AUC	Huấn luyện (s)	Dung lượng
Logistic Regression (balanced)	88,89	0,4266	0,8941	0,5776	0,9629	0,1	1,9 KB
SVM nhân RBF (balanced)	89,58	0,4451	**0,9188**	0,5997	0,9718	93,1	308,2 KB
Random Forest (200 cây)	96,39	0,8226	0,7335	0,7755	0,9738	1,3	20,6 MB
MLP NumPy (scratch)	97,21	**0,9957**	0,6747	0,8043	0,9775	11,4	81,7 KB
MLP Keras	**97,21**	0,9897	0,6794	**0,8057**	**0,9776**	24,5	150,8 KB
MLP PyTorch	97,18	0,9781	0,6835	0,8047	0,9775	23,7	43,7 KB

Precision, Recall, F1 tính cho lớp dương (mắc bệnh), ngưỡng 0,5; chữ đậm: tốt nhất cột.

Bảng 2.8. Đối chuẩn mọi mô hình trên tập giá nhà Việt Nam.

Mô hình	MAE (tỷ đ)	RMSE (tỷ đ)	R^2	Huấn luyện (s)	Dung lượng
Linear Regression (OLS)	1,150	1,492	0,5436	0,3	7,8 KB
Ridge ($\alpha = 1$)	1,150	1,492	0,5437	0,2	6,8 KB
SVR nhân RBF	1,024	1,380	0,6095	27,8	970,2 KB
Random Forest (200 cây)	**0,994**	**1,351**	**0,6258**	3,7	46,5 MB
MLP NumPy (scratch)	1,032	1,364	0,6185	5,6	185,7 KB
MLP Keras	1,027	1,354	0,6240	11,1	306,8 KB
MLP PyTorch	1,024	1,355	0,6233	6,4	95,7 KB

MLP học nhãn đã chuẩn hóa z-score rồi đổi ngược về tỷ đồng; chữ đậm: tốt nhất cột.

Bảng 2.9. Đối chuẩn mọi mô hình trên tập nhận xét khách hàng.

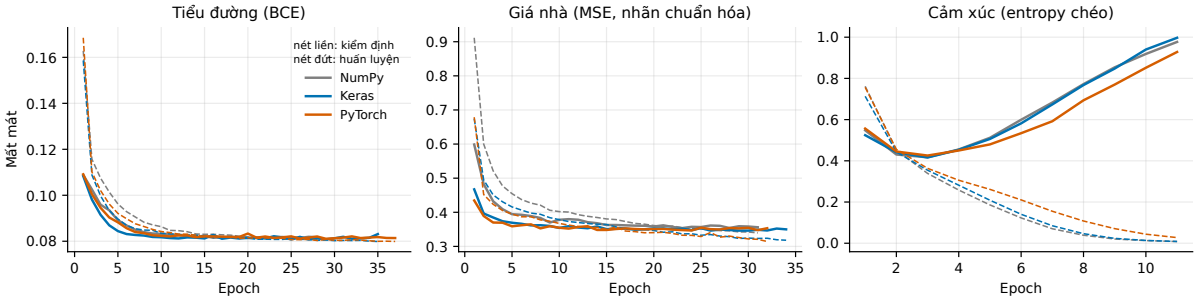
Mô hình	Acc (%)	Precision	Recall	F1	Huấn luyện (s)	Dung lượng
Logistic Regression + TF-IDF	79,02	0,6055	**0,6653**	0,6273	1,7	169,9 KB
Linear SVM (LinearSVC)	81,70	0,6181	0,6278	0,6227	1,7	169,9 KB
Random Forest + TF-IDF	82,09	0,6249	0,5661	0,5850	4,7	30,1 MB
MLP NumPy (scratch)	**83,95**	**0,6575**	0,6317	0,6353	22,6	4,9 MB
MLP Keras	83,57	0,6488	0,6299	**0,6354**	5,2	7,5 MB
MLP PyTorch	83,64	0,6413	0,6177	0,6191	1,8	2,5 MB

Precision, Recall, F1 lấy trung bình macro trên ba lớp; chữ đậm: tốt nhất cột.

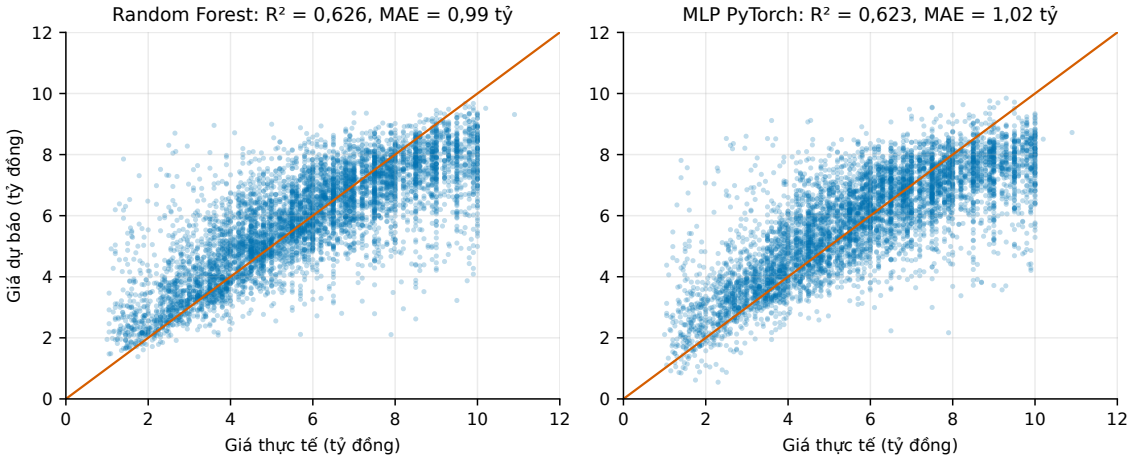
Bảng 2.10. Cùng một kiến trúc MLP trên ba nền tảng.

Tập dữ liệu	Nền tảng	F1 / R^2	Tham số	Epoch	Huấn luyện (s)	Dung lượng
Tiểu đường	NumPy (scratch)	0,8043	10.369	28	11,4	81,7 KB
	Keras	0,8057	10.369	35	24,5	150,8 KB
	PyTorch	0,8047	10.369	37	23,7	43,7 KB
Giá nhà	NumPy (scratch)	0,6185	23.681	31	5,6	185,7 KB
	Keras	0,6240	23.681	34	11,1	306,8 KB
	PyTorch	0,6233	23.681	32	6,4	95,7 KB
Cảm xúc	NumPy (scratch)	0,6353	648.579	11	22,6	4,9 MB
	Keras	0,6354	648.579	11	5,2	7,5 MB
	PyTorch	0,6191	648.579	11	1,8	2,5 MB

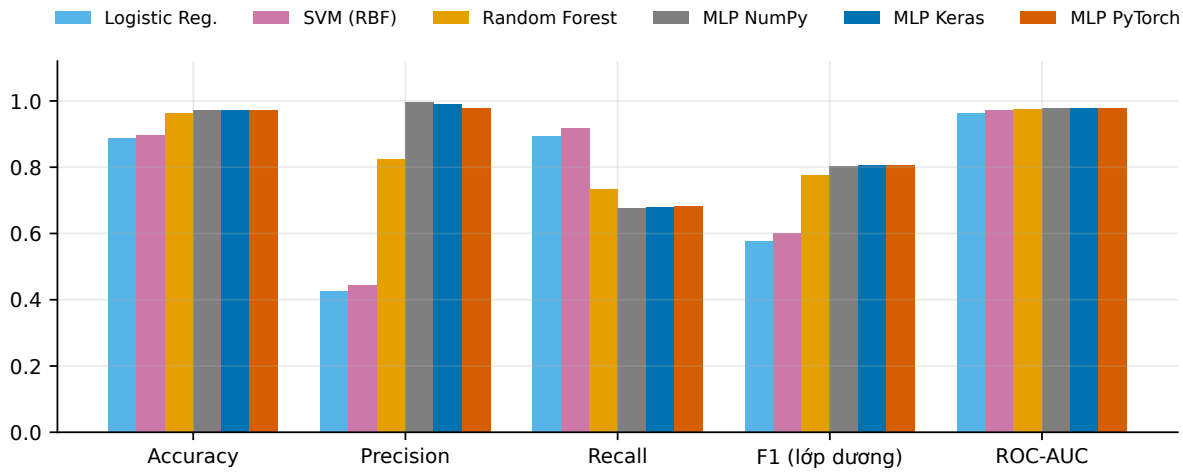
F1 cho tiểu đường (lớp dương) và cảm xúc (macro); R^2 cho giá nhà. NumPy và Keras chạy CPU, PyTorch chạy GPU; số epoch là số epoch thực chạy trước khi dừng sớm.



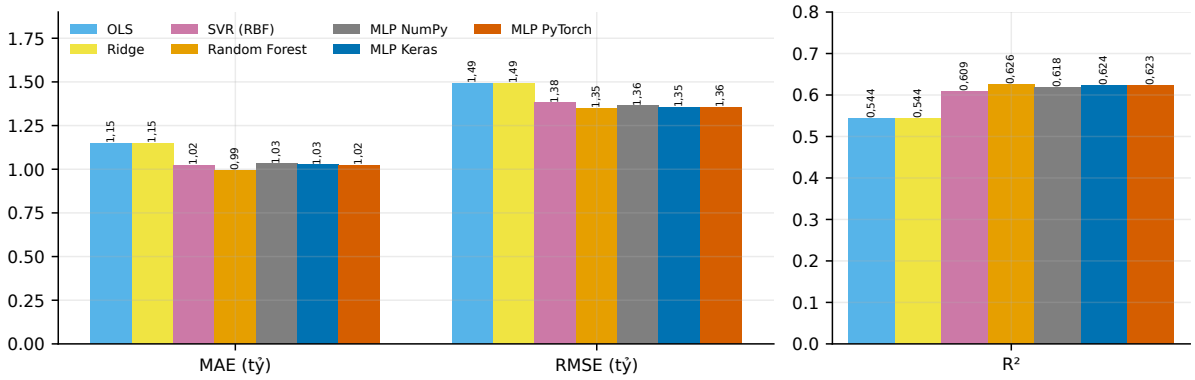
Hình 2.11. Đường cong mất mát huấn luyện (nét đứt) và kiểm định (nét liền) của MLP trên ba nền tảng.



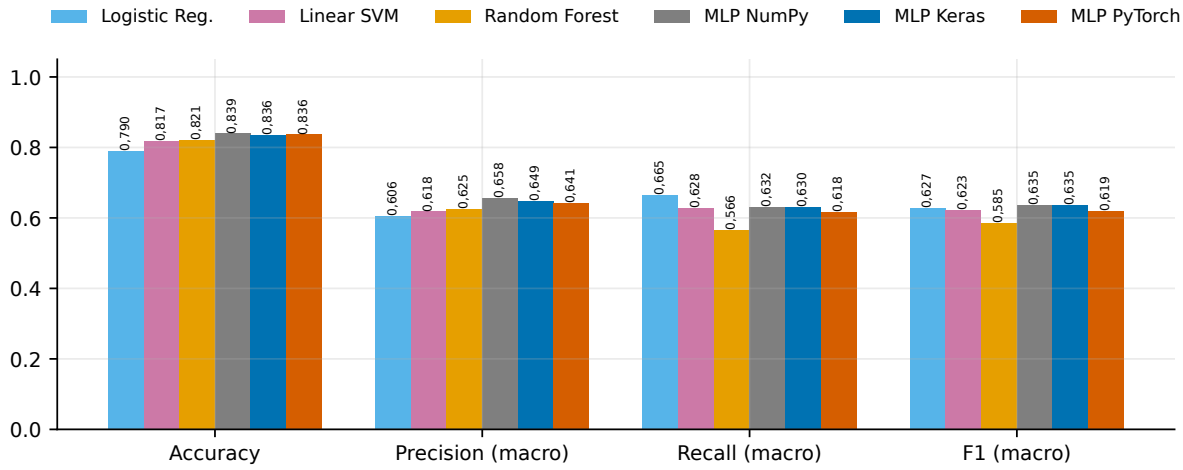
Hình 2.12. Giá thực tế và giá dự báo trên tập kiểm thử của Random Forest và MLP PyTorch.



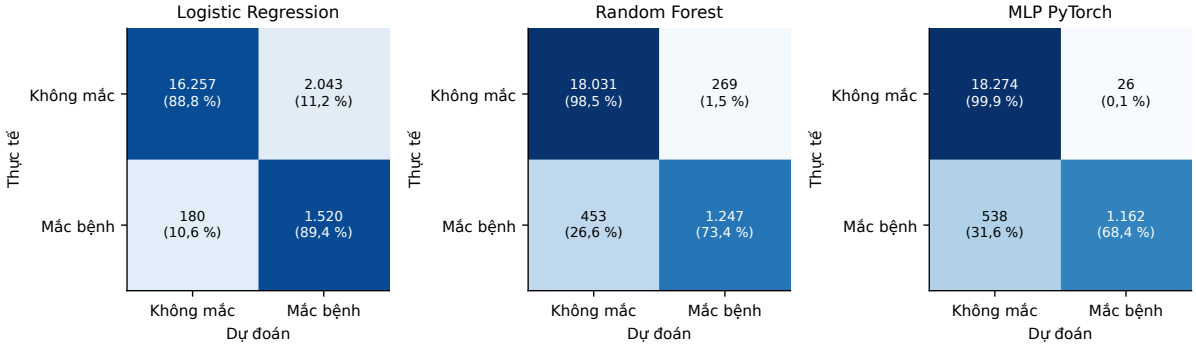
Hình 2.13. So sánh đa tiêu chí trên tập tiểu đường.



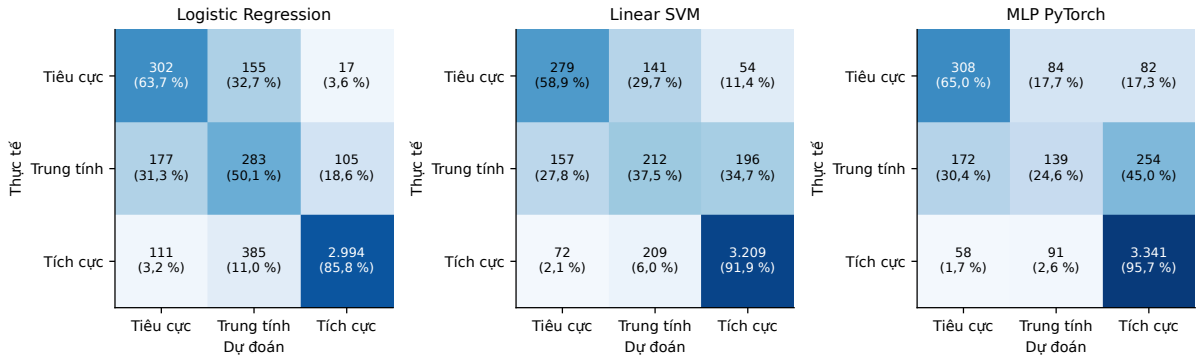
Hình 2.14. So sánh sai số (MAE, RMSE) và hệ số xác định R^2 trên tập giá nhà.



Hình 2.15. So sánh Accuracy và các chỉ số macro trên tập nhận xét khách hàng.



Hình 2.16. Ma trận nhầm lẫn trên tập kiểm thử tiểu đường: Logistic Regression, Random Forest, MLP PyTorch.



Hình 2.17. Ma trận nhầm lẫn trên tập kiểm thử nhận xét khách hàng.

Thảo luận kết quả.

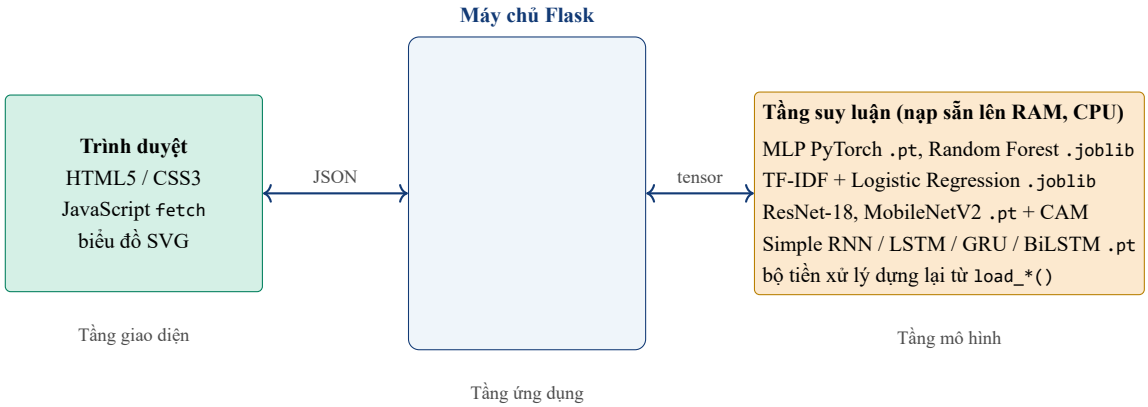
- **MLP tốt nhất trên dữ liệu y tế.** Cả ba bản MLP đạt F1 lớp dương quanh 0,8047 (PyTorch), cao hơn Random Forest (0,7755) và vượt xa Logistic Regression (0,5776). Ma trận nhầm lẫn (Hình 2.16) cho thấy MLP gần như không báo động giả (Precision 0,9781) nhưng vẫn bỏ sót khoảng một phần ba ca bệnh (Recall 0,6835). ROC-AUC của bốn mô hình phi tuyến chỉ chênh nhau ở chữ số thứ ba (0,9718 – 0,9775): khác biệt lớn về F1 phần nhiều đến từ *điểm cắt* trên đường ROC chứ không từ khả năng phân biệt. Trong sàng lọc y tế, bỏ sót đắt hơn báo động giả, nên khi triển khai nên hạ ngưỡng của MLP xuống dưới 0,5 để đổi Precision lấy Recall.
- **Random Forest vẫn nhỉnh nhất trên giá nhà.** R^2 của Random Forest (0,6258) cao hơn MLP PyTorch (0,6233) một chút, hai mô hình bám nhau trên đồ thị phân tán (Hình 2.12). Cả hai đều chỉ giải thích khoảng 62 % phương sai: phần còn lại nằm ở thông tin dữ liệu không có (vị trí chính xác trong quận, dự án, chất lượng xây dựng), nên không mô hình nào vượt xa được. Cây quyết định chia trực tiếp không gian theo từng quận/huyện, một lợi thế tự nhiên trên dữ liệu bảng có nhiều biến phân loại.
- **Văn bản: MLP chính xác nhất nhưng lớp trung tính là điểm mù chung.** MLP đạt Accuracy cao nhất (83,95 % với bản NumPy), F1 macro ngang các mô hình tuyến tính. Cả ba bản MLP quá khớp rất nhanh: mất mát kiểm định chạm đáy ở epoch 2 – 3 rồi tăng (Hình 2.11), vì 648.579 tham số học trên chưa đến 17.000 văn bản. Ma trận nhầm lẫn (Hình 2.17) cho thấy lớp trung tính (3 sao) bị lẫn sang cả hai phía: một nhận xét 3 sao thường vừa khen vừa chê.
- **Ba nền tảng cho kết quả tương đương.** Với cùng kiến trúc và siêu tham số, chênh lệch F1/R^2 giữa NumPy, Keras và PyTorch nằm trong khoảng dao động do khởi tạo ngẫu nhiên khác nhau (Bảng 2.10). Bản NumPy tự viết đạt độ chính xác ngang thư viện công nghiệp, xác nhận các công thức lan truyền ngược được cài đặt đúng. Khác biệt nằm ở chi phí kỹ thuật: PyTorch chạy GPU, Keras có tệp lưu lớn nhất do chứa cả trạng thái bộ tối ưu.

2.6 Đóng gói triển khai và giao diện Web thực tế

2.6.1 Kiến trúc triển khai hệ thống

Toàn bộ mô hình của ba chương được phục vụ bởi một ứng dụng Flask duy nhất gồm ba tầng (Hình 2.18):

1. **Tầng giao diện:** một trang HTML5/CSS3 với sáu thẻ, JavaScript gọi API bằng `fetch` và vẽ biểu đồ bằng SVG thuần, không phụ thuộc thư viện ngoài.
2. **Tầng ứng dụng:** máy chủ Flask nhận yêu cầu `POST` JSON tại sáu endpoint `/api/diabetes`, `/api/house`, `/api/sentiment`, `/api/vision`, `/api/stock`, `/api/churn`.
3. **Tầng mô hình:** mọi mô hình (`.pt`, `.joblib`) được nạp lên RAM một lần lúc khởi động và suy luận trên CPU. Bộ tiền xử lý được dựng lại bằng chính hàm `load_*` đã dùng khi huấn luyện nên dữ liệu người dùng đi qua đúng phép biến đổi mà mô hình đã học.



Hình 2.18. Kiến trúc ba tầng của Web App phục vụ sáu phân hệ thông minh.

Endpoint sàng lọc tiểu đường (Đoạn mã 2.12) minh họa khuôn chung: dựng một dòng `DataFrame` từ JSON, biến đổi bằng bộ tiền xử lý, chạy mô hình, đo thời gian và trả JSON.

```python
# app/app.py
@app.post("/api/diabetes")
def api_diabetes():
    f = request.get_json()
    row = pd.DataFrame([{k: f[k] for k in dt.DIAB_NUM + dt.DIAB_BIN + dt.DIAB_CAT}])
    x = torch.as_tensor(DIAB["pre"].transform(row).astype(np.float32))

    def run():
        with torch.no_grad():
            p_mlp = torch.sigmoid(diab_mlp(x)).item()
        return p_mlp, float(diab_rf.predict_proba(row)[0, 1])
    (p_mlp, p_rf), ms = timed(run)
    level = "cao" if p_mlp >= 0.5 else "trung bình" if p_mlp >= 0.2 else "thấp"
    advice = {"cao": "Nên làm xét nghiệm đường huyết lúc đói và HbA1c tại cơ sở y tế trong thời gian sớm.",
              "trung bình": "Nên theo dõi chế độ ăn, vận động và kiểm tra lại chỉ số định kỳ 6 tháng.",
              "thấp": "Nguy cơ thấp; duy trì lối sống lành mạnh và khám sức khỏe định kỳ."}[level]
    return jsonify(p_mlp=p_mlp, p_rf=p_rf, level=level, advice=advice, ms=ms)
```

Đoạn mã 2.12. Endpoint sàng lọc tiểu đường: MLP PyTorch và Random Forest cho cùng một hồ sơ

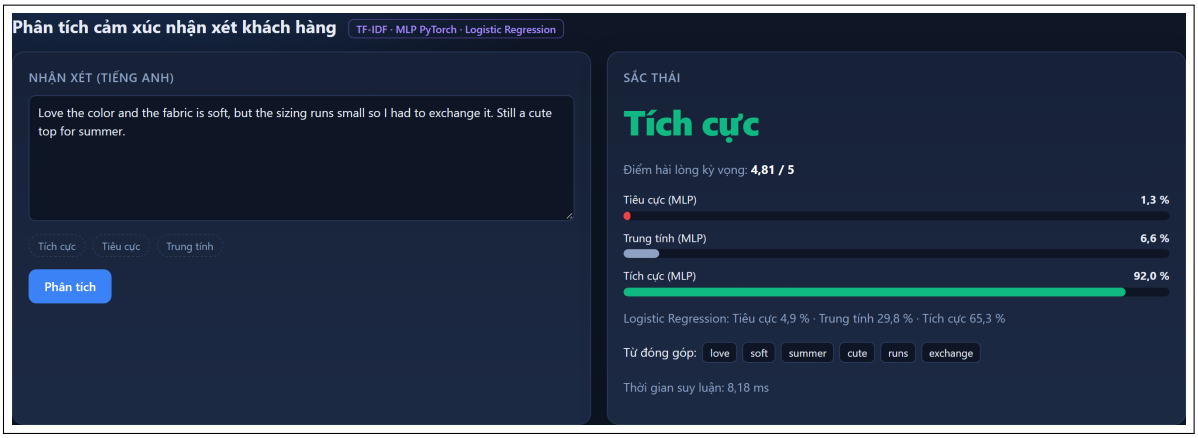
2.6.2 Giao diện thực tế các hệ thống thông minh



Hình 2.19. Phân hệ sàng lọc nguy cơ tiểu đường: người dùng nhập chỉ số lâm sàng, hệ thống trả xác suất của MLP và Random Forest, mức nguy cơ và khuyến nghị.



Hình 2.20. Phân hệ định giá bất động sản: giá ước tính của Random Forest, khoảng hợp lý ± RMSE kiểm thử, dự báo của MLP và đơn giá theo m².



Hình 2.21. Phân hệ phân tích cảm xúc nhận xét: xác suất ba lớp của MLP, đối chiếu Logistic Regression và các từ đóng góp nhiều nhất.

Ba phân hệ trả kết quả trong vài đến vài chục mili-giây trên CPU, cho thấy các mô hình học máy cơ bản và MLP chuyển được trực tiếp vào một quy trình nghiệp vụ thực tế.

Chương 3

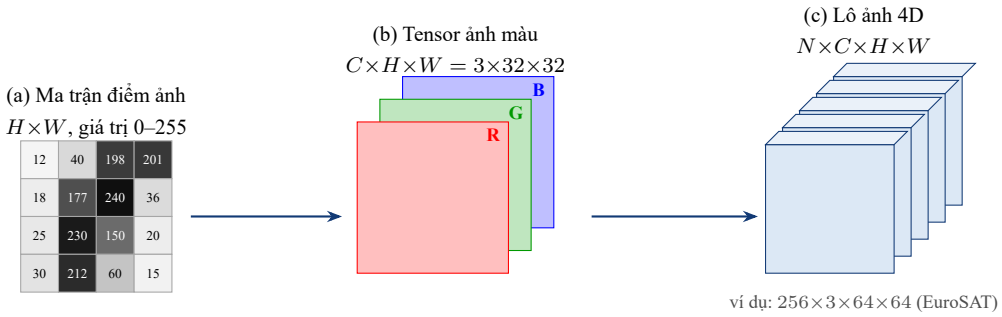
Mạng Nơ-ron Tích Chập (CNN)

3.1 Cơ sở lý thuyết mạng nơ-ron tích chập và biểu diễn không gian

MLP của Chương 2 gặp hai trở ngại khi nhận đầu vào là ảnh:

1. **Bùng nổ tham số.** Duỗi một ảnh màu $224\times224\times3$ thành vector 150.528 chiều, chỉ một tầng ẩn 1.024 nơ-ron đã cần hơn 154 triệu trọng số, vừa tốn bộ nhớ vừa dễ quá khớp.
2. **Mất cấu trúc không gian.** Duỗi phẳng xóa thông tin "điểm nào cạnh điểm nào"; cùng một nét chữ dịch sang phải hai điểm ảnh trở thành một vector hoàn toàn khác.

Mạng nơ-ron tích chập [12] khắc phục bằng ba nguyên lý: *vùng tiếp nhận cục bộ* (mỗi nơ-ron chỉ nhìn một ô nhỏ), *chia sẻ trọng số* (cùng một bộ lọc quét toàn ảnh, nên một mẫu hình được nhận ra ở mọi vị trí) và *lấy mẫu giảm* (pooling). Đầu vào được biểu diễn thành tensor (Hình 3.1).



Hình 3.1. Biểu diễn ảnh số: từ ma trận điểm ảnh đến tensor ba kênh và lô ảnh 4D trong học sâu.

3.1.1 Toán học của phép tích chập hai chiều

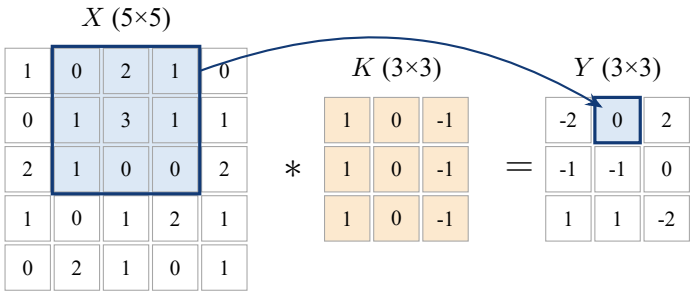
Trong học sâu, "tích chập" thực chất là tương quan chéo. Với tensor vào $X \in \mathbb{R}^{C_{in}\times H\times W}$ và C_{out} bộ lọc $K \in \mathbb{R}^{C_{out}\times C_{in}\times k_h\times k_w}$, giá trị tại vị trí (i, j) của kênh ra c là

$$Y_{c,i,j} = \sum_{m=1}^{C_{in}} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} X_{m,\, is+u,\, js+v}\, K_{c,m,u,v} + b_c, \tag{3.1}$$

với bước trượt s. Kích thước bản đồ đặc trưng ra phụ thuộc đệm số 0 p:

$$H_{out} = \left\lfloor \frac{H - k_h + 2p}{s} \right\rfloor + 1, \qquad W_{out} = \left\lfloor \frac{W - k_w + 2p}{s} \right\rfloor + 1. \tag{3.2}$$

Hình 3.2 tính tay một ví dụ với bộ lọc phát hiện cạnh dọc. Số tham số của một tầng chỉ là $C_{out}(C_{in}k_hk_w + 1)$, không phụ thuộc kích thước ảnh.



$Y_{0,1} = \sum_{u=0}^{2}\sum_{v=0}^{2} X_{u,\,1+v}\,K_{u,v} = (0-1)+(1-1)+(1-0) = 0$: tổng tích của vùng tô xanh với bộ lọc. Bộ lọc này cho giá trị lớn khi cột trái sáng hơn cột phải, tức là phát hiện *cạnh dọc*; trượt cửa sổ (bước $s = 1$, không đệm) qua mọi vị trí được bản đồ đặc trưng 3×3.

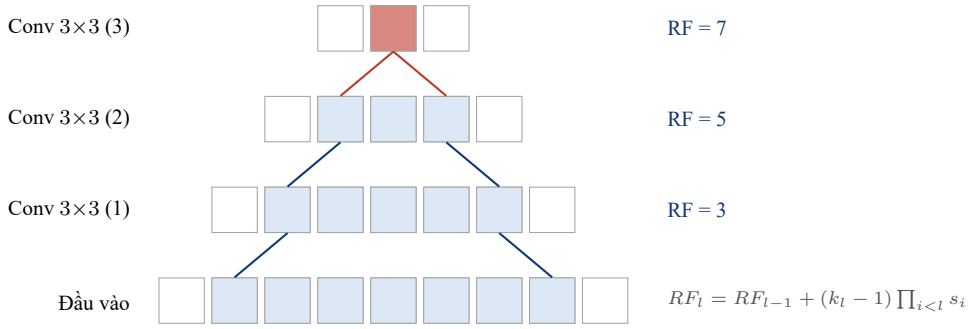
Hình 3.2. Minh họa số học từng bước của phép tích chập 2D: ảnh 5×5, bộ lọc cạnh dọc 3×3, bước 1.

3.1.2 Vùng cảm nhận và phép gộp (pooling)

Vùng cảm nhận (receptive field, RF) là vùng ảnh vào mà một nơ-ron ở tầng sâu "nhìn thấy". Xếp chồng các tầng $k\times k$ làm RF tăng dần:

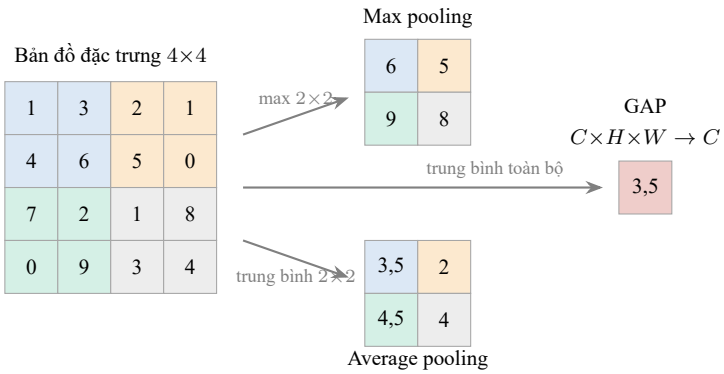
$$RF_l = RF_{l-1} + (k_l - 1)\prod_{i=1}^{l-1} s_i. \tag{3.3}$$

Với bước 1, ba tầng 3×3 cho RF $= 7$ (Hình 3.3); mỗi lần pooling bước 2 nhân đôi tốc độ tăng của RF ở các tầng sau. Đây là lý do VGG chỉ dùng bộ lọc 3×3: hai tầng 3×3 nhìn được vùng 5×5 với 18 thay vì 25 trọng số mỗi cặp kênh và có hai hàm ReLU thay vì một.



Hình 3.3. Cơ chế tích lũy vùng cảm nhận khi xếp chồng ba tầng tích chập 3×3 bước 1.

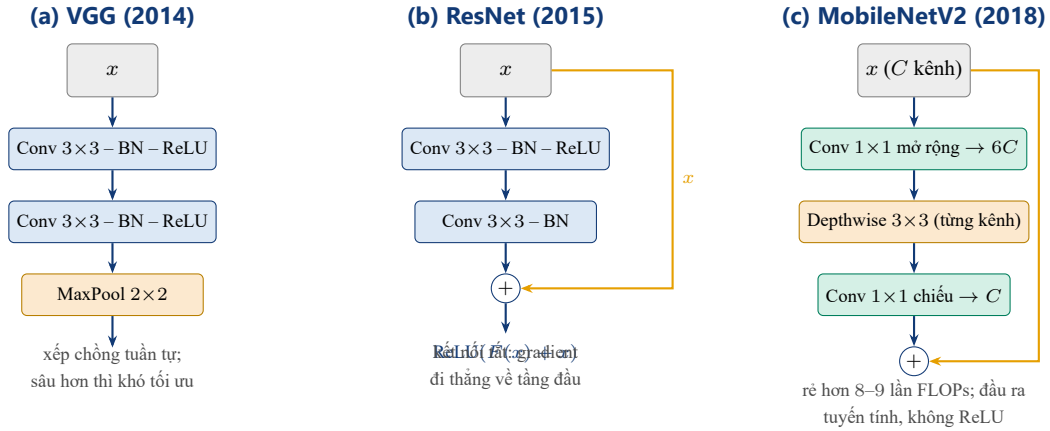
Pooling giảm kích thước không gian, tăng khả năng chịu dịch chuyển nhỏ và mở rộng nhanh vùng cảm nhận (Hình 3.4). **Max pooling** giữ giá trị lớn nhất trong cửa sổ (đặc trưng mạnh nhất); **average pooling** lấy trung bình. **Global Average Pooling** (GAP) [33] lấy trung bình toàn bộ mỗi bản đồ đặc trưng, biến $C\times H\times W$ thành vector C chiều, thay cho tầng Flatten + Dense khổng lồ. Mọi kiến trúc trong chương này kết thúc bằng GAP, nên cùng một mạng nhận được cả ảnh 32×32 lẫn 64×64.



Hình 3.4. So sánh max pooling, average pooling (2×2, bước 2) và Global Average Pooling trên cùng một bản đồ đặc trưng.

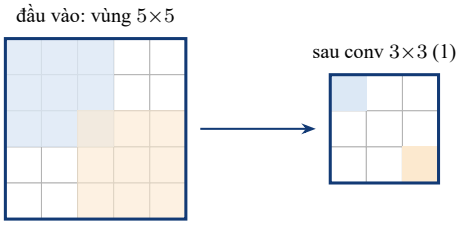
3.1.3 Tiến hóa kiến trúc: VGG, ResNet và MobileNet

Hình 3.5 tóm tắt ba thế hệ khối xây dựng được hiện thực trong chương này.



Hình 3.5. Sơ đồ tiến hóa khối xây dựng CNN: khối VGG phẳng, khối Residual và khối Inverted Residual (tích chập tách biệt chiều sâu).

VGG [15]. Các khối gồm hai hoặc ba tầng Conv 3×3 rồi MaxPool, số kênh nhân đôi sau mỗi lần giảm kích thước. Thiết kế đơn giản, đều đặn nhưng nhiều tham số và khó tối ưu khi sâu (Hình 3.6).



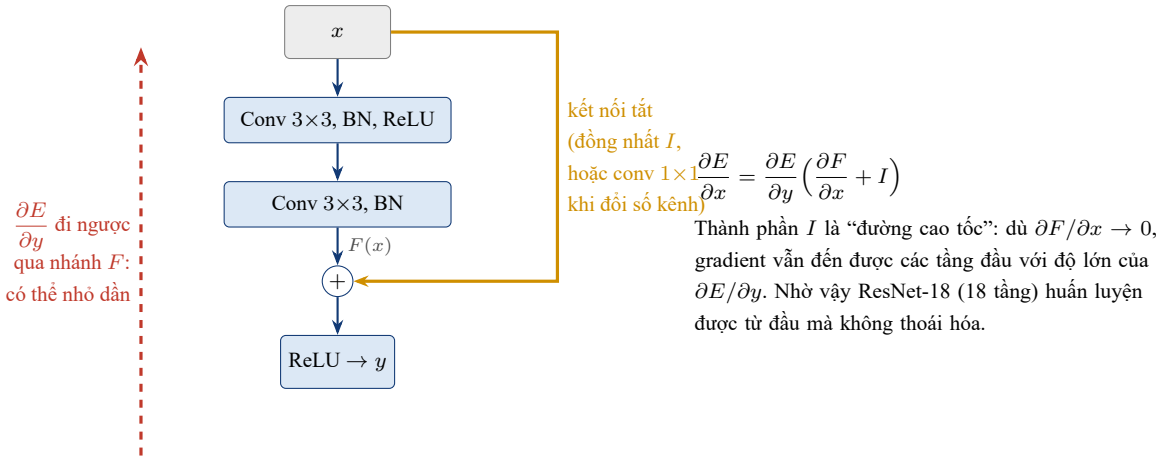
	1 conv 5×5	2 conv 3×3
vùng nhìn	5×5	5×5
trọng số (mỗi cặp kênh)	25	18 (−28 %)
hàm phi tuyến ReLU	1	2

Hình 3.6. Khối VGG: hai tầng 3×3 có cùng vùng nhìn 5×5 với một tầng 5×5 nhưng ít trọng số hơn.

ResNet [16]. Khi mạng sâu quá khoảng 20 tầng, sai số *huấn luyện* lại tăng (thoái hóa), không phải do quá khớp mà do tối ưu khó. Khối Residual học phần dư $F(\mathbf{x})$ rồi cộng với đầu vào, $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$. Theo quy tắc dây chuyền

$$\frac{\partial E}{\partial \mathbf{x}} = \frac{\partial E}{\partial \mathbf{y}}\left(\frac{\partial F}{\partial \mathbf{x}} + I\right), \tag{3.4}$$

ma trận đơn vị I là "đường cao tốc" để gradient về tới tầng đầu ngay cả khi $\partial F/\partial \mathbf{x}$ nhỏ (Hình 3.7).

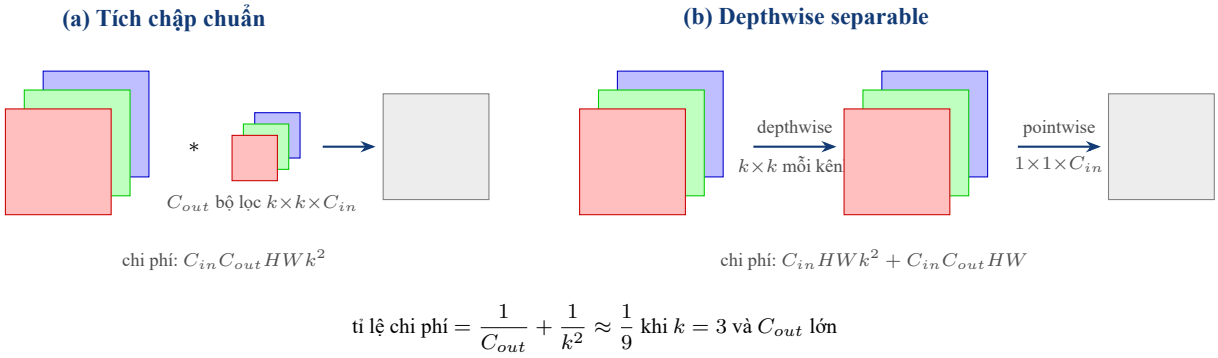


Hình 3.7. Khối Residual và cơ chế "đường cao tốc" của dòng gradient.

MobileNet [17, 18]. Tích chập tách biệt chiều sâu chia tích chập chuẩn thành *depthwise* (mỗi kênh một bộ lọc $k\times k$, chi phí $C_{in}HWk^2$) và *pointwise* (Conv 1×1 trộn kênh, chi phí $C_{in}C_{out}HW$). Tỉ lệ chi phí so với tích chập chuẩn là

$$\frac{C_{in}HWk^2 + C_{in}C_{out}HW}{C_{in}C_{out}HWk^2} = \frac{1}{C_{out}} + \frac{1}{k^2} \approx \frac{1}{9} \quad (k=3). \tag{3.5}$$

MobileNetV2 bọc phép này trong khối *Inverted Residual*: Conv 1×1 mở rộng số kênh lên 6 lần, depthwise 3×3 ở không gian rộng, rồi Conv 1×1 chiếu về ít kênh *không* qua ReLU (nút cổ chai tuyến tính, tránh mất thông tin ở không gian hẹp), kèm kết nối tắt khi kích thước giữ nguyên (Hình 3.8).



Hình 3.8. Tích chập chuẩn so với tích chập tách biệt chiều sâu: lọc không gian riêng từng kênh rồi trộn kênh bằng 1×1.

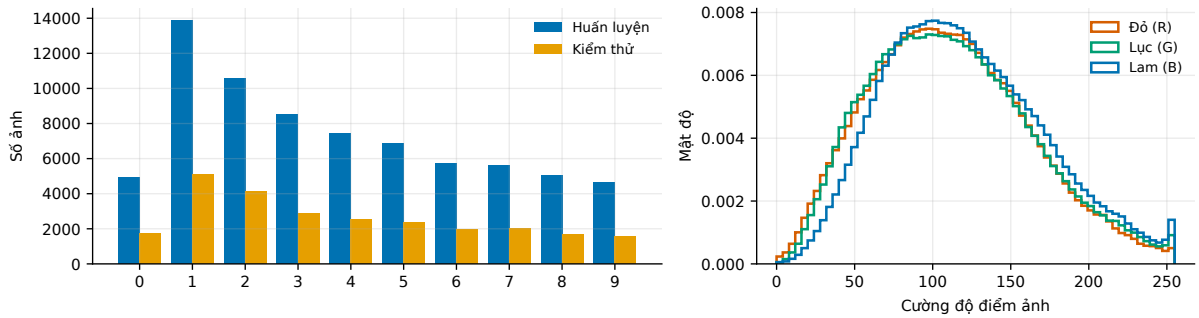
3.2 Khảo sát thực nghiệm và tiền xử lý ba bộ dữ liệu đa miền

3.2.1 Tập dữ liệu 1: nhận dạng chữ số nhà SVHN

- **Nguồn:** Street View House Numbers của Stanford [34], cắt từ ảnh Google Street View; tải qua lớp `SVHN` của `torchvision`.
- **Quy mô:** 73.257 ảnh huấn luyện và 26.032 ảnh kiểm thử theo cách chia gốc, ảnh màu $32\times32\times3$, 10 lớp chữ số 0 – 9.



Hình 3.9. Bốn ảnh mẫu mỗi lớp của SVHN: đa dạng phông chữ, ánh sáng, màu nền và có chữ số lân cận lọt vào khung.

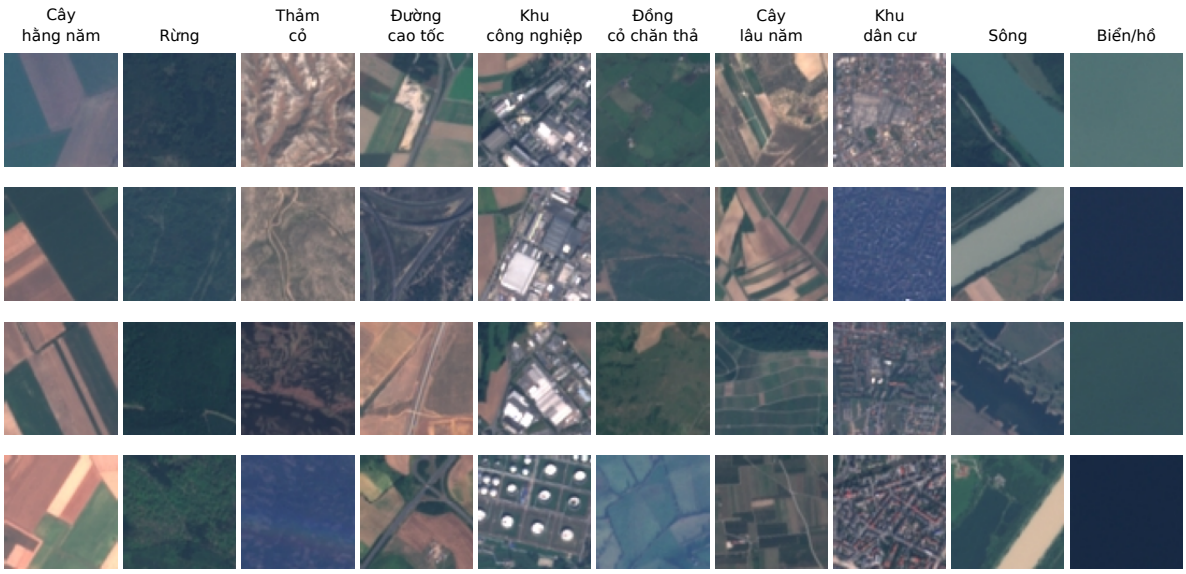


Hình 3.10. Trái: số ảnh theo lớp của SVHN. Phải: phân bố cường độ điểm ảnh trên ba kênh màu.

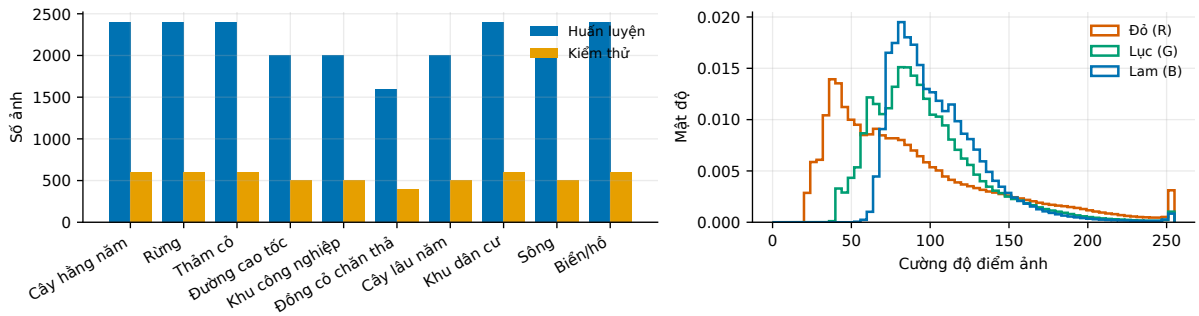
Nhận xét EDA. Khác MNIST viết tay trên nền trắng, ảnh SVHN có nền thật (tường, cửa, biển kim loại), độ tương phản thấp và thường có chữ số hàng xóm lọt vào hai bên (Hình 3.9). Phân bố lớp lệch: chữ số 1 nhiều nhất (13.861 ảnh), chữ số 9 ít nhất (4.659 ảnh), vì số nhà bắt đầu bằng 1 nhiều hơn (Hình 3.10). Ba kênh có phân bố gần nhau, trung bình $(0{,}4377; 0{,}4438; 0{,}4728)$ và độ lệch chuẩn $(0{,}1980; 0{,}2010; 0{,}1970)$, dùng để chuẩn hóa z-score từng kênh. Tăng cường dữ liệu chỉ dùng dịch ngẫu nhiên ± 4 điểm ảnh (đệm phản chiếu); *không* lật ảnh vì chữ số lật không còn là chữ số đó.

3.2.2 Tập dữ liệu 2: phân loại ảnh vệ tinh EuroSAT

- **Nguồn:** EuroSAT [35], ảnh vệ tinh Sentinel-2 của Cơ quan Vũ trụ châu Âu, bản RGB, tải qua lớp `EuroSAT` của `torchvision`.
- **Quy mô:** 27.000 ảnh $64 \times 64 \times 3$, 10 lớp sử dụng đất (cây hằng năm, rừng, thảm cỏ, đường cao tốc, khu công nghiệp, đồng cỏ chăn thả, cây lâu năm, khu dân cư, sông, biển/hồ). Không có tập kiểm thử chính thức nên chia phân tầng 80/20: 21.600 ảnh huấn luyện, 5.400 ảnh kiểm thử.



Hình 3.11. Bốn ảnh mẫu mỗi lớp của EuroSAT.



Hình 3.12. Trái: số ảnh theo lớp của EuroSAT. Phải: phân bố cường độ điểm ảnh trên ba kênh màu.

Nhận xét EDA. Các lớp gần cân bằng, từ 1.600 đến 2.400 ảnh huấn luyện mỗi lớp (Hình 3.12). Phân biệt lớp dựa nhiều vào *kết cấu* (vân ruộng, lưới đường phố) và *màu* (nước xanh thẫm, rừng xanh tối) hơn là hình dạng đối tượng. Ảnh chụp từ trên xuống không có chiều "trên dưới", nên ngoài dịch ngẫu nhiên còn lật ngang, lật dọc và chuyển vị (tương đương xoay $90°$) mà không đổi nhãn.

3.2.3 Tập dữ liệu 3: chuỗi đặc trưng lâm sàng cho 1D CNN

Để thấy tích chập không chỉ dành cho ảnh, ta áp dụng 1D CNN lên tập tiểu đường của Chương 2. Mười lăm đặc trưng sau tiền xử lý (4 biến số chuẩn hóa, 2 biến nhị phân, 9 cột one-hot) được xếp thành tensor $[B, 1, 15]$; hai tầng Conv1D kích thước 1×3 (32 và 64 bộ lọc) học tương tác giữa các đặc trưng kề nhau, rồi tầng Dense 64 và đầu ra sigmoid, tổng cộng 68.097 tham số. Mô hình dùng cùng cách chia và cùng dừng sớm như MLP.

3.3 Hiện thực hóa mã nguồn trên ba nền tảng

3.3.1 Tích chập 2D và Max Pooling viết tay bằng NumPy

Đoạn mã 3.1 cài đặt đúng công thức (3.1) bằng bốn vòng lặp để dễ đối chiếu. Cách này quá chậm cho cả mạng, nên Đoạn mã 3.2 dùng `sliding_window_view` để lấy mọi cửa sổ thành một tensor rồi nhân một lần bằng `einsum`, cũng là ý tưởng im2col mà các thư viện thật dùng.

```python
# tl/cnn_scratch.py
def conv2d_loop(X, W, b, stride=1, padding=0):
    """X: [N, Cin, H, W], W: [Cout, Cin, kh, kw], b: [Cout] -> [N, Cout, Hout, Wout]."""
    N, C_in, H, W_ = X.shape
    C_out, _, kh, kw = W.shape
    Xp = np.pad(X, ((0, 0), (0, 0), (padding, padding), (padding, padding))) if padding else X
    H_out = (H + 2 * padding - kh) // stride + 1
    W_out = (W_ + 2 * padding - kw) // stride + 1
    out = np.zeros((N, C_out, H_out, W_out), dtype=X.dtype)
    for i in range(H_out):
        for j in range(W_out):
            patch = Xp[:, :, i * stride:i * stride + kh, j * stride:j * stride + kw]   # [N, Cin, kh, kw]
            for c in range(C_out):
                out[:, c, i, j] = np.sum(patch * W[c], axis=(1, 2, 3)) + b[c]
    return out
```

Đoạn mã 3.1. Tích chập 2D nhiều kênh bằng vòng lặp, bám sát công thức (3.1)

```python
# tl/cnn_scratch.py
def conv2d_im2col(X, W, b, stride=1, padding=0):
    Xp = np.pad(X, ((0, 0), (0, 0), (padding, padding), (padding, padding))) if padding else X
    C_out, C_in, kh, kw = W.shape
    win = sliding_window_view(Xp, (kh, kw), axis=(2, 3))[:, :, ::stride, ::stride]    # [N, Cin, Ho, Wo, kh, kw]
    out = np.einsum("nchwij,ocij->nohw", win, W, optimize=True)
    return out + b[None, :, None, None]
```

Đoạn mã 3.2. Tích chập 2D kiểu im2col bằng NumPy

```python
# tl/cnn_scratch.py
def maxpool2d(X, size=2, stride=2):
    win = sliding_window_view(X, (size, size), axis=(2, 3))[:, :, ::stride, ::stride]
    return win.max(axis=(-1, -2))
```

Đoạn mã 3.3. Max Pooling bằng NumPy

Để kiểm chứng, ta nạp trọng số của Custom CNN đã huấn luyện bằng PyTorch trên SVHN vào các hàm NumPy và chạy toàn bộ mạng (ba khối conv – BN – ReLU, GAP, Dense) trên 2.000 ảnh kiểm thử:

- bản vòng lặp và bản im2col lệch nhau tối đa $1{,}78 \times 10^{-15}$ (sai số làm tròn), nhưng tốn 262,97 ms so với 2,276 ms mỗi ảnh cho riêng tầng đầu;
- logit của mạng NumPy lệch PyTorch tối đa $3{,}45 \times 10^{-6}$; nhãn dự đoán trùng 100,00 %, độ chính xác 96,55 % so với 96,55 %;
- cả mạng NumPy mất 33,43 ms mỗi ảnh, PyTorch trên cùng CPU 3,480 ms: hiểu thuật toán thì viết tay được, nhưng hiệu năng cần thư viện tối ưu.

3.3.2 Custom CNN với khối kiểu VGG bằng Keras / TensorFlow

Mô hình cơ sở gồm ba khối kiểu VGG (32, 64, 128 kênh), mỗi khối hai Conv 3×3 kèm BatchNorm [36] và ReLU, MaxPool, Dropout tăng dần; sau đó GAP, Dense 128 và softmax. Cùng kiến trúc này được viết lại bằng PyTorch để so sánh hai nền tảng.

```python
# tl/cnn_keras.py
def build_vgg_block(x, filters, num_convs=2, drop=0.2):
    """Khối VGG: các Conv 3x3 xếp chồng (mỗi conv kèm BatchNorm + ReLU) rồi MaxPool 2x2."""
    for _ in range(num_convs):
        x = layers.Conv2D(filters, 3, padding="same", use_bias=False)(x)
        x = layers.BatchNormalization()(x)
        x = layers.ReLU()(x)
    x = layers.MaxPooling2D(2)(x)
    return layers.Dropout(drop)(x)
```

Đoạn mã 3.4. Khối tích chập kiểu VGG bằng Keras

```python
# tl/cnn_keras.py
def build_keras_custom_cnn(input_shape=(32, 32, 3), n_classes=10, widths=(32, 64, 128), drops=(0.2, 0.3, 0.4)):
    inputs = keras.Input(shape=input_shape)
    x = inputs
    for w, p in zip(widths, drops):
        x = build_vgg_block(x, w, 2, p)
    x = layers.GlobalAveragePooling2D()(x)          # GAP thay cho Flatten + Dense lớn
    x = layers.Dense(128, activation="relu")(x)
    x = layers.Dropout(0.4)(x)
    outputs = layers.Dense(n_classes, activation="softmax")(x)
    model = keras.Model(inputs, outputs)
    model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    return model
```

Đoạn mã 3.5. Custom CNN bằng Keras Functional API

3.3.3 Khối Inverted Residual kiểu MobileNetV2 bằng PyTorch

```python
# tl/cnn_torch.py
class InvertedResidual(nn.Module):
    """1x1 mở rộng kênh -> 3x3 depthwise (groups = số kênh) -> 1x1 chiếu tuyến tính (không ReLU)."""

    def __init__(self, cin, cout, stride, expand):
        super().__init__()
        hidden = cin * expand
        self.use_res = stride == 1 and cin == cout
        layers = [] if expand == 1 else [conv_bn(cin, hidden, 1, act=nn.ReLU6)]
        layers += [conv_bn(hidden, hidden, 3, stride, groups=hidden, act=nn.ReLU6),
                   conv_bn(hidden, cout, 1, act=None)]
        self.conv = nn.Sequential(*layers)

    def forward(self, x):
        return x + self.conv(x) if self.use_res else self.conv(x)
```

Đoạn mã 3.6. Khối Inverted Residual: mở rộng 1×1, depthwise 3×3, chiếu tuyến tính 1×1

Toàn mạng MobileNetV2 xếp 17 khối theo bảng cấu hình của bài báo gốc, đổi bước của hai nhóm khối đầu từ 2 về 1 vì ảnh vào nhỏ hơn ImageNet nhiều lần. Với ảnh EuroSAT 64×64, tầng thân giữ bước 2 như bản gốc để bản đồ đặc trưng sau thân là 32×32, đúng bằng trường hợp SVHN. Lần chạy đầu giữ nguyên 64×64 qua các khối mở rộng 6 lần đã cần hơn 8 GB bộ nhớ GPU ở lô 256: VRAM tràn sang bộ nhớ chia sẻ của Windows và một epoch kéo dài hơn 35 phút, nên cấu hình này được thay.

3.3.4 Kiến trúc ResNet-18 bằng PyTorch

```python
# tl/cnn_torch.py
class ResidualBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)
        self.shortcut = nn.Sequential()
        if stride != 1 or cin != cout:                    # chiếu 1x1 khi đổi kích thước hoặc số kênh
            self.shortcut = nn.Sequential(nn.Conv2d(cin, cout, 1, stride, bias=False), nn.BatchNorm2d(cout))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.shortcut(x))            # y = F(x) + x: "đường cao tốc" của gradient
```

Đoạn mã 3.7. Khối Residual cơ bản với kết nối tắt chiếu 1×1 khi đổi kích thước

```python
# tl/cnn_torch.py
class ResNet18(nn.Module):
    def __init__(self, n_classes=10):
        super().__init__()
        self.stem = conv_bn(3, 64)
        self.cin = 64
        self.layer1 = self._make_layer(64, 2, 1)
        self.layer2 = self._make_layer(128, 2, 2)
        self.layer3 = self._make_layer(256, 2, 2)
        self.layer4 = self._make_layer(512, 2, 2)
        self.fc = nn.Linear(512, n_classes)

    def _make_layer(self, cout, n, stride):
        layers = []
        for s in [stride] + [1] * (n - 1):
            layers.append(ResidualBlock(self.cin, cout, s))
            self.cin = cout
        return nn.Sequential(*layers)

    def feature_maps(self, x):
        return self.layer4(self.layer3(self.layer2(self.layer1(self.stem(x)))))

    def forward(self, x):
        return self.fc(F.adaptive_avg_pool2d(self.feature_maps(x), 1).flatten(1))
```

Đoạn mã 3.8. Kiến trúc ResNet-18: thân 3×3, bốn tầng $[2, 2, 2, 2]$ khối, GAP và FC

3.3.5 Quy trình huấn luyện trên GPU

Mọi mô hình PyTorch huấn luyện từ đầu với SGD Nesterov (momentum 0,9, weight decay $5 \cdot 10^{-4}$), lịch học One-Cycle [37] đỉnh 0,1, lô 256; SVHN 15 epoch, EuroSAT 30 epoch. Toàn bộ ảnh `uint8` nạp lên GPU một lần; chuẩn hóa và tăng cường dữ liệu làm ngay trên GPU (Đoạn mã 3.9), và lan truyền chạy ở độ chính xác hỗn hợp bfloat16. Nhờ vậy CPU rảnh cho các tiến trình Keras và scikit-learn chạy song song.

```python
# scripts/run_ch3_torch.py
def augment(x, name):
    """x: (B,3,H,W) float trên GPU. Dịch ngẫu nhiên +-4 điểm ảnh từng mẫu bằng chỉ số nâng cao."""
    B, C, H, W = x.shape
    xp = F.pad(x, (4, 4, 4, 4), mode="reflect")
    iy = torch.randint(0, 9, (B,), device=x.device)
    ix = torch.randint(0, 9, (B,), device=x.device)
    ar_h = torch.arange(H, device=x.device)
    ar_w = torch.arange(W, device=x.device)
    rows = (iy[:, None] + ar_h)[:, None, :, None]
    cols = (ix[:, None] + ar_w)[:, None, None, :]
    x = xp[torch.arange(B, device=x.device)[:, None, None, None], torch.arange(C, device=x.device)[None, :, None, None],
    ↪  rows, cols]
    if name == "eurosat":                              # ảnh chụp từ trên xuống: không có chiều "trên/dưới"
        m = torch.rand(B, 1, 1, 1, device=x.device)
        x = torch.where(m < 0.5, x.flip(3), x)
        m = torch.rand(B, 1, 1, 1, device=x.device)
        x = torch.where(m < 0.5, x.flip(2), x)
        m = torch.rand(B, 1, 1, 1, device=x.device)
        x = torch.where(m < 0.5, x.transpose(2, 3), x)
    return x
```

Đoạn mã 3.9. Tăng cường dữ liệu trên GPU: dịch ngẫu nhiên từng ảnh bằng chỉ số nâng cao, lật và chuyển vị cho ảnh vệ tinh

3.4 Đánh giá và đối chuẩn các biến thể kiến trúc CNN

Phương pháp. Bốn kiến trúc được huấn luyện từ trọng số ngẫu nhiên trên cùng dữ liệu, cùng siêu tham số, không dùng trọng số tiền huấn luyện ImageNet, để phản ánh năng lực trích đặc trưng của từng cấu trúc trên dữ liệu đích:

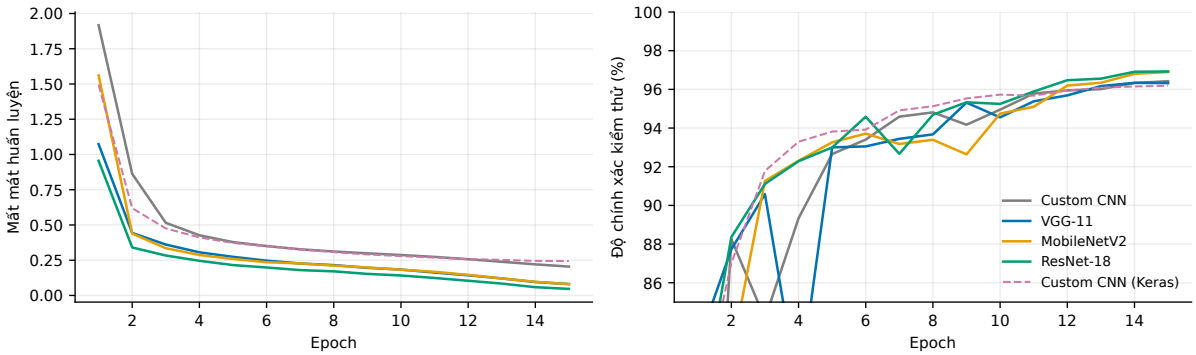
1. **Custom CNN** (mô hình cơ sở): 6 tầng Conv trong ba khối kiểu VGG và GAP, 305.258 tham số.
2. **VGG-11**: cấu hình A (8 Conv có BN), bỏ ba tầng FC 4.096 nơ-ron thay bằng GAP, 9.228.362 tham số.
3. **MobileNetV2**: 17 khối Inverted Residual, 2.236.682 tham số.
4. **ResNet-18**: 8 khối Residual, 11.173.962 tham số.

Độ trễ suy luận được đo lại sau khi mọi huấn luyện kết thúc (máy không chạy việc khác): lô 1 ảnh, trung vị của 300 lần trên GPU và 100 lần trên CPU 4 luồng; thông lượng FPS đo trên GPU với lô 256.

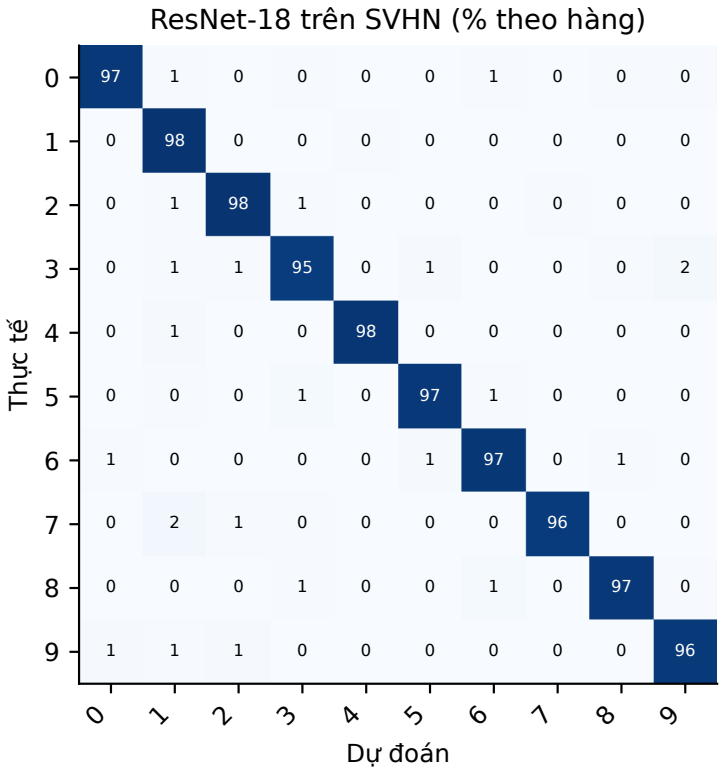
Bảng 3.1. Đối chuẩn đa chiều các biến thể CNN huấn luyện từ đầu trên SVHN, EuroSAT và 1D CNN trên dữ liệu tiểu đường.

Dữ liệu	Kiến trúc	Acc (%)	F1	Tham số (M)	GFLOPs	GPU (ms)	CPU (ms)	FPS	Huấn luyện (s)	Dung lượng
SVHN	Custom CNN	96,42	0,9611	0,31	0,077	0,79	0,88	59.146	278	1,2 MB
	VGG-11	96,32	0,9605	9,23	0,306	0,99	9,01	24.221	415	35,2 MB
	MobileNetV2	96,90	0,9666	2,24	0,176	4,39	8,29	4.774	1.953	8,8 MB
	ResNet-18	**96,93**	0,9674	11,17	1,111	2,12	12,92	5.403	772	42,7 MB
	Custom CNN (Keras)	96,19	0,9582	0,31	0,077	–	1,50	–	4.203*	3,6 MB
EuroSAT	Custom CNN	97,41	0,9733	0,31	0,309	0,83	3,07	11.037	404	1,2 MB
	VGG-11	97,78	0,9772	9,23	1,222	1,01	12,03	5.242	622	35,2 MB
	MobileNetV2	97,72	0,9767	2,24	0,176	5,13	8,70	4.699	1.293	8,8 MB
	ResNet-18	**98,26**	0,9819	11,17	4,443	2,32	28,98	1.232	1.219	42,7 MB
	Custom CNN (Keras)	94,28	0,9414	0,31	0,309	–	2,45	–	4.992*	3,6 MB
Tiểu đường	1D CNN (Conv1D)	97,23	0,8063	0,068	0,0003	0,54	0,18	470.242	12,5	273,3 KB

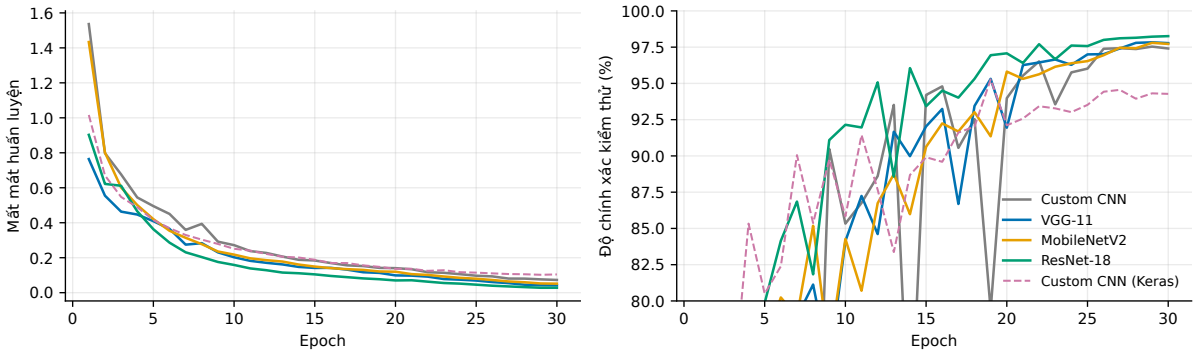
F1 là macro trên 10 lớp (1D CNN: lớp dương). FLOPs = 2× số phép nhân-cộng của một ảnh. Độ trễ đo lô 1 (trung vị), CPU 4 luồng; FPS đo trên GPU với lô 256. Huấn luyện trên GPU RTX 4060 Laptop, * Keras chạy CPU (TensorFlow trên Windows không dùng GPU).



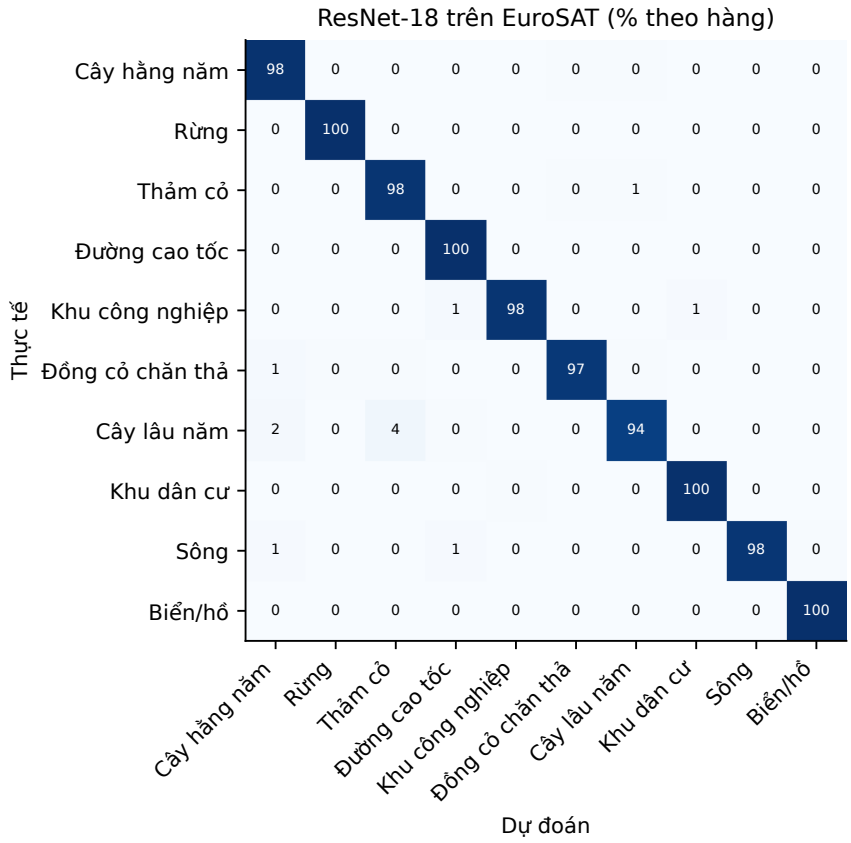
Hình 3.13. Mất mát huấn luyện và độ chính xác kiểm thử theo epoch của các kiến trúc trên SVHN.



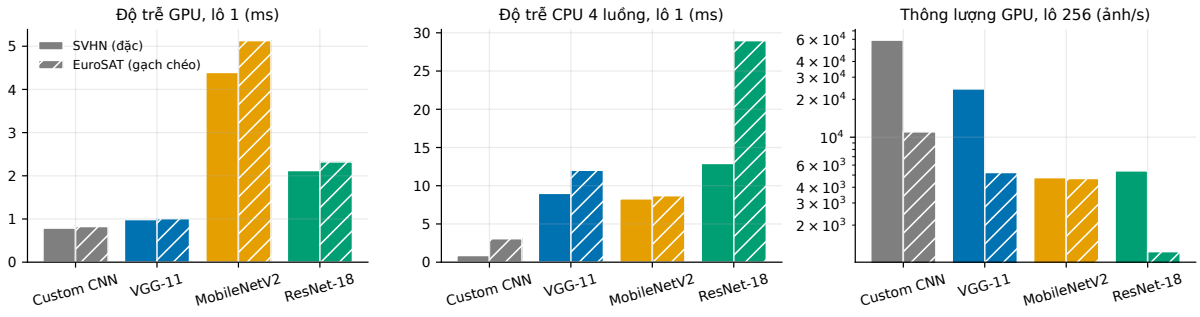
Hình 3.14. Ma trận nhầm lẫn (% theo hàng) của ResNet-18 trên tập kiểm thử SVHN.



Hình 3.15. Mất mát huấn luyện và độ chính xác kiểm thử theo epoch của các kiến trúc trên EuroSAT.



Hình 3.16. Ma trận nhầm lẫn (% theo hàng) của ResNet-18 trên tập kiểm thử EuroSAT.



Hình 3.17. Độ trễ suy luận (GPU và CPU, lô 1) và thông lượng GPU (lô 256) của bốn kiến trúc.

Phân tích đánh đổi thiết kế.

- **Độ chính xác.** Trên SVHN, ResNet-18 đạt 96,93 %, MobileNetV2 96,90 %, VGG-11 96,32 % và Custom CNN 96,42 %: bốn kiến trúc cách nhau chưa tới một điểm phần trăm, vì SVHN 32×32 không đòi hỏi mạng rất sâu. Trên EuroSAT 64×64, ResNet-18 dẫn đầu rõ hơn với 98,26 %, tiếp theo là VGG-11 97,78 % và MobileNetV2 97,72 % gần như ngang nhau, Custom CNN 97,41 %: ảnh lớn hơn, kết cấu phức tạp hơn thì độ sâu và kết nối tắt mới phát huy tác dụng. Lỗi còn lại tập trung ở các cặp lớp có kết cấu giống nhau (cây hằng năm – cây lâu năm, thảm cỏ – đồng cỏ chăn thả) và ở SVHN là các chữ số có nét gần nhau (Hình 3.14, 3.16).
- **Chi phí tính toán.** MobileNetV2 có 2.236.682 tham số, ít hơn ResNet-18 khoảng 5 lần và VGG-11 khoảng 4 lần, FLOPs cũng thấp hơn ResNet-18 nhiều lần (Bảng 3.1). Nhưng trên GPU, ở lô 1, MobileNetV2 *không* nhanh hơn: nó có hơn 50 tầng nhỏ nối tiếp, mỗi tầng một lần khởi chạy kernel, và tích chập depthwise có cường độ tính toán thấp nên GPU không tận dụng được. Ít FLOPs không đồng nghĩa nhanh hơn trên mọi phần cứng;

lợi thế của MobileNet thể hiện rõ nhất trên CPU và thiết bị di động, nơi số phép tính mới là nút thắt.

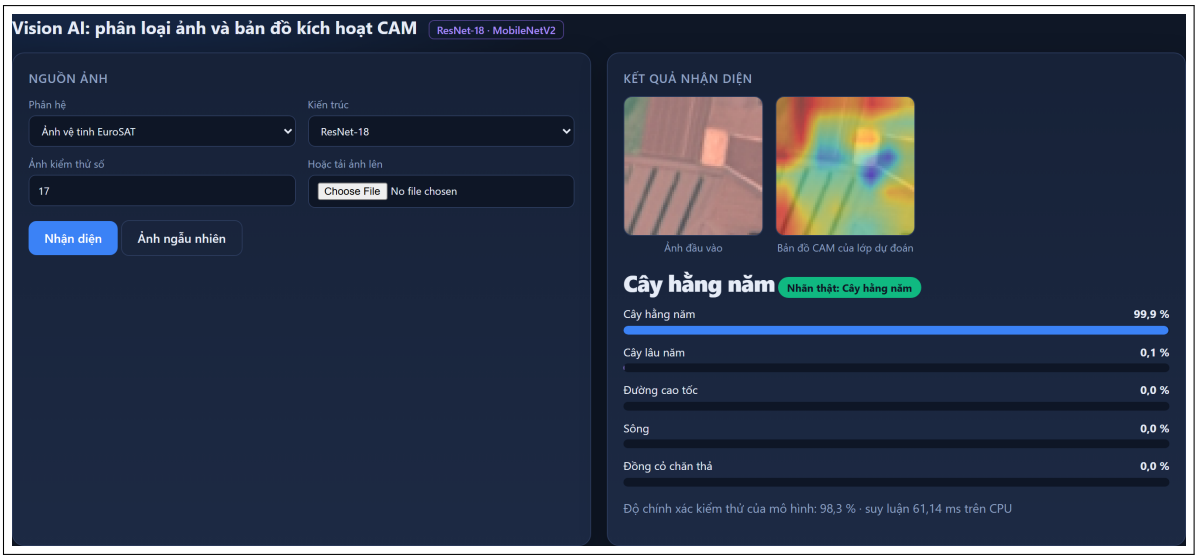
- **VGG-11.** Nhiều tham số thứ hai nhưng kiến trúc phẳng, ít tầng, nên độ trễ GPU thấp; độ chính xác đứng sau hai kiến trúc hiện đại hơn. Đây là minh chứng cho bước tiến của kết nối tắt và tích chập tách biệt chiều sâu.
- **Custom CNN.** Chỉ 305.258 tham số, tệp vài MB, đủ tốt cho SVHN và là mốc kiểm chứng ban đầu hợp lý. Bản Keras cùng kiến trúc, cùng số epoch nhưng huấn luyện bằng Adam (tốc độ học 10^{-3}, giảm theo cosine) và tăng cường dữ liệu bằng các lớp có sẵn của Keras (dịch, lật; không có chuyển vị). Trên SVHN nó đạt 96,19 %, sát bản PyTorch (96,42 %); trên EuroSAT chỉ đạt 94,28 % so với 97,41 %, và đường độ chính xác kiểm thử của nó dừng ở mức thấp hơn trong khi các mô hình PyTorch hội tụ dần khi tốc độ học One-Cycle giảm về cuối (Hình 3.15). Khoảng cách này đến từ công thức huấn luyện (SGD Nesterov với lịch One-Cycle tổng quát hóa tốt hơn Adam trên CNN có BatchNorm, cộng thêm phép chuyển vị), không phải từ nền tảng. Thời gian huấn luyện Keras dài hơn nhiều lần vì TensorFlow trên Windows không dùng được GPU.
- **1D CNN trên dữ liệu bảng** đạt F1 0,8063, ngang MLP của Chương 2 (0,8047): tích chập 1D dùng được, nhưng với dữ liệu bảng thứ tự cột là tùy ý nên "tính cục bộ" không mang ý nghĩa như với ảnh, và lợi ích so với MLP là không đáng kể.

3.5 Đóng gói triển khai hệ thống Web App thị giác máy tính

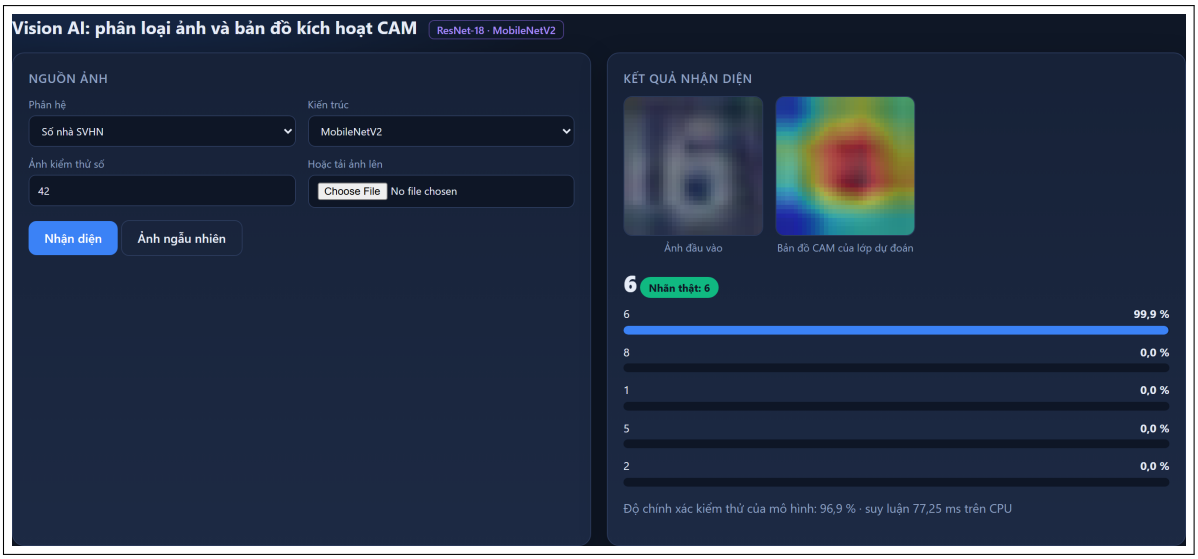
ResNet-18 và MobileNetV2 của cả hai tập dữ liệu được nạp vào phân hệ *Vision AI* của Web App (mục 2.6). Người dùng chọn một ảnh kiểm thử hoặc tải ảnh lên; hệ thống trả năm lớp có xác suất cao nhất cùng *bản đồ kích hoạt lớp* (Class Activation Map, CAM) [38]. Vì mạng kết thúc bằng GAP rồi một tầng tuyến tính, điểm của lớp c là $\sum_k w_k^c \cdot \mathrm{GAP}(f_k)$; đổi thứ tự lấy tổng và trung bình cho bản đồ $M_c(x,y) = \sum_k w_k^c f_k(x,y)$, chỉ ra vùng ảnh đã đẩy mô hình về lớp c (Đoạn mã 3.10).

```python
# app/app.py
def cam_png(model, arch, x, cls):
    """Class Activation Map: tổng có trọng số các bản đồ đặc trưng cuối theo trọng số FC của lớp dự đoán."""
    with torch.no_grad():
        fmap = model.feature_maps(x) if arch == "resnet18" else model.features(x)
        w = model.fc.weight if arch == "resnet18" else model.fc[1].weight
        cam = torch.einsum("c,chw->hw", w[cls], fmap[0])
    cam = torch.relu(cam)
    cam = (cam / (cam.max() + 1e-8)).numpy()
    return cam
```

Đoạn mã 3.10. Tính bản đồ kích hoạt lớp CAM từ bản đồ đặc trưng cuối và trọng số tầng FC



Hình 3.18. Phân hệ Vision AI: ảnh vệ tinh EuroSAT, bản đồ CAM của lớp dự đoán và xác suất năm lớp hàng đầu của ResNet-18.



Hình 3.19. Phân hệ Vision AI với ảnh số nhà SVHN và mô hình MobileNetV2.

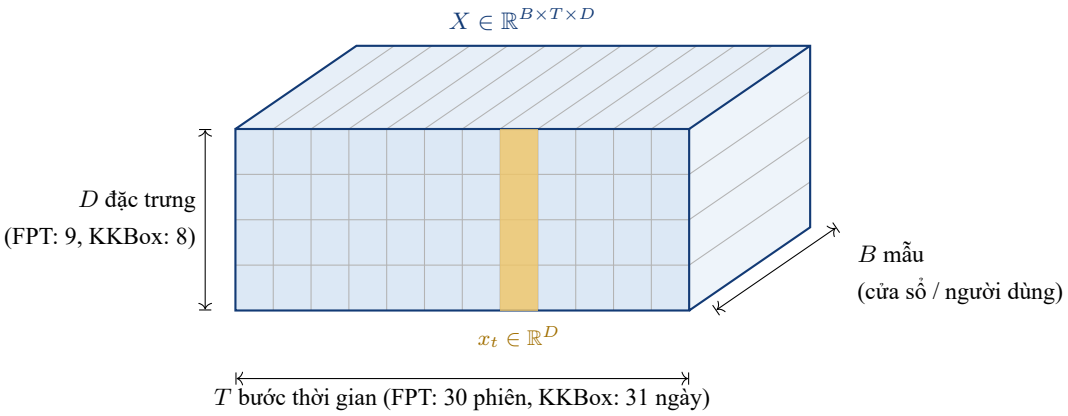
Bản đồ CAM tập trung vào đúng chữ số ở giữa ảnh SVHN (bỏ qua chữ số hàng xóm) và vào vùng kết cấu đặc trưng của ảnh vệ tinh, cho thấy mô hình dựa vào đúng bằng chứng thị giác. Suy luận một ảnh kèm CAM trên CPU của máy chủ mất vài chục mili-giây (Hình 3.18, đo khi máy đang đồng thời huấn luyện mô hình Keras), đủ cho tương tác thời gian thực.

Chương 4

Mạng Nơ-ron Hồi Quy (RNN)

4.1 Cơ sở lý thuyết mạng nơ-ron hồi quy và mô hình hóa chuỗi

Dữ liệu bảng và ảnh tĩnh ở hai chương trước thỏa giả định các mẫu độc lập cùng phân phối. Dữ liệu chuỗi vi phạm giả định đó: giá hôm nay phụ thuộc giá hôm qua, việc người dùng bỏ dịch vụ cuối tháng được báo trước bằng hành vi những ngày giữa tháng. Thứ tự là thông tin. Một tập dữ liệu chuỗi được biểu diễn thành tensor ba chiều $X \in \mathbb{R}^{B\times T\times D}$ với B mẫu, T bước thời gian và D đặc trưng tại mỗi bước (Hình 4.1).



Hình 4.1. Tensor chuỗi ba chiều $[B, T, D]$ dùng trong mô hình hóa học sâu tuần tự.

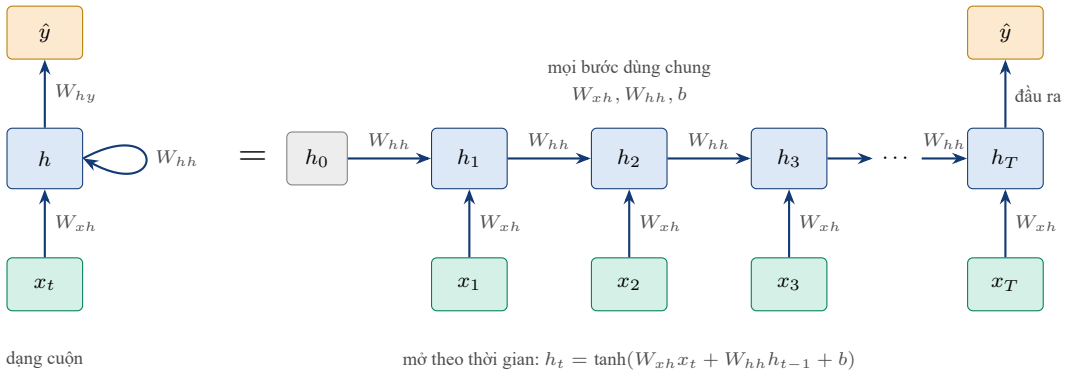
4.1.1 Kiến trúc Simple RNN và đồ thị tính toán mở rộng theo thời gian

Mạng hồi quy đơn giản (Elman RNN) giữ một vector trạng thái ẩn $\mathbf{h}_t \in \mathbb{R}^H$ làm bộ nhớ. Tại mỗi bước:

$$\mathbf{h}_t = \tanh\left(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h\right), \tag{4.1}$$

$$\hat{y} = W_{hy}\mathbf{h}_T + b_y \quad \text{(dạng many-to-one dùng trong chương này)}, \tag{4.2}$$

với $W_{xh} \in \mathbb{R}^{H\times D}$, $W_{hh} \in \mathbb{R}^{H\times H}$ được *dùng chung* ở mọi bước. Mở cuộn theo thời gian (Hình 4.2), RNN trở thành một mạng truyền thẳng sâu T tầng có chung trọng số.



Hình 4.2. Đồ thị tính toán của RNN: dạng cuộn và dạng mở rộng theo thời gian với trọng số dùng chung.

4.1.2 Lan truyền ngược qua thời gian (BPTT) và hiện tượng triệt tiêu đạo hàm

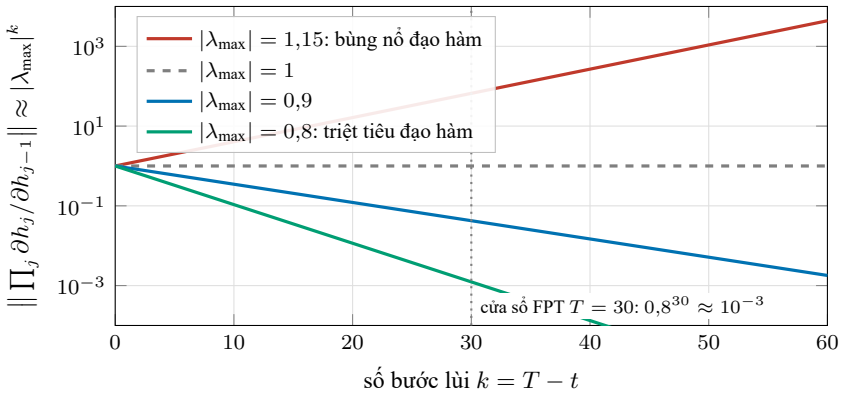
Huấn luyện RNN dùng lan truyền ngược qua thời gian (Backpropagation Through Time). Gradient của mất mát ở bước T theo W_{hh} cộng dồn đóng góp của mọi bước trước:

$$\frac{\partial \mathcal{L}_T}{\partial W_{hh}} = \sum_{k=1}^{T} \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \left(\prod_{j=k+1}^{T} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial W_{hh}}, \qquad \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) W_{hh}. \tag{4.3}$$

Tích $T-k$ ma trận Jacobian quyết định bước k còn ảnh hưởng bao nhiêu [19]. Vì $|1-h^2| \le 1$, chuẩn của tích bị chặn bởi $|\lambda_{\max}(W_{hh})|^{T-k}$:

- nếu $|\lambda_{\max}| < 1$: tích giảm theo hàm mũ về 0, gradient của các bước xa *triệt tiêu*, mạng không học được phụ thuộc dài hạn;
- nếu $|\lambda_{\max}| > 1$: tích tăng theo hàm mũ, gradient *bùng nổ*, trọng số dao động mạnh hoặc tràn số.

Hình 4.3 cho thấy với $|\lambda_{\max}| = 0{,}8$, tín hiệu từ 30 bước trước (độ dài cửa sổ của bài toán chứng khoán) chỉ còn cỡ 10^{-3}.



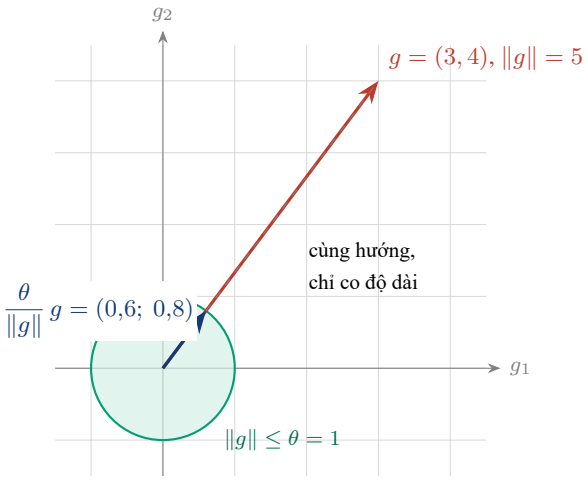
Hình 4.3. Bản chất toán học của triệt tiêu và bùng nổ đạo hàm: chuẩn tích chuỗi Jacobian theo số bước lùi.

4.1.3 Kỹ thuật cắt tỉa đạo hàm (Gradient Clipping)

Chống bùng nổ đạo hàm, Pascanu và cộng sự [39] đề xuất co gradient khi chuẩn vượt ngưỡng θ:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g}, & \|\mathbf{g}\|_2 \le \theta, \\ \dfrac{\theta}{\|\mathbf{g}\|_2}\,\mathbf{g}, & \|\mathbf{g}\|_2 > \theta. \end{cases} \tag{4.4}$$

Phép co giữ nguyên *hướng* của bước cập nhật, chỉ giới hạn *độ dài* (Hình 4.4). Mọi mô hình hồi quy trong chương dùng $\theta = 1$ (`clip_grad_norm_` trong PyTorch, `clipnorm` trong Keras). Cắt tỉa không chữa được triệt tiêu đạo hàm; việc đó cần thay đổi kiến trúc.



Hình 4.4. Hình học của cắt tỉa đạo hàm: vector $\mathbf{g}$ được co về hình cầu bán kính θ, giữ nguyên hướng.

4.1.4 Kiến trúc tế bào LSTM và GRU

LSTM [20]. LSTM tách bộ nhớ thành trạng thái ô nhớ $\mathbf{c}_t$ và trạng thái ẩn $\mathbf{h}_t$, điều khiển bằng ba cổng sigmoid nhận $[\mathbf{h}_{t-1}, \mathbf{x}_t]$:

$$\text{cổng quên:} \quad \mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), \tag{4.5}$$

$$\text{cổng vào:} \quad \mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i), \tag{4.6}$$

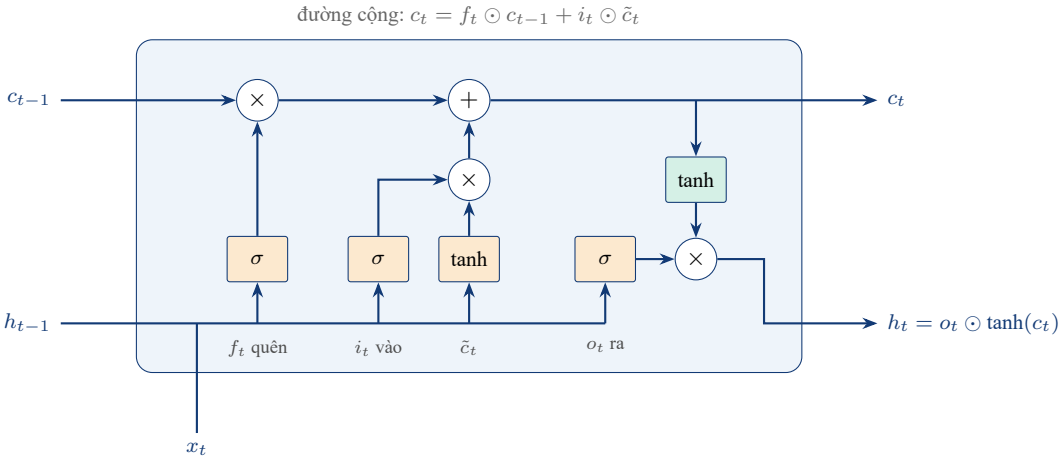
$$\text{ứng viên:} \quad \tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c), \tag{4.7}$$

$$\text{cập nhật ô nhớ:} \quad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{4.8}$$

$$\text{cổng ra:} \quad \mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \tag{4.9}$$

$$\text{trạng thái ẩn:} \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{4.10}$$

Phương trình (4.8) là phép *cộng*: $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f}_t)$, không có ma trận trọng số nhân vào, nên khi cổng quên gần 1 gradient đi qua nhiều bước mà gần như không suy giảm (Hình 4.5).



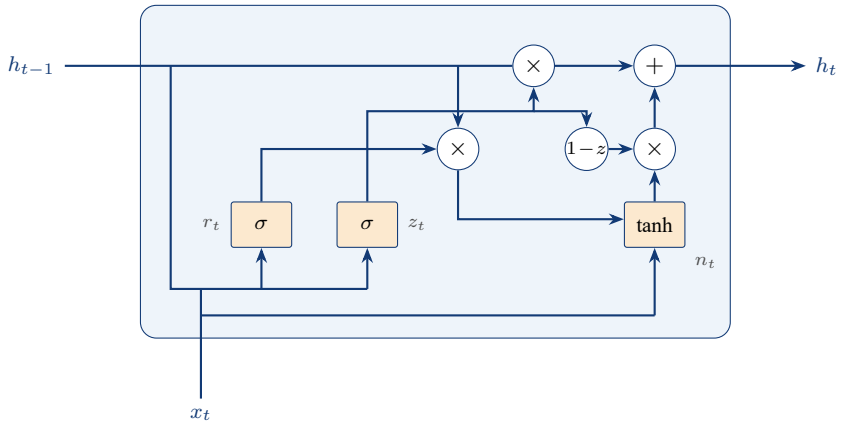
Hình 4.5. Kiến trúc vi mô của tế bào LSTM: đường cộng của ô nhớ và ba cổng điều khiển.

GRU [21]. GRU gộp ô nhớ và trạng thái ẩn, dùng hai cổng:

$$\mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r), \qquad \mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z), \tag{4.11}$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h), \qquad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{4.12}$$

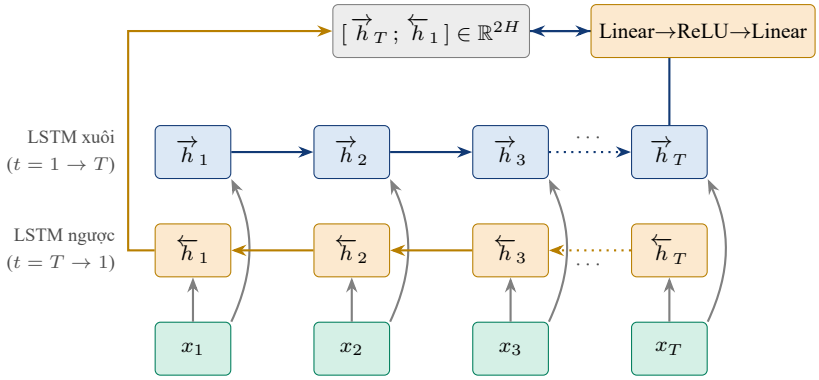
Trạng thái mới là tổ hợp lồi của trạng thái cũ và ứng viên (Hình 4.6); với ba khối trọng số thay vì bốn, GRU ít hơn LSTM 25 % tham số ở tầng hồi quy.



Hình 4.6. Kiến trúc vi mô của tế bào GRU với cổng đặt lại r_t và cổng cập nhật z_t.

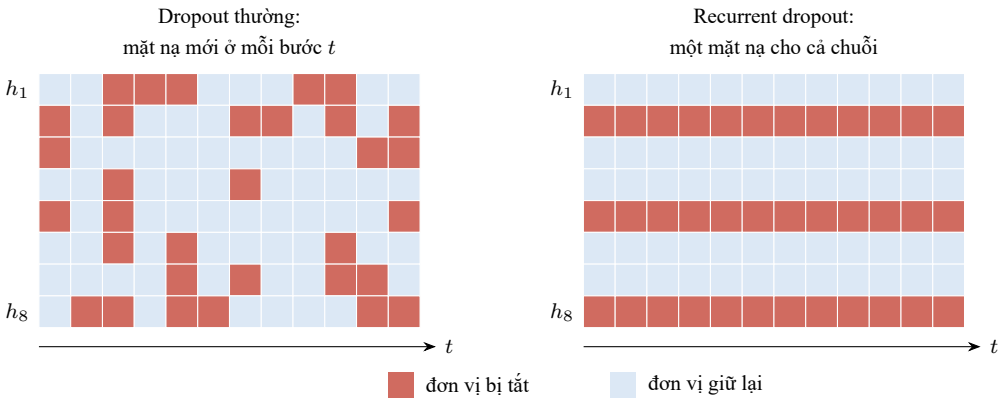
4.1.5 Mạng hồi quy hai chiều (BiLSTM) và Recurrent Dropout

Khi toàn bộ chuỗi đã có sẵn lúc dự đoán (phân loại cả một tháng hành vi), đọc theo cả hai chiều cho nhiều ngữ cảnh hơn. BiLSTM [40] chạy một LSTM xuôi $\overrightarrow{\mathbf{h}}_t$ và một LSTM ngược $\overleftarrow{\mathbf{h}}_t$, rồi ghép $[\overrightarrow{\mathbf{h}}_T; \overleftarrow{\mathbf{h}}_1] \in \mathbb{R}^{2H}$ (Hình 4.7). Lưu ý với dự báo chuỗi thời gian, chiều ngược vẫn chỉ đọc dữ liệu *trong cửa sổ quá khứ*, không nhìn trước tương lai.



Hình 4.7. Mạng hồi quy hai chiều BiLSTM tổng hợp ngữ cảnh theo cả hai chiều thời gian.

Dropout thông thường lấy mặt nạ mới ở mỗi bước thời gian, làm nhiễu bộ nhớ của mạng. Gal và Ghahramani [41] chỉ ra cách đúng là dùng *một* mặt nạ cho mọi bước của cùng một chuỗi (Hình 4.8). Trong thực nghiệm, dropout được đặt giữa hai tầng hồi quy và trước tầng ra, không đặt trong vòng hồi quy, để PyTorch và Keras dùng được nhân cuDNN/oneDNN tối ưu.



Hình 4.8. Dropout thông thường đổi mặt nạ ở mỗi bước; recurrent dropout giữ một mặt nạ cố định cho cả chuỗi.

4.2 Khảo sát thực nghiệm và tiền xử lý hai bộ dữ liệu chuỗi

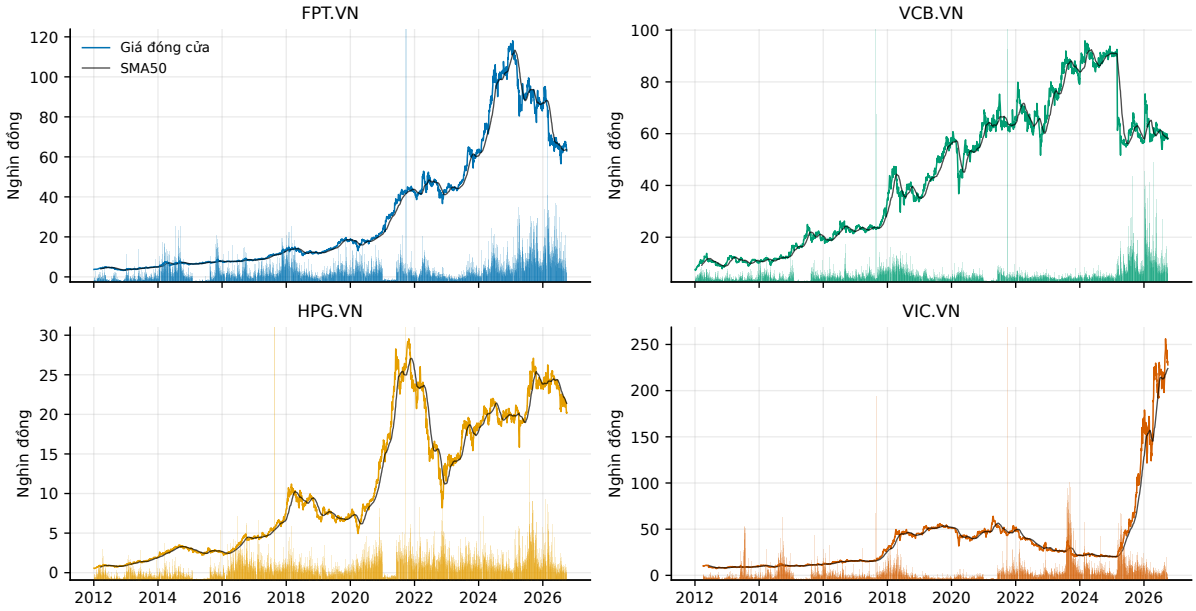
4.2.1 Tập dữ liệu 1: chuỗi giá chứng khoán Việt Nam (VN30, 2012-01-03 – 2026-09-30)

- **Nguồn:** lịch sử giao dịch trên Sở Giao dịch Chứng khoán TP. Hồ Chí Minh của bốn mã trụ cột trong rổ VN30, tải từ Yahoo Finance bằng thư viện `yfinance` [42]: FPT (công nghệ – viễn thông), VCB (ngân hàng), HPG (thép) và VIC (bất động sản – dịch vụ). Giá đã điều chỉnh theo chia tách và cổ tức.
- **Quy mô:** tổng 14.338 phiên: FPT 3.606, VCB 3.590, HPG 3.608, VIC 3.534 phiên. Mỗi phiên có 9 đặc trưng: Open, High, Low, Close, Volume và bốn chỉ báo kỹ thuật tự tính – trung bình trượt 20 và 50 phiên (SMA20, SMA50), chỉ số sức mạnh tương đối 14 phiên theo Wilder (RSI-14) và độ biến động 30 phiên (độ lệch chuẩn lợi suất ngày).
- **Bài toán:** từ 30 phiên gần nhất của FPT, dự báo giá đóng cửa phiên kế tiếp.

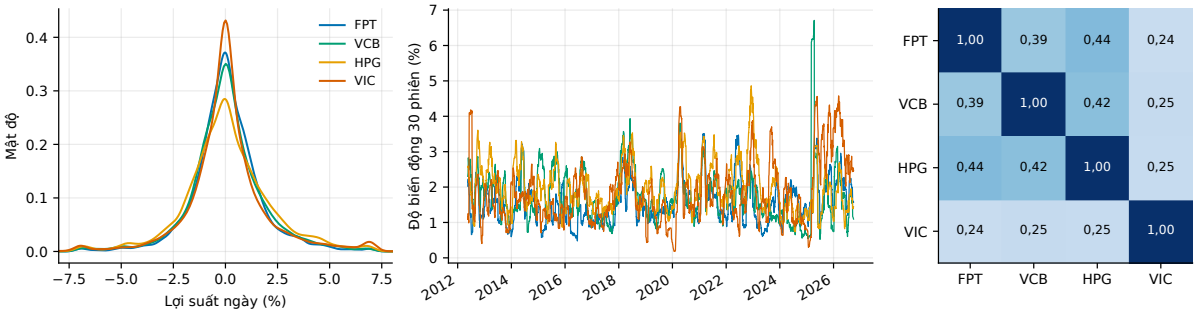
Bảng 4.1. Năm phiên giao dịch tiêu biểu của FPT kèm chỉ báo kỹ thuật.

Ngày	Open	High	Low	Close	Volume (cp)	SMA20	RSI-14
2012-06-18	4,78	4,78	4,72	4,74	220.216	4,74	49,64
2015-11-06	8,64	8,80	8,40	8,48	14.057.051	7,83	75,04
2019-07-03	14,77	14,84	14,73	14,80	3.387.397	14,35	62,56
2023-04-11	44,59	44,75	44,53	44,64	893.361	44,15	54,32
2026-09-30	63,40	64,00	63,00	63,00	3.041.400	65,65	41,35

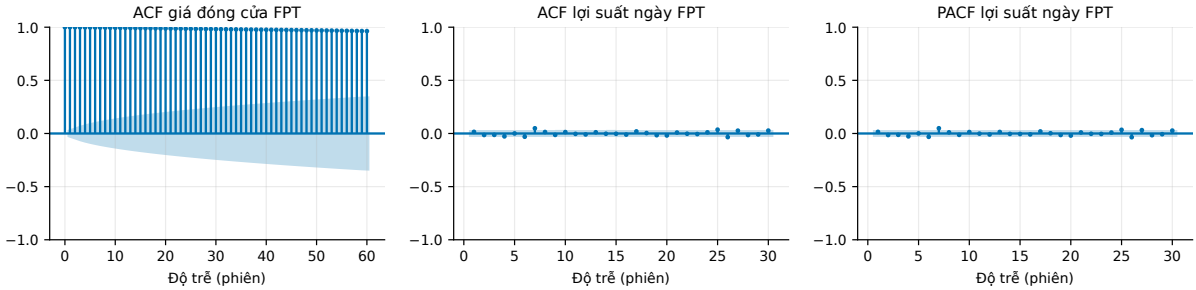
Giá tính bằng nghìn đồng, đã điều chỉnh theo chia tách và cổ tức (auto_adjust của Yahoo Finance).



Hình 4.9. Giá đóng cửa (nghìn đồng), đường SMA50 và khối lượng giao dịch (cột mờ) của bốn mã VN30.



Hình 4.10. Phân phối lợi suất ngày, độ biến động 30 phiên và ma trận tương quan lợi suất giữa bốn mã.



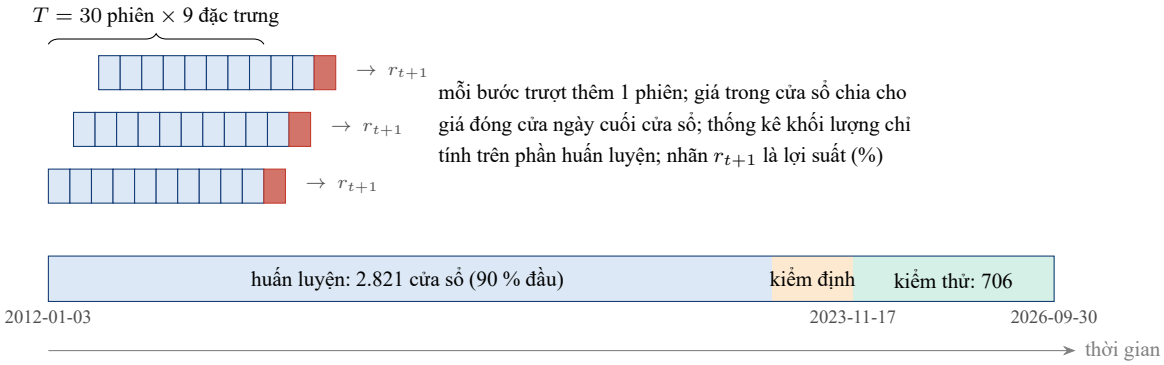
Hình 4.11. ACF của giá đóng cửa và ACF, PACF của lợi suất ngày FPT (vùng tô là khoảng tin cậy 95 %).

Nhận xét EDA.

- **Xu hướng và đổi chế độ:** giá FPT tăng hơn 20 lần trong giai đoạn khảo sát, đạt đỉnh 118.051 đồng trong giai đoạn kiểm thử, trong khi mức cao nhất của giai đoạn huấn luyện chỉ 64.893 đồng (Hình 4.9). VIC tăng vọt từ 2025. Chuỗi giá *không dừng*: kiểm định Augmented Dickey–Fuller cho $p = 0{,}8474$, không bác bỏ được nghiệm đơn vị.
- **Lợi suất gần như không tự tương quan:** lợi suất ngày là chuỗi dừng (ADF $p = 0{,}0000$), đuôi dày (độ nhọn dư của FPT 3,277), và tự tương quan bậc 1 chỉ 0,0153 (Hình 4.11). ACF của *giá* giảm rất chậm vì giá hôm nay gần

như bằng giá hôm qua, nhưng ACF của *lợi suất* gần như nằm trong dải tin cậy: phần dự báo được của biến động ngày mai là rất nhỏ.

- **Tương quan giữa các mã** ở mức trung bình (r từ 0,2379 đến 0,4438), phản ánh xu hướng chung của thị trường (Hình 4.10).



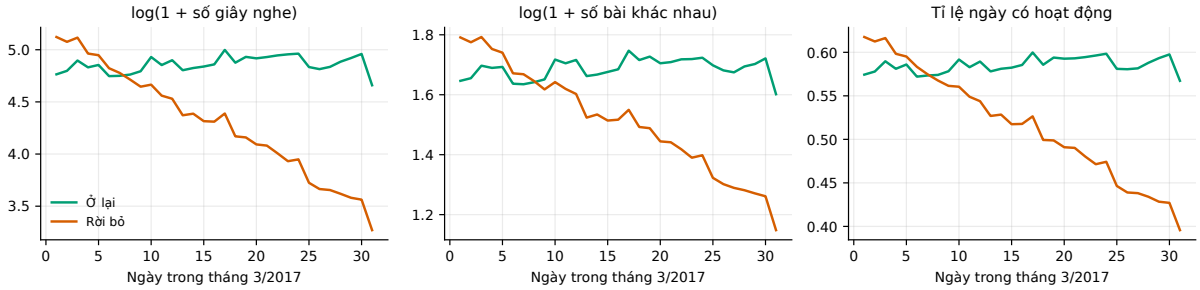
Hình 4.12. Cơ chế tạo mẫu cửa sổ trượt 30 phiên và chia theo thời gian chống rò rỉ thông tin tương lai.

Tiền xử lý chống rò rỉ và chống ngoại suy (Hình 4.12).

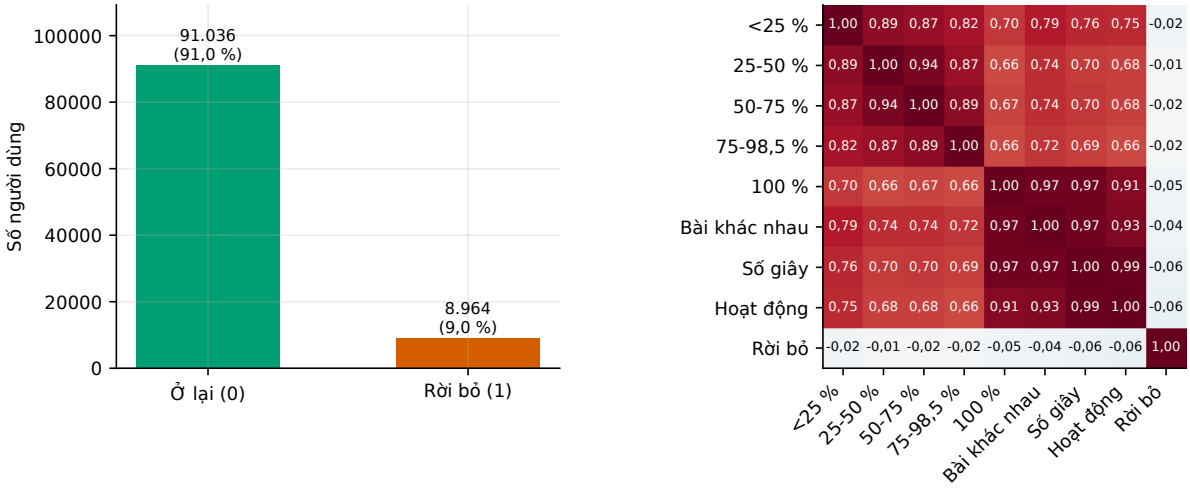
1. **Cửa sổ trượt:** cửa sổ $T = 30$ phiên, bước 1, mỗi mẫu $X_i \in \mathbb{R}^{30\times 9}$.
2. **Chia theo thời gian, không xáo trộn:** 80 % cửa sổ đầu (2.821 mẫu, đến 2023-11-16) để huấn luyện, trong đó 10 % cuối làm kiểm định; 20 % cuối (706 mẫu, từ 2023-11-17) để kiểm thử.
3. **Chuẩn hóa theo từng cửa sổ:** lần chạy đầu tiên dùng MinMaxScaler khớp trên tập huấn luyện cho R^2 âm trên tập kiểm thử, vì giá kiểm thử vượt xa mọi mức giá đã thấy và mạng buộc phải ngoại suy. Cách sửa là biểu diễn mọi cột giá trong cửa sổ bằng phần trăm chênh so với giá đóng cửa ngày cuối cửa sổ, và dự báo *lợi suất* phiên kế tiếp $r_{t+1} = (P_{t+1}/P_t - 1) \cdot 100\,\%$ thay vì mức giá; dự báo giá thu được bằng $\hat{P}_{t+1} = P_t(1 + \hat{r}_{t+1})$. Khối lượng lấy log rồi chuẩn hóa bằng thống kê của phần huấn luyện, RSI chia 100.

4.2.2 Tập dữ liệu 2: quỹ đạo hành vi người dùng KKBox

- **Nguồn:** cuộc thi WSDM – KKBox's Churn Prediction Challenge [43]: nhật ký nghe nhạc theo ngày của người dùng thật của dịch vụ nghe nhạc trực tuyến KKBox (Đài Loan) và nhãn rời bỏ sau khi hết hạn thuê bao.
- **Quy mô:** 100.000 người dùng lấy mẫu phân tầng từ tệp nhật ký 1,4 GB (giữ nguyên tỉ lệ rời bỏ), mỗi người là một chuỗi $T = 31$ ngày của tháng 3/2017 với $D = 8$ đặc trưng: số bài nghe dưới 25 %, 25 – 50 %, 50 – 75 %, 75 – 98,5 % và trọn bài; số bài khác nhau; tổng số giây nghe; cờ có hoạt động. Đặc trưng đếm và số giây lấy $\log(1 + x)$. Nhãn: 1 nếu người dùng rời bỏ, 0 nếu ở lại.
- **Chia dữ liệu:** 70/15/15 phân tầng: 70.000 huấn luyện, 15.000 kiểm định, 15.000 kiểm thử; chuẩn hóa z-score theo từng đặc trưng bằng thống kê tập huấn luyện.



Hình 4.13. Quỹ đạo trung bình theo ngày của số giây nghe, số bài khác nhau và tỉ lệ ngày hoạt động: nhóm rời bỏ đầu tháng nghe nhiều ngang hoặc hơn nhóm ở lại, rồi giảm dần về cuối tháng.



Hình 4.14. Tỉ lệ rời bỏ (8,96 %) và ma trận tương quan giữa các đặc trưng hành vi (trung bình 31 ngày) với nhãn.

Nhận xét EDA. Tỉ lệ rời bỏ là 8,96 %, mất cân bằng mạnh. Người rời bỏ có trung bình 16,01 ngày hoạt động trong tháng, so với 18,14 ngày của người ở lại (Hình 4.13). Điều đáng chú ý là *hình dạng* quỹ đạo: đầu tháng nhóm rời bỏ nghe nhiều không kém, thậm chí hơn nhóm ở lại, nhưng mức nghe giảm dần đều về cuối tháng, trong khi nhóm ở lại giữ ổn định. Tín hiệu nằm ở xu hướng theo thời gian, đúng loại thông tin mà mô hình chuỗi có thể khai thác còn đặc trưng tổng hợp cả tháng thì làm mờ đi. Tuy nhiên tương quan của từng đặc trưng với nhãn đều yếu (tỉ lệ ngày hoạt động $r = -0{,}0629$, Hình 4.14): hai nhóm chồng lấn nhau rất nhiều, vì quyết định gia hạn còn phụ thuộc giá, khuyến mãi, phương thức thanh toán, những thông tin không có trong nhật ký nghe. Đây là bài toán khó; mục tiêu thực tế là xếp hạng rủi ro tốt (ROC-AUC) chứ không phải độ chính xác tuyệt đối.

4.3 Hiện thực hóa mã nguồn trên ba nền tảng

4.3.1 Tế bào LSTM viết tay bằng NumPy

Đoạn mã 4.1 cài đặt trực tiếp sáu phương trình LSTM với thứ tự khối trọng số (i, f, g, o) giống PyTorch; lớp `LSTMRegressorScratch` xếp chồng nhiều tế bào và nạp trọng số từ mô hình PyTorch đã huấn luyện.

```python
# tl/rnn_scratch.py
class LSTMCellScratch:
    def __init__(self, input_dim, hidden_dim, seed=0):
        rng = np.random.default_rng(seed)
        k = 1 / np.sqrt(hidden_dim)
        self.H = hidden_dim
        self.W_ih = rng.uniform(-k, k, (4 * hidden_dim, input_dim))   # trọng số đầu vào của 4 khối
        self.W_hh = rng.uniform(-k, k, (4 * hidden_dim, hidden_dim))  # trọng số hồi quy của 4 khối
        self.b = np.zeros(4 * hidden_dim)

    def step(self, x_t, h_prev, c_prev):
        """x_t: [B, D], h_prev, c_prev: [B, H] -> (h_t, c_t)."""
        z = x_t @ self.W_ih.T + h_prev @ self.W_hh.T + self.b         # [B, 4H]
        H = self.H
        i = sigmoid(z[:, 0:H])                                        # cổng vào
        f = sigmoid(z[:, H:2 * H])                                    # cổng quên
        g = np.tanh(z[:, 2 * H:3 * H])                                # ứng viên ô nhớ
        o = sigmoid(z[:, 3 * H:4 * H])                                # cổng ra
        c_t = f * c_prev + i * g                                      # cập nhật cộng: đường cao tốc gradient
        h_t = o * np.tanh(c_t)
        return h_t, c_t

    def forward(self, X):
        """X: [B, T, D] -> chuỗi trạng thái ẩn [B, T, H]."""
        B, T, _ = X.shape
        h = np.zeros((B, self.H))
        c = np.zeros((B, self.H))
        out = np.empty((B, T, self.H))
        for t in range(T):
            h, c = self.step(X[:, t, :], h, c)
            out[:, t] = h
        return out
```

Đoạn mã 4.1. Tế bào LSTM và lan truyền tiến qua chuỗi viết tay bằng NumPy

Kiểm chứng. Nạp trọng số của mô hình LSTM hai tầng đã huấn luyện trên FPT vào bản NumPy và chạy trên toàn bộ 706 cửa sổ kiểm thử: đầu ra lệch PyTorch tối đa $8{,}47 \times 10^{-8}$ (sai số làm tròn giữa float32 và float64), RMSE dự báo giá 1,574 nghìn đồng so với 1,574 của PyTorch, tốc độ 0,2888 ms mỗi mẫu khi chạy theo lô. Các phương trình (4.6) – (4.11) đã được hiện thực đúng.

4.3.2 Hiện thực bằng Keras / TensorFlow

```python
# tl/rnn_keras.py
def build_keras_seq_model(cell="lstm", input_shape=(30, 9), hidden=64, num_layers=2, out="reg",
                          dropout=0.2, lr=1e-3):
    model = keras.Sequential([keras.Input(shape=input_shape)])
    for i in range(num_layers):
        rnn = RNN_LAYERS[cell](hidden, return_sequences=i < num_layers - 1)
        model.add(layers.Bidirectional(rnn) if cell == "bilstm" else rnn)
        if i < num_layers - 1:
            model.add(layers.Dropout(dropout))
    model.add(layers.Dense(32, activation="relu"))
    model.add(layers.Dropout(dropout))
    if out == "reg":
        model.add(layers.Dense(1))                        # giá đóng cửa phiên kế tiếp (thang chuẩn hóa)
        model.compile(optimizer=keras.optimizers.Adam(lr, clipnorm=1.0), loss="mse")
    else:
        model.add(layers.Dense(1, activation="sigmoid"))  # xác suất rời bỏ
        model.compile(optimizer=keras.optimizers.Adam(lr, clipnorm=1.0), loss="binary_crossentropy")
    return model
```

Đoạn mã 4.2. Mô hình hồi quy many-to-one bằng Keras cho cả hồi quy và phân loại

4.3.3 Hiện thực bằng PyTorch

Một lớp `SeqModel` phục vụ cả bốn loại tế bào; vòng huấn luyện cắt tỉa chuẩn gradient trước mỗi bước cập nhật.

```python
# tl/rnn_torch.py
class SeqModel(nn.Module):
    def __init__(self, cell="lstm", input_dim=9, hidden=64, num_layers=2, out_dim=1, dropout=0.2):
        super().__init__()
        rnn_cls = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU, "bilstm": nn.LSTM}[cell]
        kw = {"nonlinearity": "tanh"} if cell == "rnn" else {}
        self.rnn = rnn_cls(input_dim, hidden, num_layers=num_layers, batch_first=True, dropout=dropout,
                           bidirectional=cell == "bilstm", **kw)
        d = hidden * (2 if cell == "bilstm" else 1)
        self.fc = nn.Sequential(nn.Linear(d, 32), nn.ReLU(), nn.Dropout(dropout), nn.Linear(32, out_dim))

    def forward(self, x):                       # x: [B, T, D]
        out, _ = self.rnn(x)                     # out: [B, T, H] (BiLSTM: [B, T, 2H])
        return self.fc(out[:, -1, :])            # many-to-one: chỉ dùng bước thời gian cuối
```

Đoạn mã 4.3. Mô hình hồi quy tổng quát bằng PyTorch: Simple RNN, LSTM, GRU, BiLSTM

```python
# tl/rnn_torch.py
def train_epoch(model, X, y, criterion, opt, batch, max_grad_norm=1.0):
    model.train()
    order = torch.randperm(len(X), device=DEVICE)
    total = 0.0
    for s in range(0, len(X), batch):
        idx = order[s:s + batch]
        opt.zero_grad()
        loss = criterion(model(X[idx]), y[idx])
        loss.backward()
        # Cắt chuẩn gradient: giữ hướng, giới hạn độ dài bước cập nhật khi gradient bùng nổ qua BPTT
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=max_grad_norm)
        opt.step()
        total += loss.item() * len(idx)
    return total / len(X)
```

Đoạn mã 4.4. Một epoch huấn luyện PyTorch có cắt tỉa chuẩn gradient

Cấu hình chung của cả hai nền tảng: 2 tầng hồi quy 64 đơn vị, dropout 0,2, Dense 32; Adam 10^{-3}, cắt tỉa $\theta = 1$. Chứng khoán: lô 64, tối đa 100 epoch, dừng sớm sau 15 epoch không cải thiện, mất mát MSE trên lợi suất. KKBox: lô 512, tối đa 25 epoch, dừng sớm sau 5 epoch, mất mát entropy chéo nhị phân; ngưỡng phân loại chọn trên tập kiểm định để F1 lớn nhất.

4.4 Đánh giá và đối chuẩn các kiến trúc RNN trên hai tập dữ liệu

Phương pháp. Bốn kiến trúc được huấn luyện từ đầu trên cùng cách chia dữ liệu. Ngoài các chỉ số chuẩn, bài toán chứng khoán có thêm mốc tham chiếu *giữ nguyên giá* ($\hat{P}_{t+1} = P_t$, tương đương dự báo lợi suất bằng 0) và hai thước đo khó hơn R^2 trên giá: R^2 trên *lợi suất* và tỉ lệ đoán đúng chiều tăng/giảm.

4.4.1 Dự báo chuỗi thời gian cổ phiếu FPT (hồi quy)

Bảng 4.2. Đối chuẩn dự báo giá đóng cửa FPT trên tập kiểm thử (2023-11-17 – 2026-09-30).

Mô hình (PyTorch)	RMSE	MAE	R^2 giá	R^2 lợi suất (%)	Đúng chiều (%)	Tham số	Trễ CPU (ms)	Dung lượng
Simple RNN	1,575	1,116	0,9903	−0,67	46,2	15.233	0,775	62,9 KB
LSTM	1,574	1,121	0,9903	−0,68	43,3	54.593	0,454	216,7 KB
GRU	1,574	1,120	0,9903	−0,81	44,1	41.473	1,961	165,4 KB
BiLSTM	1,605	1,144	0,9899	−5,89	46,3	141.889	0,815	558,5 KB
Giữ nguyên giá ($\hat{P}_{t+1} = P_t$)	**1,568**	1,104	0,9904	0,00	–	0	–	–

RMSE, MAE tính bằng nghìn đồng trên giá đóng cửa của 706 phiên kiểm thử; chữ đậm: nhỏ nhất.

4.4.2 Phân loại chuỗi hành vi KKBox

Bảng 4.3. Đối chuẩn phân loại người dùng rời bỏ trên tập kiểm thử KKBox.

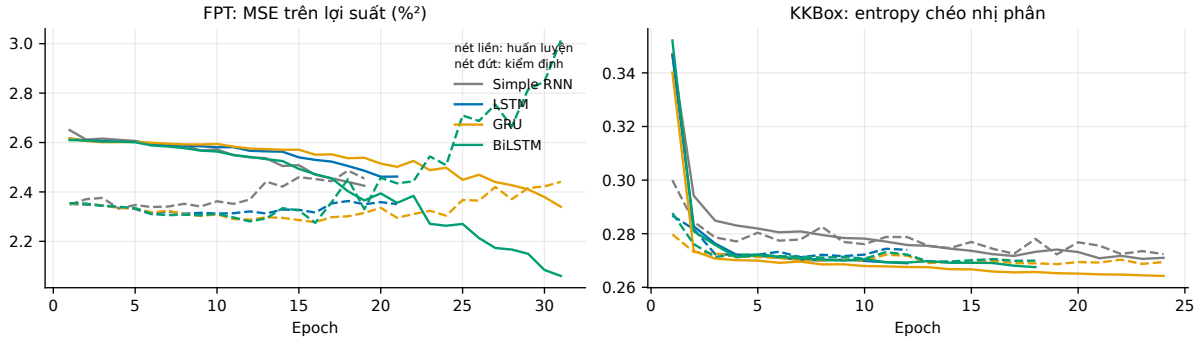
Mô hình (PyTorch)	Acc (%)	Precision	Recall	F1	ROC-AUC	Ngưỡng	Tham số	Trễ CPU (ms)	Dung lượng
Simple RNN	88,45	0,3425	0,3147	0,3280	0,6713	0,13	15.169	0,805	62,7 KB
LSTM	89,28	0,3764	0,2991	0,3333	0,6763	0,20	54.337	0,452	215,7 KB
GRU	88,87	0,3631	**0,3207**	0,3406	**0,6846**	0,18	41.281	1,656	164,7 KB
BiLSTM	**90,23**	**0,4319**	0,2857	**0,3439**	0,6760	0,22	141.377	0,922	556,5 KB

Precision, Recall, F1 cho lớp rời bỏ tại ngưỡng chọn trên tập kiểm định; chữ đậm: tốt nhất cột.

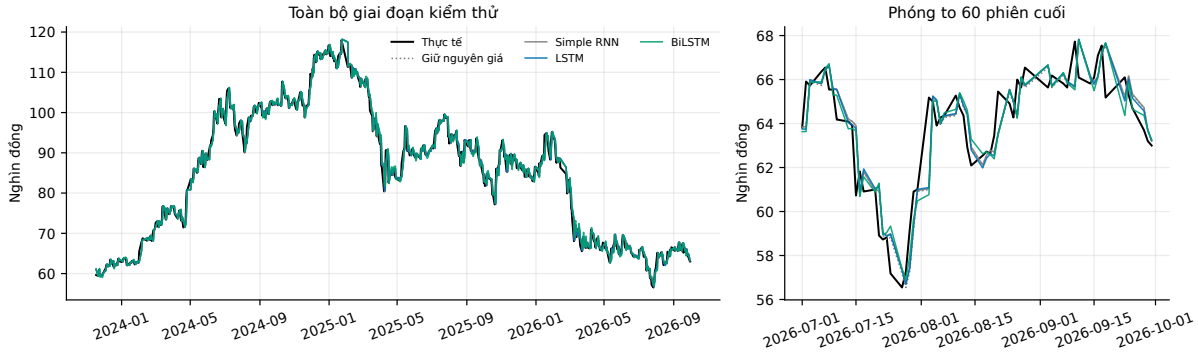
Bảng 4.4. Đối chuẩn PyTorch và Keras với cùng cấu hình mạng.

Kiến trúc	Nền tảng	RMSE FPT	F1 KKBox	AUC KKBox	Tham số (FPT)	Dung lượng	Trễ CPU (ms)	Huấn luyện (s)
Simple RNN	PyTorch	1,575	0,3280	0,6713	15.233	62,9 KB	0,775	3,3 / 13,1
	Keras	1,591	0,3246	0,6645	15.105	218,0 KB	1,487	27,7 / 100
LSTM	PyTorch	1,574	0,3333	0,6763	54.593	216,7 KB	0,454	2,2 / 19,1
	Keras	1,593	0,3357	0,6760	54.081	674,8 KB	1,880	122 / 794
GRU	PyTorch	1,574	0,3406	0,6846	41.473	165,4 KB	1,961	3,2 / 45,2
	Keras	1,575	0,3445	0,6830	41.473	527,1 KB	2,189	36,6 / 1.452
BiLSTM	PyTorch	1,605	0,3439	0,6760	141.889	558,5 KB	0,815	4,4 / 61,6
	Keras	1,572	0,3411	0,6799	140.865	1,7 MB	2,667	29,0 / 4.156

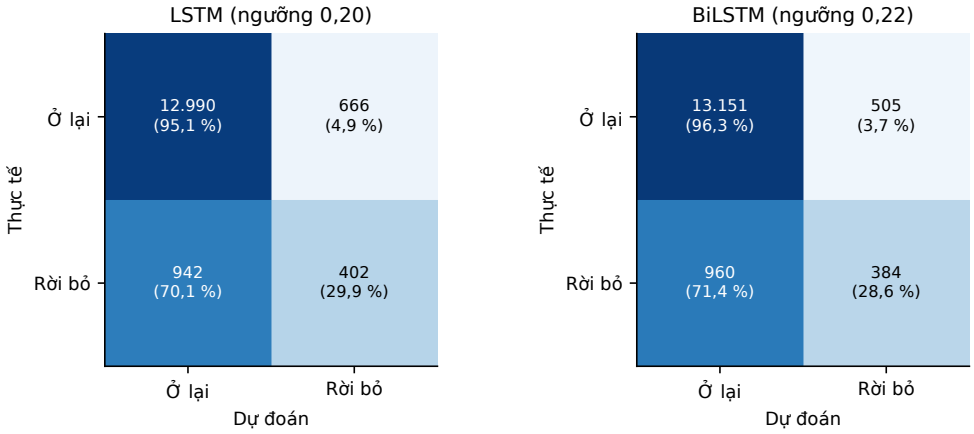
Độ trễ: mô hình FPT, lô 1, CPU 4 luồng cho cả hai nền tảng. Thời gian huấn luyện FPT / KKBox: PyTorch trên GPU, Keras trên CPU.



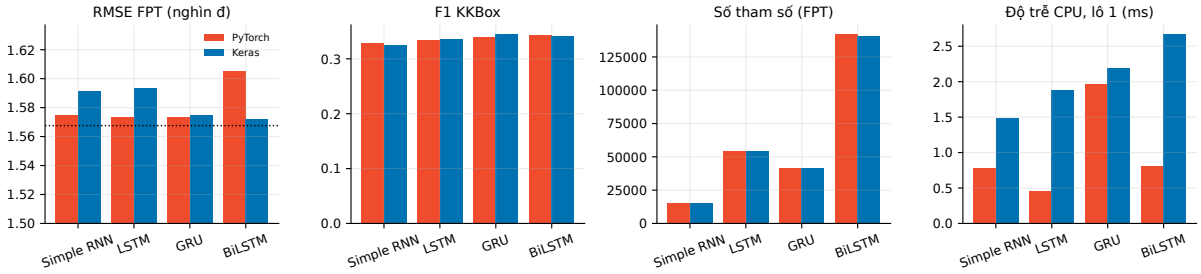
Hình 4.15. Đường cong mất mát huấn luyện (nét liền) và kiểm định (nét đứt) của bốn kiến trúc trên hai tập dữ liệu.



Hình 4.16. Giá thực tế và dự báo trên tập kiểm thử FPT, kèm mốc "giữ nguyên giá"; bên phải phóng to 60 phiên cuối.



Hình 4.17. Ma trận nhầm lẫn của LSTM và BiLSTM trên tập kiểm thử KKBox.



Hình 4.18. So sánh PyTorch và Keras: RMSE trên FPT (đường chấm là mốc giữ nguyên giá), F1 trên KKBox, số tham số và độ trễ CPU.

Thảo luận kết quả.

- **R^2 cao trên giá không chứng minh năng lực dự báo.** Cả bốn mạng đạt $R^2 \approx 0{,}9903$ trên giá FPT, nhưng mốc giữ nguyên giá cũng đạt $R^2 = 0{,}9904$ với RMSE 1,568 nghìn đồng, ngang hoặc tốt hơn mọi mạng (LSTM: 1,574). R^2 trên lợi suất của các mạng xấp xỉ 0 và tỉ lệ đoán đúng chiều chỉ 43 – 48 %, không hơn tung đồng xu. Các đường dự báo trùng gần như hoàn toàn với đường thực tế dịch đi một phiên (Hình 4.16): mạng học được rằng dự báo tốt nhất cho lợi suất ngày mai là xấp xỉ 0. Điều này khớp với EDA (tự tương quan lợi suất gần 0) và với giả thuyết thị trường hiệu quả dạng yếu. Kết luận đúng là: với dữ liệu giá – khối lượng thuần túy, không kiến trúc hồi quy nào khai thác được thông tin vượt mốc ngây thơ; muốn có lợi thế cần thêm dữ liệu ngoài chuỗi giá (tin tức, báo cáo tài chính, dòng tiền).
- **Khác biệt giữa các tế bào trên chuỗi giá** vì vậy rất nhỏ (Bảng 4.2); Simple RNN với cửa sổ 30 phiên không bị bất lợi rõ rệt vì tín hiệu dài hạn vốn không có. BiLSTM, nhiều tham số nhất, có RMSE kém nhất trên PyTorch: thêm năng lực biểu diễn chỉ thêm khả năng khớp nhiễu.
- **KKBox: cơ chế cổng có ích nhưng giới hạn nằm ở dữ liệu.** Các tế bào có cổng (GRU, LSTM, BiLSTM) đạt ROC-AUC và F1 cao hơn Simple RNN (Bảng 4.3), khớp với lý thuyết rằng cổng giúp tổng hợp thông tin qua 31 bước. Nhưng mức tuyệt đối thấp (ROC-AUC dưới 0,7, F1 lớp rời bỏ quanh 0,33 – 0,35): chỉ từ nhật ký nghe nhạc, người ở lại và người rời bỏ chồng lấn mạnh, đúng như EDA đã cảnh báo. Ma trận nhầm lẫn (Hình 4.17) cho thấy ở ngưỡng tối ưu F1, mô hình chỉ bắt được khoảng một phần ba số người rời bỏ (Recall LSTM 0,2991) mà vẫn kèm nhiều báo động giả; nếu dùng để giữ chân khách hàng, danh sách xếp hạng theo xác suất hữu ích hơn một nhãn cứng.
- **PyTorch và Keras** cho độ chính xác cùng mức với cùng cấu hình (Bảng 4.4, Hình 4.18); khác biệt nhỏ đến từ khởi tạo ngẫu nhiên và thứ tự lô. Số tham số GRU trùng khít (41.473) vì Keras mặc định `reset_after=True` cũng dùng hai vector bias như PyTorch; với Simple RNN và LSTM, PyTorch nhiều hơn vài trăm tham số do giữ hai bias riêng b_{ih}, b_{hh} (tương đương về toán học, chỉ khác cách tham số hóa). PyTorch trên GPU huấn luyện nhanh hơn Keras trên CPU nhiều lần; khi suy luận một mẫu trên CPU, độ trễ hai nền tảng cùng bậc mili-giây.

4.5 Đóng gói triển khai Web App tương tác thời gian thực

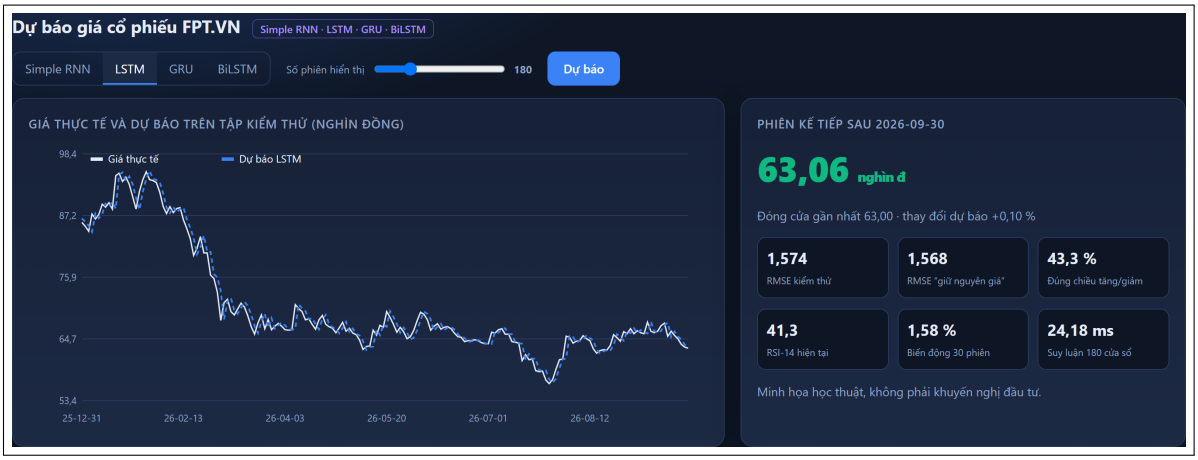
Hai phân hệ cuối của Web App (mục 2.6) dùng các mô hình PyTorch của chương này:

1. **Dự báo giá cổ phiếu FPT:** người dùng chọn kiến trúc và số phiên hiển thị; hệ thống vẽ giá thực tế và dự báo trên giai đoạn kiểm thử, dự báo giá phiên kế tiếp sau ngày dữ liệu cuối cùng, và luôn hiển thị RMSE của mốc "giữ nguyên giá" cạnh RMSE của mô hình để người dùng không hiểu nhầm năng lực dự báo (Hình 4.19).
2. **Dự đoán người dùng rời bỏ:** lấy một người dùng trong tập kiểm thử (ngẫu nhiên, hoặc chọn nhóm), vẽ quỹ đạo 31 ngày và trả xác suất rời bỏ của cả bốn kiến trúc so với ngưỡng riêng của từng mô hình (Hình 4.20).

```python
# app/app.py
@app.post("/api/churn")
def api_churn():
    f = request.get_json()
    rng = np.random.default_rng(int(f.get("seed", 0)))
    want = f.get("group", "any")
    yt = CHURN["y_test"]
    pool = np.flatnonzero(yt == 1) if want == "churn" else np.flatnonzero(yt == 0) if want == "loyal" else
    ↪  np.arange(len(yt))
    i = int(rng.choice(pool))
    x = torch.as_tensor(CHURN["X_test"][i:i + 1])

    def run():
        with torch.no_grad():
            return {c: torch.sigmoid(m(x)).item() for c, m in churn_models.items()}
    probs, ms = timed(run)
    xr = CHURN["X_test"][i] * CHURN["sd"] + CHURN["mu"]          # về lại thang log1p ban đầu
    return jsonify(index=i, truth=int(yt[i]), probs=probs, thr={c: CH4[f"churn/{c}"]["threshold"] for c in CELLS},
                   secs=np.expm1(xr[:, 6]).round().tolist(), unq=np.expm1(xr[:, 5]).round().tolist(),
                   active=xr[:, 7].round().tolist(), labels=LABELS, ms=ms)
```

Đoạn mã 4.5. Endpoint dự đoán rời bỏ: chạy bốn mô hình hồi quy trên cùng một chuỗi 31 ngày



Hình 4.19. Phân hệ dự báo giá cổ phiếu FPT: đường giá thực tế và dự báo của LSTM, dự báo phiên kế tiếp, so sánh với mốc giữ nguyên giá và các chỉ báo hiện tại.



Hình 4.20. Phân hệ dự đoán khách hàng rời bỏ: quỹ đạo nghe nhạc 31 ngày và xác suất rời bỏ của bốn kiến trúc.

Mỗi yêu cầu dự báo (bốn mô hình hoặc hàng trăm cửa sổ) được xử lý trong vài chục mili-giây trên CPU.

Kết Luận

Kết quả đạt được

Tiểu luận đã đi trọn một vòng từ lịch sử, lý thuyết, cài đặt, thực nghiệm đến triển khai cho ba họ mô hình trên bảy tập dữ liệu thật:

- **Lịch sử AI** được hệ thống hóa qua mười sáu mốc, cho thấy mỗi mô hình hiện đại là lời đáp cho một thất bại cụ thể trước đó: lan truyền ngược cho giới hạn XOR, học máy thống kê cho tính giòn của hệ chuyên gia, LSTM cho triệt tiêu đạo hàm, ResNet cho thoái hóa mạng sâu.
- **Học máy cơ bản và MLP:** trên dữ liệu y tế mất cân bằng, MLP đạt F1 lớp dương 0,8047, cao nhất trong bảy mô hình; trên giá nhà Việt Nam, Random Forest vẫn nhỉnh nhất ($R^2 = 0{,}6258$); trên văn bản, MLP chính xác nhất nhưng quá khớp nhanh. MLP cài đặt bằng NumPy thuần đạt kết quả ngang Keras và PyTorch.
- **CNN:** ResNet-18 huấn luyện từ đầu đạt 96,93 % trên SVHN và 98,26 % trên EuroSAT; MobileNetV2 đạt độ chính xác sát nút với số tham số ít hơn khoảng 5 lần, nhưng không nhanh hơn trên GPU ở lô 1. Mạng CNN chạy bằng các hàm NumPy viết tay cho cùng dự đoán với PyTorch.
- **RNN:** trên KKBox, các tế bào có cổng vượt Simple RNN; trên giá FPT, cả bốn kiến trúc chỉ ngang mốc "giữ nguyên giá" dù R^2 trên giá đạt 0,9903. LSTM viết tay bằng NumPy tái tạo đúng đầu ra của PyTorch.
- **Triển khai:** một Web App Flask sáu phân hệ phục vụ mọi mô hình trên CPU với thời gian phản hồi vài đến vài chục mili-giây, có bản đồ CAM giải thích dự đoán ảnh.

Bài học rút ra

1. **Chọn mô hình theo dạng dữ liệu, không theo độ "hiện đại".** Random Forest vẫn là đối thủ khó vượt trên dữ liệu bảng; CNN thắng trên ảnh vì cấu trúc không gian; RNN chỉ có lợi khi chuỗi thật sự chứa tín hiệu theo thời gian.
2. **Một con số đẹp cần một mốc so sánh.** $R^2 = 0{,}99$ trên giá cổ phiếu hay Accuracy 91,5 % trên dữ liệu tiểu đường đều đạt được bởi mô hình ngây thơ. Mọi đánh giá trong báo cáo vì vậy luôn đi kèm mốc tham chiếu và chỉ số phù hợp với bài toán (F1 lớp hiếm, R^2 trên lợi suất).
3. **Tiền xử lý quyết định nhiều hơn kiến trúc.** Chuẩn hóa theo mức giá khiến mọi RNN thất bại khi giá vượt vùng huấn luyện; chuẩn hóa theo từng cửa sổ sửa được ngay. Ngưỡng phân loại thay đổi F1 nhiều hơn việc đổi mô hình.
4. **FLOPs không phải độ trễ.** MobileNetV2 rẻ hơn nhiều lần về phép tính nhưng chậm hơn ResNet-18 trên GPU ở lô 1; hiệu năng thực phụ thuộc phần cứng đích.
5. **Viết tay để hiểu, dùng thư viện để chạy.** Các bản NumPy giúp kiểm chứng từng công thức; PyTorch trên GPU nhanh hơn nhiều bậc khi huấn luyện thật.

Hướng phát triển

Các hướng tiếp theo bám sát lộ trình của học phần: thay RNN bằng Transformer cho chuỗi dài; bổ sung dữ liệu ngoài chuỗi giá (tin tức, báo cáo tài chính) và thông tin giao dịch thuê bao cho bài toán rời bỏ; dùng học chuyển giao với trọng số tiền huấn luyện cho ảnh; và kết nối các mô hình trong báo cáo với đồ thị tri thức, RAG và hệ đa tác tử ở các chương sau của học phần.

Tài liệu tham khảo

[1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[2] Alan M. Turing. “Computing Machinery and Intelligence”. In: *Mind* 59.236 (1950), pp. 433–460.

[3] Warren S. McCulloch and Walter Pitts. “A Logical Calculus of the Ideas Immanent in Nervous Activity”. In: *The Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133.

[4] Frank Rosenblatt. “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain”. In: *Psychological Review* 65.6 (1958), pp. 386–408.

[5] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.

[6] James Lighthill. *Artificial Intelligence: A General Survey*. Tech. rep. London: Science Research Council, 1973.

[7] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning Representations by Back-Propagating Errors”. In: *Nature* 323.6088 (1986), pp. 533–536.

[8] Kunihiko Fukushima. “Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position”. In: *Biological Cybernetics* 36.4 (1980), pp. 193–202.

[9] Corinna Cortes and Vladimir Vapnik. “Support-Vector Networks”. In: *Machine Learning* 20.3 (1995), pp. 273–297.

[10] Yoav Freund and Robert E. Schapire. “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting”. In: *Journal of Computer and System Sciences* 55.1 (1997), pp. 119–139.

[11] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32.

[12] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[13] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems 25*. 2012, pp. 1097–1105.

[14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15 (2014), pp. 1929–1958.

[15] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *International Conference on Learning Representations (ICLR)*. 2015.

[16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep Residual Learning for Image Recognition”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[17] Andrew G. Howard et al. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications”. In: *arXiv preprint arXiv:1704.04861* (2017).

[18] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 4510–4520.

[19] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. “Learning Long-Term Dependencies with Gradient Descent is Difficult”. In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.

[20] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[21] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation”. In: *Proceedings of EMNLP 2014*. 2014, pp. 1724–1734.

[22] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems 30*. 2017, pp. 5998–6008.

[23] Jared Kaplan et al. “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361* (2020).

[24] Jordan Hoffmann et al. “Training Compute-Optimal Large Language Models”. In: *Advances in Neural Information Processing Systems 35*. 2022.

[25] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33*. 2020, pp. 9459–9474.

[26] Sebastian Raschka. *Python Machine Learning*. Packt Publishing, 2015.

[27] Mohammed Mustafa. *Diabetes Prediction Dataset*. Kaggle. 2023. URL: https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset.

[28] Huu Tri Nguyen. *Vietnam Housing Dataset*. Kaggle (tập dữ liệu công khai, đã dùng ở Assignment 01). 2024.

[29] Nick Brooks. *Women’s E-Commerce Clothing Reviews*. Kaggle. 2018. URL: https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews.

[30] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 1026–1034.

[32] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015.

[33] Min Lin, Qiang Chen, and Shuicheng Yan. “Network In Network”. In: *arXiv preprint arXiv:1312.4400* (2013).

[34] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. “Reading Digits in Natural Images with Unsupervised Feature Learning”. In: *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*. 2011.

[35] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. “EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification”. In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 12.7 (2019), pp. 2217–2226.

[36] Sergey Ioffe and Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *Proceedings of ICML 2015*. 2015, pp. 448–456.

[37] Leslie N. Smith and Nicholay Topin. “Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates”. In: *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*. Vol. 11006. 2019.

[38] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. “Learning Deep Features for Discriminative Localization”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 2921–2929.

[39] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the Difficulty of Training Recurrent Neural Networks”. In: *Proceedings of ICML 2013*. 2013, pp. 1310–1318.

[40] Mike Schuster and Kuldip K. Paliwal. “Bidirectional Recurrent Neural Networks”. In: *IEEE Transactions on Signal Processing* 45.11 (1997), pp. 2673–2681.

[41] Yarin Gal and Zoubin Ghahramani. “A Theoretically Grounded Application of Dropout in Recurrent Neural Networks”. In: *Advances in Neural Information Processing Systems 29*. 2016, pp. 1019–1027.

[42] Ran Aroussi. *yfinance: Download Market Data from Yahoo! Finance’s API*. Python package, version 1.7. 2026. URL: https://github.com/ranaroussi/yfinance.

[43] KKBOX. *WSDM - KKBox’s Churn Prediction Challenge*. Kaggle competition. 2017. URL: https://www.kaggle.com/competitions/kkbox-churn-prediction-challenge.